



ACITM

Java Server Framework Reference Guide

© 2005 by ACI Worldwide Inc. All rights reserved.

All information contained in this documentation is confidential and proprietary to ACI Worldwide Inc., and has been delivered for use pursuant to license. No part of this documentation may be photocopied, electronically transferred, modified, or reproduced in any manner that is contrary to license without the prior written consent of ACI Worldwide Inc.

Access Control Server, ACI Automated Case Management System, ACI Card Management System, ACI Claims Manager, ACI Commerce Gateway, ACI Debit Card System, ACI e-Courier, ACI Host Link, ACI Merchant Accounting System, ACI Payments Manager, ACI Proactive Risk Manager, ACI Smart Chip Manager, BASE24, BASE24-atm, BASE24-card, BASE24-check auth, BASE24-configuration manager, BASE24-es, BASE24-frequent shopper, BASE24-infobase, BASE24-pos, BASE24-refunds, BASE24-remote banking, BASE24-teller, NET24, and WINPAY24 are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Contents

Preface.	vii
Section 1	
Introduction.	1-1
Java Server Framework	1-2
Application Layers	1-3
Adapters	1-4
Components	1-4
Services	1-5
Framework	1-5
Class Loading	1-6
Application Programming Interfaces (APIs) and Third-Party JARs	1-6
Java DataBase Connectivity (JDBC)	1-7
Java Message Service (JMS)	1-7
Java Servlets	1-7
Enterprise JavaBeans (EJBs)	1-7
Java Cryptography Extension (JCE)	1-8
Java API for XML Binding (JAXB)	1-9
Log4j	1-9
Java Document Object Model (JDOM)	1-10
Java Authentication and Authorization Service (JAAS)	1-10
Java API for XML Processing (JAXP)	1-11
Commons-lang	1-11
Commons-logging	1-11
Concurrent	1-11
Javadoc	1-12
Java Server Properties	1-13
System Properties	1-13
Configuration Properties	1-13
Application Configuration Properties	1-14

Introduction *continued*

Graphical User Interface	1-15
Java Server Management	1-15
Java Server Administration.	1-16

Section 2

Subsystems.	2-1
Database Subsystem	2-2
Code Generator	2-2
Transaction Integrity.	2-2
Auditing User Actions	2-2
Caching.	2-4
Drivers and APIs.	2-4
Alternate Data Sources	2-4
Pooling and Locking.	2-5
Message Subsystem	2-6
Clustering Subsystem	2-7
Encryption Subsystem	2-8
Logging Subsystem.	2-9
Tracing Subsystem	2-10
Message Transformation Subsystem	2-11
Directory Subsystem.	2-12
Cache Management Subsystem	2-13
Configuration and Instrumentation Subsystem.	2-14
Properties Files	2-14
XML Documents	2-16
Data Sources	2-17
Alternate Data Source.	2-17
Event	2-17
Event Documentation	2-17
External Connection	2-18
Listener	2-18
Process	2-18

Section 3

Management	3-1
Alternate Data Source Configuration	3-2
Field Descriptions	3-3
Example Configuration	3-4
Application Configuration	3-6
External Connections Configuration	3-7
General Configuration Options	3-8
Protocol Configuration Options	3-9
TCP/IP	3-9
HTTP	3-10
JMS-ASYNC	3-10
JMS-SYNC	3-11
EJB	3-11
SMTP	3-11
PATHSEND	3-11
PATHSEND Protocol Configuration	3-12
Listener Configuration	3-13
General Configuration Options	3-14
Protocol Configuration Options	3-14
TCP/IP	3-15
JMS External	3-15
JMS Internal	3-16
Process Configuration	3-17

Section 4

Administration	4-1
Cache Administration	4-2
TCP/IP Socket Administration	4-3
Property Administration	4-5
Database Administration	4-6
System Monitoring	4-8
Event Administration	4-9
Event Message Documentation	4-13
Additional References	4-13

Administration *continued*

Trace Administration	4-14
Normal Tracing.....	4-14
JDAL Tracing.....	4-15
Trace Output Configuration	4-16

Section 5

Configuration Properties	5-1
System-Level Properties	5-2
Required System Properties	5-2
System Property Configuration	5-3
Non-Web Application.....	5-3
Web Application.....	5-4
Optional System Properties	5-5
Configuration Properties.....	5-7
Password Encryption Utility.....	5-7
Application Configuration Properties.....	5-10
JSF Properties	5-11

Appendix A

BASE24-es User Interface	A-1
Third-Party JARs	A-2
System Properties	A-3
Application Properties	A-5
Sample Config.properties File	A-20

Appendix B

BASE24-es ATM Device Handler and IFX Manager Processes.....	B-1
Properties	B-2
Sample Config.properties File	B-25

Index.....	Index-1
-------------------	----------------

Preface

ACI provides server management and administration facilities for Java servers built using ACI's Java Server Framework (JSF). This user guide describes how to effectively use these facilities to configure and administer Java server processes in ACI products that use JSF-based Java server processes. It includes an overview of the JSF and its data sources and describes the user interface windows and properties used to configure and administer JSF-based Java server processes.

Note: Because the look and feel of the user interface employed to configure and administer JSF applications can differ across ACI applications, no window or screen shots are provided in this user guide. The fields displayed on these windows or screens are the same across all JSF-based applications. Refer to your application-specific documentation or ACI Online Help for guidance in accessing JSF configuration and administration windows or screens.

Audience

This user guide is intended for management and operations personnel responsible for configuring and administering BASE24-es Java server processes built on the JSF. This guide contains both generic JSF information as well as JSF information that is specific to the following BASE24-es applications:

- ACI desktop user interface (for BASE24-es and BASE24 TSS customers) (JSF version 04.4 or greater)
- BASE24-es ATM Device Handler processes (JSF version 04.4 or greater)

Prerequisites

Readers of this user guide should be familiar with basic Java terminology and the user interfaces on which ACI JSF-based applications are deployed. Familiarity or experience with Java programming and Java operating systems is recommended.

Additional Documentation

The ACI documentation set is arranged so that each ACI manual or guide presents a topic or group of related topics in detail. When one ACI manual or guide presents a topic that has already been covered in detail in another ACI manual or guide, the topic is summarized and the reader is directed to the other manual or guide for additional information. Information has been arranged in this manner to be more efficient for readers who do not need the additional detail and at the same time provide the source for readers who require the additional information. This guide contains references to the following ACI publications:

- The ***ACI Desktop User Interface Manual*** provides descriptions of task IDs associated with ACI desktop user interface windows as well as detailed information on auditing user actions.
- The ***BASE24-es Event Management User Guide*** describes the fields on the Integrated Server Event Configuration and Management window and information on configuring event messages.
- The ***BASE24-es ATM Acquirer Management Guide*** provides detailed information on additional properties files and properties used by BASE24-es ATM Device Handler processes. Some of the properties used by BASE24-es ATM Device Handler processes are defined in ATM Bundle and device property files.

This guide references Sun's ***Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide*** for more information on JSF properties used to initialize the Java Secure Socket Extension (JSSE) environment.

This guide references the following Public-Key Cryptography Standards specifications produced by RSA Laboratories:

- The Public-Key Cryptography Standards (PKCS) #12 specification, ***Personal Information Exchange Syntax Standard*** describes Version 1 and Version 3 X.509 certificates and files.
- The Public-Key Cryptography Standards (PKCS) #7 specification, ***Cryptographic Message Syntax Standard*** describes Secure MIME (S/MIME) and Cryptographic Message Syntax (CMS).

This guide references the book, ***Concurrent Programming in Java***, by Doug Lea for the rationale and applications of several of the utility classes provided in the third-party JAR, `concurrent.jar`, which is used by the JSF.

Software

This guide documents standard processing as of its publication date. Software that is not current and custom software modifications (CSMs) may result in processing that differs from the material presented in this guide. The customer is responsible for identifying and noting these changes.

Guide Summary

The following is a summary of the contents of this guide.

Section 1, “Introduction,” describes the ACI Java Server Framework, and introduces you to the database, management, and administration facilities provided in the framework. It also introduces you to the properties used to configure Java server processes running in the framework.

Section 2, “Subsystems,” describes each of the subsystems provided in the framework layer of the Java Server Framework as well as the APIs and third-party JARs used by these subsystems.

Section 3, “Management,” shows you how to manage the configuration for Java server processes. This includes managing the configuration of alternate data sources, application-level properties, connections to and from external entities, and process relationships in clustered configurations for broadcasting messages.

Section 4, “Administration,” describes how to administer cache, TCP/IP sockets, configuration properties, and database usage for ACI JSF-based applications. It also describes how to monitor system usage and control event and trace message logging.

Section 5, “Configuration Properties,” explains the three levels of Java properties in the JSF—system, configuration, and application. It also describes properties common to the JSF, and an ACI utility you can use to encrypt the database user password in the config.properties file.

Appendix A, “BASE24-es User Interface,” describes the JSF properties specific to the BASE24-es Java User Interface servlets. It also provides a sample config.properties file for the BASE24-es Java User Interface servlets.

Appendix B, “BASE24-es ATM Device Handler Processes,” describes the JSF properties specific to BASE24-es ATM Device Handler processes. It also provides a sample config.properties file for BASE24-es ATM Device Handler processes.

Publication Identification

Three entries appearing at the bottom of each page uniquely identify this ACI publication. The publication number (for example, CF-DA000-02-ES for the ***ACI Java Server Framework Reference Guide***) appears on every page to assist readers in identifying the guide from which a page of information was printed. The publication date (for example, 05/2005 for May 2005) indicates the issue of the guide. The software release information (for example, R1.0v05.2 for release 1.0, version 05.2) specifies the software that the guide describes. This information matches the document information on the copyright page of the guide.

Section 1

Introduction

This section describes the ACI Java Server Framework, and introduces you to the database, and management and administration facilities provided in the framework. It also introduces you to the properties used to configure Java server processes running in the framework.

Java Server Framework

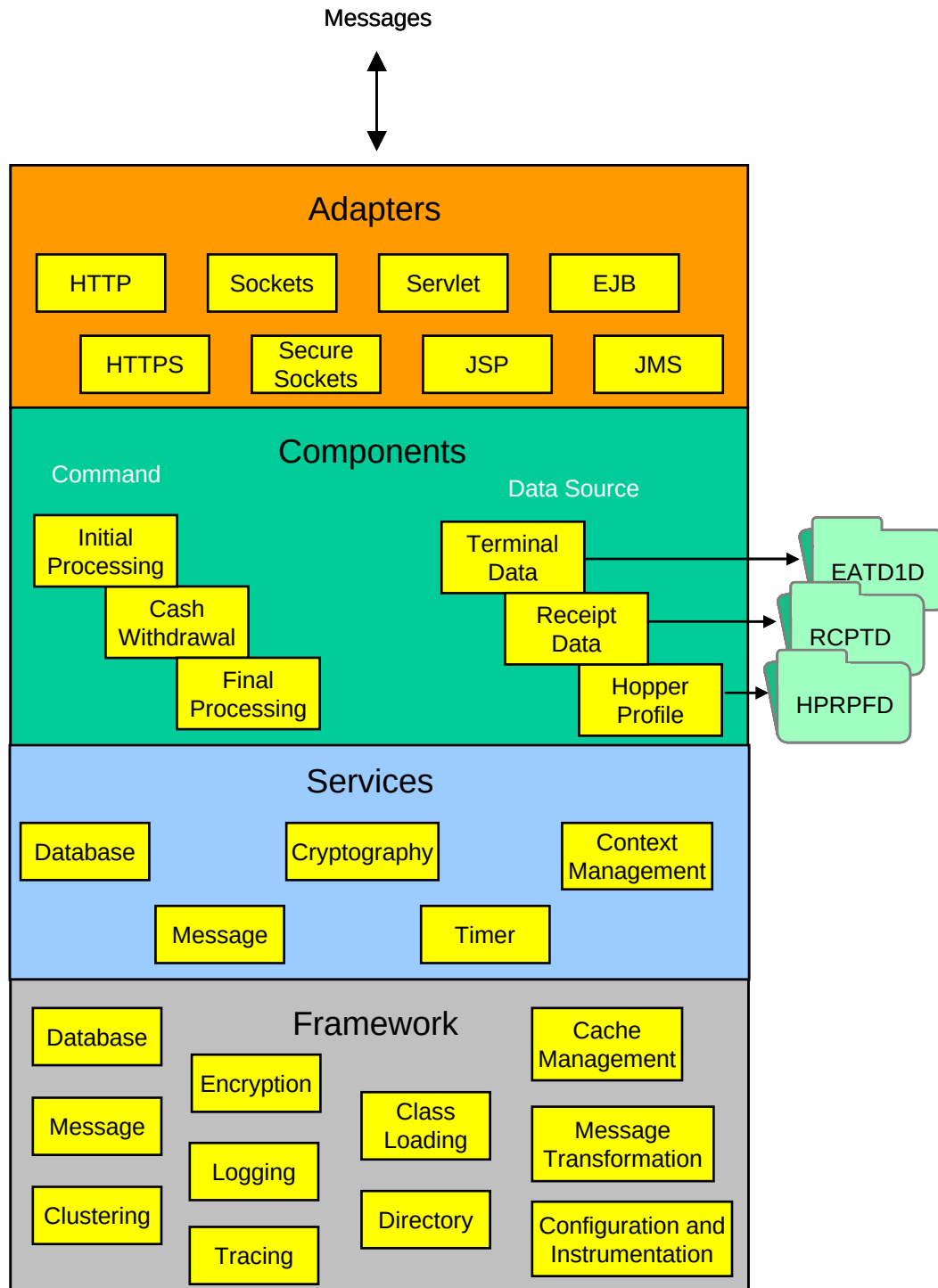
Java is an object-oriented programming language that uses a set of public application programming interfaces (APIs) to access operating system resources or other applications.

The ACI Java Server Framework allows you to manage and administer several ACI application products written in Java across multiple platforms using common management and administration functions built into ACI products. Java is an application-independent language that allows applications to be insulated from the different operating systems on various hardware platforms. ACI has deployed ACI Java Server Framework applications on the HP NSK, IBM Zseries (USS/CICS), Sun Solaris, IBM PSeries/AIX and Windows NT platforms, however, not all platforms are supported for all JSF-based applications. Check the application-specific system requirements to determine whether a platform is supported.

A benefit from writing Java-based applications is that operating system manufacturers and third party organizations such as HP, IBM, and Sun Microsystems, build the OS-specific implementation free of charge to allow Java applications to run on their servers. By writing to the Java API, the ACI applications will run, without change and with only one version of the source code, on each platform that provides a Java Virtual Machine (JVM). The Java Server Framework uses the JVM to eliminate the need for platform-specific code.

Application Layers

The ACI Java Server Framework is comprised of four application layers—adapters, components, services, and framework—as shown in the following illustration. Each of these layers is described on the following pages.



Adapters

Adapters provide a lightweight insulation layer for acquiring work from different mediums without impacting the underlying application. Currently, the JSF provides adapters for Hypertext Transfer Protocol (HTTP), Secure Hypertext Transfer Protocol (S-HTTP), Sockets, Secure Sockets, Servlets, Java Server Pages (JSPs), Enterprise JavaBeans (EJBs), Java Message Service (JMS), and as a standalone Java application.

The adapter layer interfaces with middleware and therefore dictates the robustness of the application. The Servlet, JSP, EJB and JMS adapters greatly increase the robustness and availability of the underlying application because the middleware products they interface with perform operations such as resource management, automatic restart in a failure situation, load balancing and queuing.

Components

Components are coarse-grained objects that make up the business logic of a particular application. Two types of components are supported—command components and data source components.

Command components are specific components that adhere to the command design pattern. These components are the business logic that makes up an application. Since these components all have the same interface, the framework has a layer that allows applications to be created by defining the order that the components are invoked through a Transaction Meta Map XML document. A Transaction Meta Map XML document is a work flow descriptor that defines the series of actions required to process a transaction. A number of ACI applications, such as the BASE24-es ATM Device Handler processes, are built using this technique.

Data source components are encapsulation components that provide a single view to a data source. All transaction and user interface logic in command components use these components to access individual data sources. This layer performs routing to change the access to a data source from a database to potentially a message interface, an EJB, or a Lightweight Directory Access Protocol (LDAP) directory. Part of the framework consists of a code generator that builds a data source component for any relational database table through a metadata file. This same metadata file is also used to build database services described below.

Services

Services encapsulate specific entities of the application such as database and message services. They do not have a standard interface, but they do provide many commonly executed functions in a common reusable location. These are utility services that need to be present when building an online transaction processing application and include the following:

- Database services
- Message services
- Cryptography services
- Timer services
- Context management services

Framework

The framework layer encapsulates all system resources including messaging, message routing, XML formatting, database access, logging, file system access, object caching, tracing, class loading and application configuration. The main function of the framework layer is to ensure that the applications written on top of it run on a variety of platforms and take advantage of the features of each platform from a scalability and performance standpoint. The framework layer also provides common application functionality required for gaining code reuse among the applications built on it. The benefit of this is quicker time to market for new applications. The framework layer can be broken down into the following subsystems, each of which are described in detail in [section 2](#).

- The Database subsystem provides support for database connectivity, database performance optimizing, and database auditing facilities.
- The Message subsystem provides the ability to message synchronously and asynchronously and includes callback functionality configured through a routing database to plug in message formatters at run time.
- The Clustering subsystem provides the ability to configure applications as multiple single-threaded applications on clustered servers such as the HP NonStop server, or as one or a few multithreaded applications on SMP servers such as Windows NT, UNIX, and IBM zSeries.
- The Encryption subsystem performs a number of cryptographic functions such as DES encryption/decryption and adheres to the Java Cryptographic Extensions (JCE) API.

- The Logging subsystem provides for the logging of event messages generated by Java server processes using a third-party open source application called log4j. Log4j is an open source project, of the Apache group, which is based on the work of multiple authors.
- The Tracing subsystem provides for extensive tracing of all messaging, database access, and cache management using log4j.
- The Message Transformation subsystem provides a set of base classes for building and parsing external messages.
- The Directory subsystem contains an interface to naming and directory services.
- The Cache Management subsystem contains a Cache Manager component that can maintain cache information by Service type. Services make requests to place information in memory and retrieve information from memory. You can configure the database services for which data is cached and whether message context is stored in cache memory.
- The Configuration and Instrumentation subsystem provides mechanisms for configuring how Java server processes run, configuring how they connect to external entities, and configuring the format and descriptions of event messages that can be generated by JSF applications. This subsystem also allows you to display and monitor database connections, socket connections, cache information, and threads in use.

Class Loading

The class loading mechanism of the JSF uses Java reflection to automatically load Java classes and instantiate instances of these classes at run time. This mechanism is used mainly in the Message subsystem for dynamically creating message formatters for building and parsing messages. These classes are declared in the routing database (e.g., Transaction Map XML file) and invoked at run time.

Application Programming Interfaces (APIs) and Third-Party JARs

The ACI Java Server Framework uses several industry-standard Java application programming interfaces (APIs), each of which is described below. Several of these APIs are provided by third-party JARs. ACI provides these third-party Java Archive (JAR) files with applications built on the JSF.

Third-party JARs specific to JSF-based applications are described in the application-specific appendices of this guide.

Note that the Java 2 Platform Enterprise Edition (J2EE) JAR is not provided by ACI. Visit the following website for more information on obtaining J2EE:

<http://java.sun.com/j2ee/index.jsp>

Java DataBase Connectivity (JDBC)

See the “[Database Subsystem](#)” topic in section 2 for information on how this API is used in the JSF.

Java Message Service (JMS)

The Java Message Service (JMS) API is a messaging standard that allows application components based on the Java 2 Platform, Enterprise Edition (J2EE) to create, send, receive, and read messages. It enables distributed communication that is loosely coupled, reliable, and asynchronous.

See the “[Message Subsystem](#)” topic in section 2 for more information on how this API is used in the JSF.

Java Servlets

Java Servlet technology provides a simple, consistent mechanism for extending the functionality of a Web server and for accessing existing business systems. The ACI desktop user interface runs as a servlet under the JVM.

Enterprise JavaBeans (EJBs)

The JSF ensures that the applications written on top of it are able to be deployed as EJBs, however, ACI does not currently deploy the Java applications in EJB containers.

Java Cryptography Extension (JCE)

The Java Cryptography Extension (JCE) provides a framework and implementations for encryption, key generation and key agreement, and message authentication code (MAC) algorithms. Support for encryption includes symmetric, asymmetric, block, and stream ciphers. The software also supports secure streams and sealed objects.

JCE is based on the same design principles found elsewhere in the Java Cryptography Architecture (JCA)—implementation independence and, whenever possible, algorithm independence. It uses the same *provider* architecture. Providers signed by a trusted entity can be plugged into the JCE framework, and new algorithms can be added seamlessly.

The JCE API covers the following:

- Symmetric bulk encryption, such as the Data Encryption Standard (DES), Rivest's Cipher 2 (RC2), and the International Data Encryption Algorithm (IDEA)
- Symmetric stream encryption, such as RC4
- Asymmetric encryption, such as Rivest, Shamir, and Adleman (RSA)
- Password-based encryption (PBE)
- Key agreement
- Message authentication codes (MACs)

See the [“Encryption Subsystem”](#) topic in section 2 for more information on how this API is used in the JSF. Third-party JARs used by this API are described below.

Local_policy for Java Cryptography Extension (JCE). The local_policy for JCE provides the ability to do Full Function versus Limited Key Size Cryptography through the JCE API.

Sunjce_provider for JCE. This is a standard cryptographic service provider named “SunJCE” (which implements a session key generator for DES). You must explicitly register the “SunJCE” provider with every JCE installation. The JCE and JSSE must be installed before an ACI JSF-based application can be installed. Note that with Java Virtual Machine 1.4 and above, the JCE and JSSE are already installed and registered.

US_export_policy for JCE. The US_export_policy for JCE provides the ability to do Full Function versus Limited Key Size Cryptography through the JCE API.

Bouncy Castle. The JSF uses the Bouncy Castle cryptographic APIs JAR for database password encryption. BouncyCastle is a lightweight cryptography API in Java created by the Legion of the Bouncy Castle that provides the following:

- A provider for the Java Cryptography Extension (JCE) and Java Cryptography Architecture (JCA).
- A clean room implementation of the JCE.
- A library for reading and writing encoded Abstract Syntax Notation One (ASN.1) objects.
- Generators for Version 1 and Version 3 X.509 certificates and files compliant with the Public-Key Cryptography Standards (PKCS) #12 specification, ***Personal Information Exchange Syntax Standard***.
- Generators/Processors for Secure MIME (S/MIME) and Cryptographic Message Syntax (CMS). CMS is a superset of the Public-Key Cryptography Standards (PKCS) #7 specification, ***Cryptographic Message Syntax Standard***, from RSA Laboratories.
- Generators/Processors for Online Certificate Status Protocol (OCSP). Refer to the Internet Engineering Task Force (IETF) specification for OCSP (RFC 2560) for more information on OCSP.
- A signed JAR version suitable for the Java Development Kit (JDK) and the Sun JCE.

For more information on Bouncy Castle, refer to the following website:

<http://www.bouncycastle.org/>

Java API for XML Binding (JAXB)

See the “[Message Transformation Subsystem](#)” topic in section 2 for information on how this API is used in the JSF.

Log4j

Log4j is an open source project of The Apache Software Foundation based on the work of many authors. It allows the developer to control which log statements are output with arbitrary granularity. It is fully configurable at runtime using external configuration files. Best of all, log4j has a gentle learning curve.

See the “[Logging Subsystem](#)” and “[Tracing Subsystem](#)” topics in section 2 for more information on how this API is used in the JSF. Third-party JARs used by this API are described below.

Java Document Object Model (JDOM)

JDOM is an API for manipulating XML documents. JDOM is both Java-centric and Java-optimized. It behaves like Java, uses Java collections, is a completely natural API for current Java developers, and provides a low-cost entry point for using XML. Log4j uses JDOM.

While JDOM interoperates well with existing standards such as the Simple API for XML (SAX) and the Document Object Model (DOM), it is not an abstraction layer or enhancement to those APIs. Rather, it provides a robust, light-weight means of reading and writing XML data without the complex and memory-consumptive options that current API offerings provide.

Java Authentication and Authorization Service (JAAS)

Java Authentication and Authorization Service (JAAS) is a set of APIs that can be used for the following two purposes:

- For authentication of users, to reliably and securely determine who is currently executing Java code, regardless of whether the code is running as an application, an applet, a bean, or a servlet.
- For authorization of users, to determine reliably and securely who is currently executing Java code, regardless of whether the code is running as an application, an applet, a bean, or a servlet.

JAAS authentication is performed in a pluggable fashion. This permits Java applications to remain independent from underlying authentication technologies. New or updated technologies can be plugged in without requiring modifications to the application itself. JAAS authorization extends the existing Java security architecture that uses a security policy to specify what access rights are granted to executing code. That architecture, as introduced in the Java 2 platform, was code-centric. That is, the permissions were granted based on code characteristics: where the code was coming from and whether it was digitally signed and if so by whom. With the integration of JAAS into the Java 2 Software Development Kit (SDK), the `java.security.Policy` API handles Principal-based queries, and the default policy implementation supports Principal-based grant entries. Thus, access control can now be based not just on what code is running, but also on who is running it.

Java API for XML Processing (JAXP)

Provides a common interface for creating and using the standard Simple API for XML (SAX), XML Document Object Model (DOM), and Extensible Stylesheet Language Transformations (XSLT) APIs in Java, regardless of which vendor's implementation is actually being used. Third-party JARs used by this API are described below.

Xercesimpl. Xerces (named after the Xerces Blue butterfly) provides world-class XML parsing and generation. Fully-validating parsers are available for both Java and C++, implementing the W3C XML and DOM (Level 1 and 2) standards, as well as the de facto SAX (version 2) standard. The parsers are highly modular and configurable. Initial support for XML Schema (draft W3C standard) is also provided.

XMLParserAPIs. The Java API for XML Processing (JAXP) provides a common interface for creating and using the standard SAX, DOM, and XSLT APIs in Java, regardless of which vendor's implementation is actually being used.

Commons-lang

The Lang Component provides a host of helper utilities for the java.lang API, notably string manipulation methods, basic numerical methods, object reflection, creation and serialization, and system properties. Additionally it contains an inheritable enum type, an exception structure that supports multiple types of nested-Exceptions, basic enhancements to java.util.Date and a series of utilities dedicated to help with building methods, such as hashCode, toString and equals.

Commons-logging

The Logging package is an ultra-thin bridge between different logging libraries. Commons components can use the Logging API to remove compile-time and run-time dependencies on any particular logging package, and contributors can write Log implementations for the library of their choice.

Concurrent

This package provides standardized, efficient versions of utility classes commonly encountered in concurrent Java programming. This code consists of implementations of ideas that have been around for some time, and is merely

intended to save you the trouble of coding them. Discussions of the rationale and applications of several of these classes can be found in the second edition of the *Concurrent Programming in Java* book, by Doug Lea.

Javadoc

All ACI JSF-based applications use Sun's Javadoc tool provided in the Java 2 SDK to generate API documentation (in HTML files) for all classes, methods, and constants used by the application. These files are delivered with all JSF-based applications and are typically provided within a "docs" folder or an "apidocs" WinZip archive (apidocs.zip). Java programmers may find these documents useful for developing or customizing JSF-based applications.

Java Server Properties

In Java, configuration often takes the form of properties. Java server processes running under the ACI Java Server Framework are configured using properties. Each property consists of a key name and a value. Both are string values. The ACI JSF uses three levels of properties—system, configuration, and application. The following paragraphs introduce each level of properties used for JSF. Each of the properties common to the JSF are described in section 5. Application-specific properties are described in appendices.

System Properties

System properties are set by the operating system, by a user logged on to the operating system, or in a -D option JVM runtime argument or configuration properties file accessed when a Java server process is started. The ACI JSF requires a minimum of four system properties: process ID, configuration filename (i.e., the location of the config.properties file used to bootstrap a Java server process), file encoding (specifies the character set used), and file separator (specifies the default name-separator character, e.g., /).

The file encoding and file separator properties are set by the operating system and the process ID and configuration filename properties are set when a Java server process is started.

You can view the system properties currently defined for a Java server process on the System Properties window. This window is accessed from **System Operations > Server Management > Server Administration > System Properties** on the ACI desktop user interface.

Configuration Properties

Configuration properties are used to bootstrap a Java server process when it is started. These properties are configured in a config.properties file, the location of which can be specified in the config.filename system property. If the location is not specified in the -Dconfig.filename system property, it defaults to a file named “config.properties”.

For any properties defined in the config.properties file that are also defined in the Application Configuration data source (APPLD), the value in the config.properties file supersedes the value in the APPLD, except for the trace.enabled property. For this property, the value in the APPLD overrides the value in the config.properties file once the Java server process reads the APPLD.

As stated above, the location of the config.properties file must be provided when a Java server is started. Only those properties necessary for a process to initialize should be defined in the config.properties file. These include the following:

- User ID and password for database access
- Database connection properties for accessing the database
- Location of the log4J configuration files for events and tracing

Initialization manager function properties for the Map Managers can be configured in the config.properties file or the APPLD. In either location, the entries in this list must be sequentially numbered. For example, if entries are numbered 0, 1, 3, etc., the Java server process would stop initializing with entry 1 because the next function expected is entry 2.

ACI provides a utility to encrypt the user password stored in the config.properties file. This utility is described in [section 5](#).

You can view the configuration properties currently defined for a Java server process on the Configuration Properties window. On the ACI desktop user interface, you access this window from **System Operations > Server Management > Server Administration > Configuration Properties**.

Application Configuration Properties

The Application Configuration data source (APPLD) defines application-level properties for Java server processes. This file is maintained from the Application Configuration window (accessed from **System Operations > Server Management > Application** on the ACI desktop user interface). Common properties defined in this file include connection IDs, database properties, queue locations, whether data sources are audited or accessed from cache memory, and whether tracing is enabled. Properties for application-specific functions are also set in the APPLD.

Graphical User Interface

JSF applications use graphical user interface screens or windows to manage and administer the JSF environment. These user interface screens or windows are introduced below and described in detail in sections 3 and 4.

Java Server Management

Java server management consists of maintaining the configuration for Java server processes. This includes managing the configuration of application-level properties, connections to and from external entities, process relationships in clustered configurations for broadcasting messages, and event messages.

ACI provides formatted screens or windows to maintain this information. Depending on the product in which JSF is employed, these screens or windows can be browser- or Java-based. Although the look and feel may differ from one product to the next, the fields displayed and the meta data maintained are the same across all products. The following four screens or windows are provided to maintain Java server configuration information:

- The Alternate Data Source data source (DATASRC) maintains alternate data source information. Currently, there is no user interface window for the DATASRC. The fields in the DATASRC are described in section 3.
- The Application Configuration window or screen maintains application-level properties in the Application Configuration data source (APPLD). The fields on this window/screen are described in section 3. Application-level properties common to the JSF are described in section 5. Product-specific application-level properties are described in product-specific appendices to this user guide.
- The External Connection Configuration window or screen maintains data in the External Connections data source (EXTCND). The fields on this window/screen are described in section 3.
- The Listener Configuration window or screen maintains data in the Listener data source (LSTND). The fields on this window/screen are described in section 3.
- The Process Configuration window or screen maintains data in the Process data source (PROCD). The fields on this window/screen are described in section 3.

- The Integrated Server Event Configuration and Management window maintains data in the Event (EVENTD) and Event Document (EVDOD) data sources. The fields on this window/screen are described in application-specific documentation.

Java Server Administration

The JSF allows you to view and clear objects cached in the Cache Manager within the executing Java Virtual Machine (JVM) process and stop and start TCP/IP sockets. In addition, you can manage properties, view database usage, view system resource usage, and view system properties used by a Java server process.

ACI provides formatted screens or windows to perform these functions. Depending on the product in which JSF is employed, these screens or windows may be browser- or Java-based. Although the look and feel may differ from one product to the next, the fields and data displayed and the functions you can perform are the same across all products. The following six screens or windows are provided to administer Java server processes:

- The Cache Administration window or screen displays cached objects and allows you to clear objects from cache.
- The Configuration Properties window or screen displays the current properties for a Java server process from the config.properties, APPLD, and inherited from the JVM. From this window or screen, you can also reload application-level properties after making changes to the APPLD.
- The Database Administration window or screen displays database clients and connections used by Java server processes.
- The Socket Administration window or screen lets you stop and start TCP/IP socket connections used in the JVM.
- The System Monitor window or screen lets you view the dynamic use of system resources by a Java server process.
- The System Properties window or screen lets you view the system-level properties used by a Java server process.

The fields on the above windows/screens are described in section 4.

Section 2

Subsystems

This section describes each of the subsystems provided in the Java Server Framework.

Database Subsystem

The database subsystem of the JSF includes several features to enhance ACI applications. Each of these features is discussed below.

Code Generator

A code generator allows developers to build a database access service to any relational database table through a metadata file. This increases the speed at which applications can be developed. The interface to these services is modeled after the EJB entity bean interface. The database services convert all SQL statements into compiled statement objects and stores them for future use to avoid the overhead of recompiles inherent in most Open DataBase Connectivity (ODBC) and Java DataBase Connectivity (JDBC) applications. This greatly increases the performance of an application accessing a relational database.

Transaction Integrity

Database services encapsulate the functionality to provide transaction protection across the different transaction protection monitors, such as the Transaction Monitoring Facility (TMF) on the HP NonStop platform. The database services also provide extensive tracing of all database access and failure recovery.

Auditing User Actions

The database auditing mechanism enables you to track all adds (inserts), updates (before and after images), deletes, and views of the application database by user ID and time of change. When a database service is audited, records are written to a User Audit Log data source (UALOGD). You can turn auditing on or off using the `db.audited` property. The values for this property are `true` (auditing is turned on) or `false` (auditing is turned off). When auditing is turned on, you can configure the database services that are audited when your ACI system is installed and the extent to which database services are audited in `db.audited.db-service-name` properties, where *db-service-name* is the name of a database service to be audited. The values for these properties are `none`, `medium`, and `full`. For full auditing of a database service, header information as well as detail information about the before and after images of the affected record are logged to the UALOGD. For medium auditing, only the header information is logged to the UALOGD. For the `none` value, no auditing records are generated for the specified database service.

Additional JSF properties (i.e., `cleanup.userauditlog` properties and a `timertask.function.n.classname` property) enable you to configure a User Audit Log Cleanup component to remove old data from the UALOGD after a specified retention period expires. Application-specific properties can configure the directory location of an XML file for extracts as well as timed automatic extract of the UALOGD by the User Audit Log XML Extract component.

Auditing records are written to the UALOGD in XML message format and can be extracted for detailed reporting or analysis. Each record contains a timestamp, a context description, ticket information, activity type, and data source name.

Context descriptions provide context for the audited activity. For example, a typical context description for an application might be “CardInsertRq” or “VisaViewRequest”. You can check the source code to find all possible context descriptions for a JSF-based application. Currently, the JSF does not provide a context description for entries it writes to the UALOGD.

Activity types generic to the JSF environment include: Delete, Insert, Update, and View. Each JSF-based application may define additional activity types. For example, the USEC application for the ACI desktop user interface includes the following activity types:

- AuditReportGeneration
- ChangePassword
- Delete
- ExpirePassword
- Insert
- Logoff
- LogoffSelected
- Logon
- SessionExpired
- SessionTimeout
- Update
- VerifyPassword
- View

Refer to application-specific documentation for the context descriptions and activity types supported for a specific JSF-based application.

Caching

The database caching mechanism coordinates cache synchronization across all executing Java server processes. You can configure the database services cached when your ACI system is installed for individual database services. Database caching improves performance for database rows or records accessed frequently and updated infrequently.

Database caching uses the Cache Manager to store database objects (see the [“Cache Management Subsystem”](#) topic for more information). The Cache Manager provides synchronization across all executing Java applications.

Drivers and APIs

Additional aspects of the database subsystem are the ACI-authored type 2 and type 3 (for the IBM Zseries) JDBC drivers written within a JDAL layer. The JDBC API is the industry standard for database-independent connectivity between the Java programming language and a wide range of databases. The JDBC API provides a call-level API for SQL-based database access. JDBC technology allows ACI to use the Java programming language to exploit “Write Once, Run Anywhere” capabilities for applications that require access to enterprise data. These ACI-written drivers, Java Database Access Layer (JDAL) (type 2) and Java Database Access Layer 2 (JDAL2) (type 3), allow ACI’s Java applications to access Enscribe, c-tree, and Virtual Storage Access Method (VSAM) data sources as a configurable JDBC driver at deployment time. They also provide an open industry standard API to the BASE24-es applications data sources.

Alternate Data Sources

Through the use of the Alternate Data Source data source (DATASRC), you can configure specific data sources to be maintained in data formats that differ from the data format of the main application database. When attempting to access a data source for a particular Java database service, the database system will access the alternate data source if one is configured. If an alternate data source is not configured, it accesses the data source from the main application database. For more information on the DATASRC, refer to [section 3](#).

Pooling and Locking

Other features of the database subsystem include: connection pooling, statement pooling, *optimistic locking* (i.e., one user cannot overwrite another user's data without locking database rows), *pessimistic locking* (i.e., transaction protection). See the [glossary](#) for descriptions of optimistic and pessimistic locking.

Connections to the database are obtained from a static connections pool. Static connections allow all the expense of connecting to the database to be incurred only during process startup.

Statement pooling (available from JDBC 3.0) caches SQL queries that have been previously optimized and run so that, should they be needed again, they do not have to go through optimization preprocessing again (avoiding optimization steps, such as checking syntax, validating addresses, and optimizing access paths and execution plans). Statement pooling can be a significant performance booster.

Statement pooling and connection pooling in JDBC 3.0 can cooperate to share statement pools, so that connections that can use a cached statement from another connection, thus incurring statement preparation overheads only once on the first execution of some SQL by any connection.

Message Subsystem

The framework has the ability to message synchronously and asynchronously and includes callback functionality configured through a routing database to plug in message formatters at run time. This allows you to write your own message formatters and dynamically configure them for easy implementations of custom interfaces to other applications. ACI has written message formatters for several ACI products including the following: various XML formats, ISO 8583:1993, ANSI X9.15, NCR NDC+, Diebold 911/912, IFX, and payer authentication formats for Visa and MasterCard. Message interfaces and their endpoints are configured through an extensible database and provide access to send Transmission Control Protocol/Internet Protocol (TCP/IP), HTTP, and JMS messages with different message parameters such as two-byte headers and end-of-text (EOT) characters, timeout settings and retry counts.

The Java Message Service (JMS) defines the standard for reliable Enterprise Messaging. Enterprise messaging, often also referred to as Messaging Oriented Middleware (MOM), is universally recognized as an essential tool for building enterprise applications. By combining Java technology with enterprise messaging, the JMS API provides a powerful tool for solving enterprise computing problems.

JMS parameters allow Java JMS applications to communicate with non-Java applications through WebSphere MQ or NET24-XPNET. The JMS implementation also has the capability to enable the JMS queue listeners to run multithreaded on symmetric multiprocessing (SMP) platforms. The framework performs connection pooling to TCP/IP sockets and JMS providers to eliminate the overhead of the connection operations during the transaction path. A JMS API has been implemented under NET24-XPNET for sending asynchronous messages, allowing applications written on the JSF to look like any XPNET satellite process. On platforms such as Windows NT, UNIX, and IBM zSeries, the application can message asynchronously through JMS providers such as IBM's WebSphere MQ and Sonic MQ.

Clustering Subsystem

The framework has the ability to configure applications as multiple single-threaded applications on clustered servers such as the HP NonStop server, or as one or a few multithreaded applications on SMP servers such as Windows NT, UNIX, and IBM zSeries. The framework contains a database to define application processes and the mechanism for communicating among the processes using the message services defined above. Parts of the application that utilize the clustering services are the Database, Cache Management, and Configuration and Instrumentation subsystems.

Encryption Subsystem

The framework performs a number of cryptographic functions such as DES encryption/decryption. The encryption subsystem adheres to the Java Cryptographic Extensions (JCE) API. Currently, it is used with Thales e-Security and Eracom hardware security modules in some ACI applications to verify and generate digital signatures, generate MAC keys, generate RSA keys, generate random numbers, verify and generate message digests, perform certificate chain validation, verify root certificates, verify self signed certificates, and generate and parse digital envelopes. The encryption subsystem can support both software and hardware encryption interfaces that adhere to the JCE.

Note: BASE24-es ATM Device Handler processes do not use the JCE. Instead, they interface to the Transaction Security Services process through JMS to perform cryptographic functions. In this case, none of the *crypto* properties described in [section 4](#) need to be configured.

Logging Subsystem

You can configure the logging subsystem to log event messages generated by Java server processes to a variety of locations. The framework uses a third party open source application called log4j. Log4j is an open source project, of the Apache group, which is based on the work of multiple authors. It allows the developer to control which log statements are output with arbitrary granularity and is fully configurable at runtime using external configuration files. You can configure log4j to send messages to the following destinations in the EventConfig.xml file: the JVM console, daily rolling log files, a listening JVM over a socket, the Windows NT Event Log, the Unix Syslog daemon, and a JMS topic. ACI has also written appenders for the HP NonStop Event Management Subsystem (EMS), an SQL database, and a CICS Transaction Data Queue (TDQ) for integrating with other ACI Commerce Framework products and technologies. The text of event messages can be configured to be extracted from a database or a properties file for internationalization. The event database also contains documentation about the probable cause, effect, and remedy for each event message. Refer to the following website for more information on log4j.

<http://logging.apache.org/log4j/docs/documentation.html>

Tracing Subsystem

The framework event environment provides extensive tracing of all messaging, database access, and cache management. The tracing subsystem can be configured to log trace messages to the same locations as the event logging subsystem because the tracing subsystem also uses log4j. The tracing subsystem is configured in the TraceConfig.xml file.

Message Transformation Subsystem

The message transformation subsystem is a set of base classes for building and parsing external messages. Currently, ISO 8583:1993, ANSI X9.15, IFX, and XML components exist for building messages. The framework also contains a tag-based extensible Data Object for sending messages between Java applications. This object can be serialized and deserialized using Java-specific transport mechanisms such as JMS and RMI.

The framework XML message subsystem is a part of the Message Transformation subsystem. Included in the XML message subsystem is an XML parser that uses the output from the Castor framework of the Apache group. The use of this parser with the data binding output of the Castor data binding yields much faster parsing and building of XML documents than the standard parser mechanisms. The Java API for XML binding (JAXB) has also been utilized for building XML formatters more quickly and that perform well. The framework also automatically pools XML parsers for performance reasons.

Directory Subsystem

The directory services subsystem contains an interface to naming and directory services such as Lightweight Directory Access Protocol (LDAP), Network Information Service (NIS), Common Object Request Broker Architecture (CORBA) Common Object Services (COS) Naming, and files.

Cache Management Subsystem

Because of the need to build extended memory tables for database information that changes infrequently, the framework contains a Cache Manager component that can maintain cache information by Service type. Services make requests to place information in memory and retrieve information from memory. ACI application-specific user interfaces allow you to peruse the information in cache by service and object key. Cache management is used mostly by the database services and the message services for maintaining message context. The Cache Manager has timers that you can configure using properties to automatically clean up cache pools on a regular basis such as nightly or even hourly. You can also manually clear cache through the application-specific user interface.

Configuration and Instrumentation Subsystem

The framework's configuration options are supplied in a Java properties file, XML documents, or a data source at runtime. Bootstrap properties such as the database location and logging subsystem configuration locations are defined in the config.properties file. ACI recommends that the rest of the application properties be configured in the Application Configuration data source (APPLD). You can modify the APPLD and refresh the in-memory copy of the APPLD through the supplied user interface while the Java server application is running. Thus, you can keep the application running while reconfiguring it. You can also display all of the runtime properties (bootstrap, system, and application) for a Java server process through the user interface.

XML documents control logging of event and trace messages. In addition, some Java server processes use additional XML files for the scripting of work flow.

The Java Server Framework uses a number of data sources other than properties files and XML documents to configure how Java server processes connect to external entities and to format and describe event messages generated by JSF applications.

The user interface for the JSF also allows you to display and monitor database connections, socket connections, cache information and threads in use. Refer to [section 4](#) for more information on these user interface windows or screens.

Properties Files

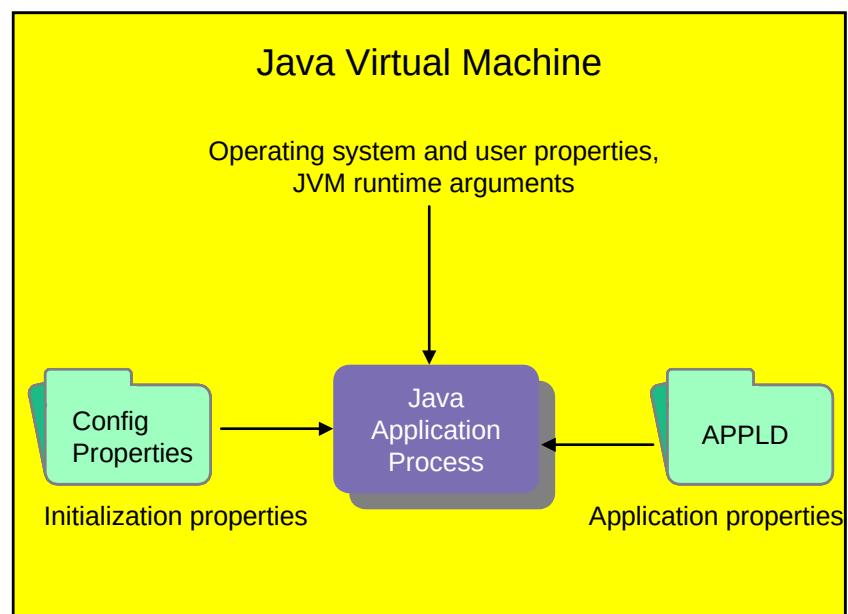
The Java Server Framework uses two files to configure properties. The config.properties file configures properties used to bootstrap a Java server process. This properties file must be specified in an argument in the Java Virtual Machine (JVM) command used to start a Java server process. For more information on configuration properties, refer to the [“Configuration Properties”](#) topic provided in section 1. The Application Configuration data source (APPLD) defines properties that configure how a Java server performs processing. For more information on application-level properties, refer to the [“Application Configuration Properties”](#) topic provided in section 1. If the same property is configured in both the config.properties file and the APPLD, the value in the config.properties overrides the value in the APPLD, except for the trace.enabled property. For this property, the value in the APPLD overrides the value in the config.properties file once the Java server process reads the APPLD.

During initialization, a Java server process loads properties from the config.properties file into an internal table. The process always attempt to find a property in this internal table before attempting to retrieve the property from the APPLD. Therefore, if a property is configured in the config.properties file, the process does not even attempt to retrieve it from the APPLD. The only exception is the trace.enabled property. The process uses the trace.enabled property from the config.properties file until database connections have been established. It then reads the APPLD for the trace.enabled property. This allows for tracing of the logic up until the database connections have been established.

Note: Some JSF applications can use additional properties files. For example, BASE24-es ATM Device Handler processes use device type-specific properties files to define text for display on receipts, and other configuration attributes for ATM terminals.

The Java Server Framework also uses properties defined at the system level. These properties are inherited from the operating system, set by the user logged on to the JVM, or set in JVM runtime arguments when a Java server process is started.

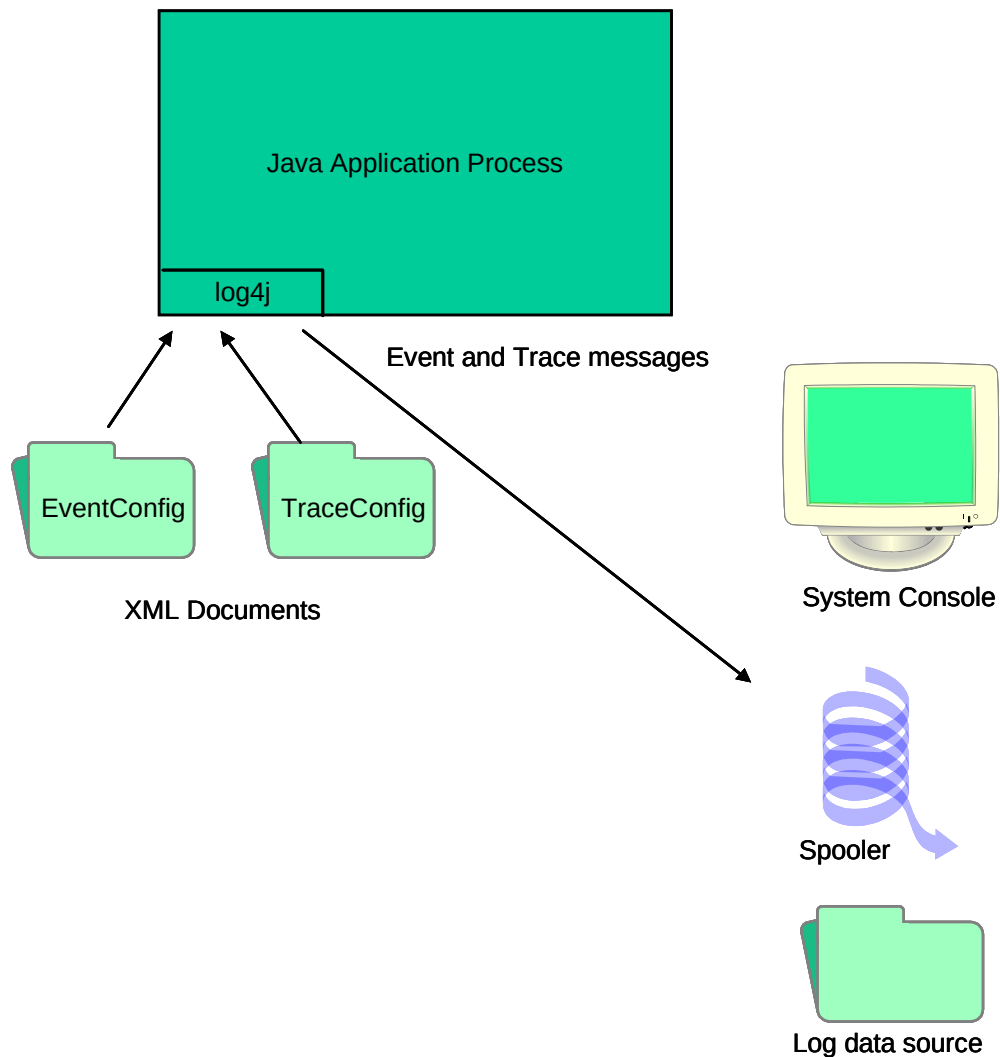
The relationships between properties and a Java server process are depicted in the following illustration.



XML Documents

All Java servers in the JSF use two XML document files to configure how event and trace messages are generated by log4j. The EventConfig.xml file configures log4j for generating event messages. The TraceConfig.xml file configures log4j for generating trace messages. Some Java server processes may use additional XML document files for the scripting of work flow. These XML document files are either described in an appendix to this guide or in an application-specific user guide or manual. The EventConfig.xml and TraceConfig.xml files are described in detail in section 4.

Use of the EventConfig.xml and TraceConfig.xml files by log4j in a Java server process is depicted in the following illustration.



Data Sources

The JSF uses several data sources other than properties files and XML document files to configure how Java server processes connect to external entities and to configure and describe event messages that can be generated by Java server processes. Each of these data sources is introduced below and described in more detail in section 3.

Alternate Data Source

The Alternate Data Source data source (DATASRC) enables you to maintain certain data sources in a data format that differs from the main application database. This allows you to mix and match data formats as needed for your application. The DATASRC contains all the information needed to access an alternate data source, including the relative URL location, database driver, user ID and password, table prefix, and database connection parameters. The DATASRC does not currently have a user interface window. For detailed information on the [DATASRC](#), refer to section 3.

Event

Event messages generated by Java server processes are configured in the Event data source (EVENTD). The EVENTD also configures whether events are generated or suppressed. The EVENTD is maintained from the Integrated Server Event Configuration and Management window on the ACI desktop user interface (accessed from **System Operations > Event > Event Management**).

Note: JSF-based applications do not use tokens in event messages. Therefore, you cannot use token IDs (e.g., <1>, <2>, etc.) for text substitution in event messages generated by JSF-based applications. JSF-based applications use braces ({}) instead of tokens to denote text substitution in event messages. Brace-based text substitution is hard-coded in JSF-based application code.

Event Documentation

The probable cause, effect, and remedy for an event are stored in the Event Documentation data source (EVDOCD). Token descriptions for an event message are also stored in the EVDOCD. The EVDOCD is maintained from the Integrated Server Event Configuration and Management window on the ACI desktop user interface (accessed from **System Operations > Event > Event Management**).

External Connection

The External Connections data source (EXTCND) is used to configure connections from a Java server process to external entities. The Java server process uses the EXTCND when it requests the external entity to perform some unit of work on its behalf. The protocol used to connect to the external entity is also configured in this data source. The EXTCND is maintained from the External Connection Configuration window on the ACI desktop user interface (accessed from **System Operations > Server Management > External Connection**). For detailed information on this window, refer to section 3.

Listener

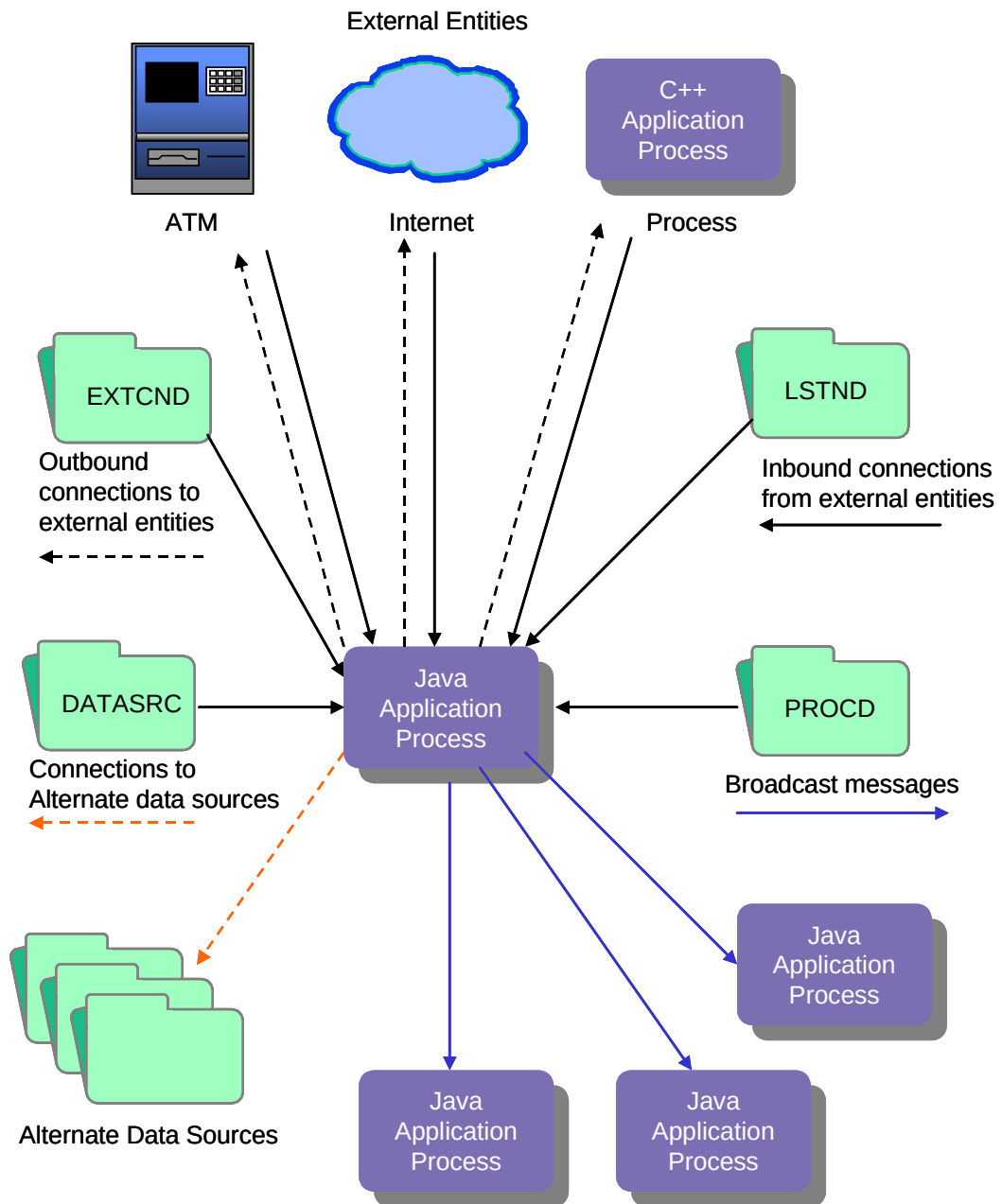
The Listener data source (LSTND) is used to configure connections from external entities to a Java server process. The Java server process uses the LSTND when it needs to receive a unit of work, and it cannot be received through the standard Servlet mechanism. The protocol used by the external entity to connect to the Java server process is also configured in this data source. The LSTND is maintained from the Listener Configuration window on the ACI desktop user interface (accessed from **System Operations > Server Management > Listener**). For detailed information on this window, refer to section 3.

Process

The Process data source (PROCD) is used to configure references to process entities running in the JSF. A Java server process uses the PROCD when it needs to broadcast messages to other processes running under the JVM, whether on a clustered or SMP platform. In addition, some products (e.g., BASE24-es) require an entry in the PROCD for every process running in the JVM because drop down lists in the ACI desktop user interface are populated from it.

All processes configured in the PROCD must be configured with a system property named “process.id”. This property is set in a JVM argument when the process is started (e.g., -Dprocess.id=P1A^JDH1). The PROCD is maintained from the Process Configuration window on the ACI desktop user interface (accessed from **System Operations > Server Management > Process**). For detailed information on this window, refer to section 3.

The following graphic illustrates how the DATASRC, EXTCND, LSTND, and PROCD are used in the JSF.



ACI Worldwide Inc.

Section 3

Management

This section describes how to manage the configuration for Java server processes. This includes managing the configuration of alternate data sources, application-level properties, connections to and from external entities, and process relationships in clustered configurations for broadcasting messages. These configuration items are maintained in the following JSF data sources:

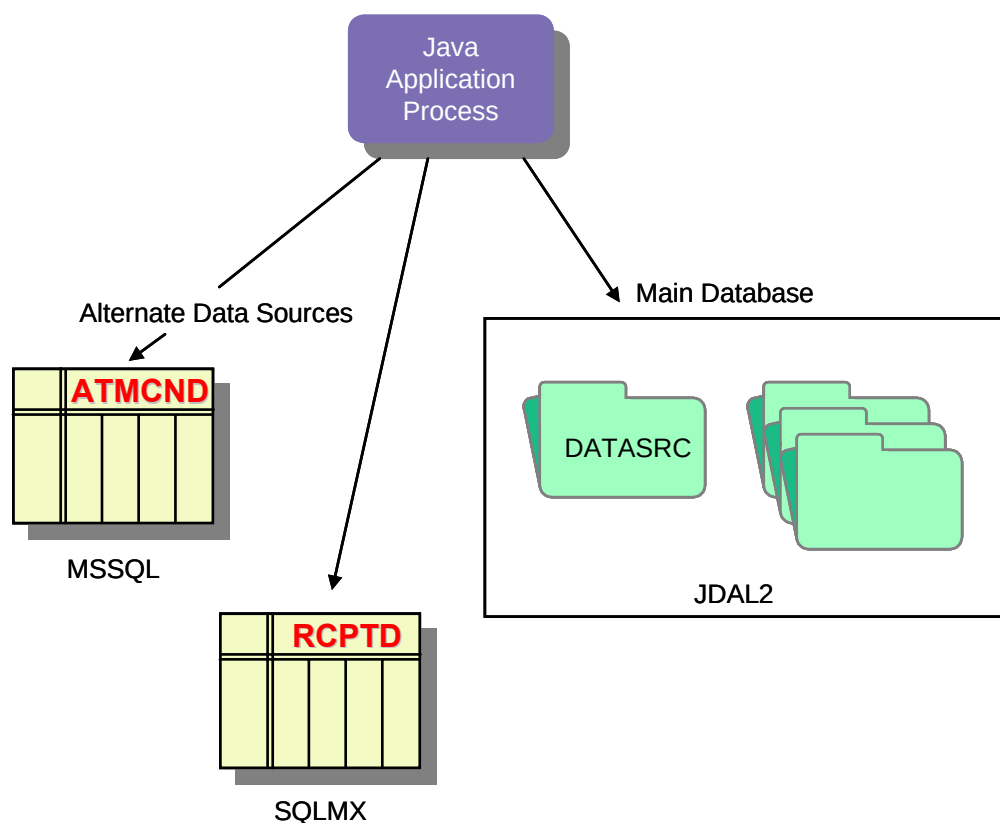
- Alternate Data Source data source (DATASRC)
- Application Configuration data source (APPLD)
- External Connections data source (EXTCND)
- Listener data source (LSTND)
- Process data source (PROCD)

JSF-based ACI applications provide formatted user interface screens or windows to maintain configuration data in these data sources. Although the look and feel of these screens and windows may differ from one product to the next, the fields displayed and the meta data maintained are the same across all products. This section describes how to configure these fields to meet the configuration requirements for your system.

Alternate Data Source Configuration

For various reasons, you may need to maintain certain data sources in databases other than the main application database. You can do this by configuring a `db.alt.datasource.name.db-service-name` property with the name of the data source and a corresponding Alternate Data Source data source (DATASRC) row that defines how to access the data source. This property and the DATASRC allow you to store data in different databases as needed for your application. The DATASRC contains all the information needed to access an alternate data source, including the relative URL location, database driver, user ID and password, table prefix, and database connection parameters.

When a Java server process needs to access the database, the Database Service first determines whether a `db.alt.datasource.name.db-service-name` property exists for the database service. If so, it reads the DATASRC from cache or disk and uses that information for the database access operation. If not, it accesses the database in the usual manner using the main database (i.e., “db.”) property values. The use of alternate data sources is depicted in the following illustration.



Field Descriptions

The following paragraphs describe each field in the DATASRC.

DataSrcNamTx — The name of the alternate data source. This name must match the value of a `db.alt.datasource.name.db-service-name` property.

DrvNamTx — The class name of the JDBC driver to use for the alternate database connections. You should enter the entire package name as it is found in the JAR containing the driver. Refer to the vendor documentation for the JDBC driver for supporting information.

DataSrcURLTx — The data source URI specification that allows the JDBC driver to access the correct alternate database. You can typically find the configuration requirements for this property in the vendor documentation for the JDBC driver. JDBC driver requirements may include information such as the host or IP address where alternate database resides, the port to connect to for the database, the name of database on the host, and other JDBC driver settings.

DataSrcTypTx — The type of the alternate data source. Valid values are as follows:

DB2	= IBM relational data source
JDAL	= ACI's Java Data Access Layer data source
JDAL2	= ACI's Java Data Access Layer 2 data source
MSSQL	= Microsoft SQL Server data source
ORACLE	= Oracle data source
SQLMX	= HP NonStop SQL/MX data source

UserID — The JDBC user ID needed to authenticate a connection to the data source. This user ID is used to securely access the underlying data source and is typically established by a database or system administrator. ACI applications use this user ID to access the data source.

UserPswdTx — The JDBC password needed to authenticate a connection to the data source. This user ID is used to securely access the underlying data source and is typically established by a database or system administrator and encrypted.

TblPrfxTx — The prefix, if one exists, to prepend to this alternate data source table name. This allows you to use multiple databases with the same underlying data source names prepended with a unique identifier for each database, thereby making the full data source names in each database unique across all of the application's databases.

MaxConns — The maximum number of concurrent database connections that the Database Connection Manager (DBConnectionManager) can have open at one time to this alternate data source.

StatConns — The number of connections the Database Connection Manager acquires to this data source when the Java server process is started. These connections remain active throughout the life of the process.

ConnWaitTime — The number of milliseconds the Database Connection Manager waits for another thread to free a connection to this data source. This value is used when the maximum number of connections has been reached as specified in the MaxConns field.

LastUpdtTm — The date and time, in MM/DD/YYYY hh:mm:ss AM/PM format, this row was last updated.

Example Configuration

The following is an example DATASRC configuration:

Field Name	Value
DataSrcNamTx	ALTDATASRC1
DrvrNamTx	com.ashna.jturbo.driver.Driver
DataSrcURLTx	jdbc:JTurbo://172.16.1.53:1433/Contextv10/sp=false
DataSrcTypTx	MSSQL
UserID	pserve
UserPswdTx	pserve
TblPrfxTx	null

Field Name	Value
MaxConns	50
StatConns	1
ConnWaitTime	10
LastUpdtTm	1/16/2003 1:01:45 PM

Application Configuration

The Application Configuration window or screen maintains property values in the APPLD. Using this window or screen, you can add, delete, or modify property values and descriptions. You can enter the name of a specific Java server process for a property or you can wildcard the process name so the property applies to all Java server processes. You can also enter a user-defined description for a property or modify the description provided with preloaded properties.

The Property ID field contains the name of the property.

Note: Property names are case-sensitive.

The Process ID field specifies a logical name for an executing Java Virtual Machine process and is used to differentiate one process from another in the configuration of an application. The value you enter in this field must either match the value of the process.id property specified in a JVM runtime argument or configuration properties file when the process was started or be a wildcard character, an asterisk (*), representing all JVM processes. For BASE24-es, the value in this field must also be defined in the Process Configuration data source (PROCD) before you can enter it here. Using this field, you can associate more than one process with a property having the same name but having a different value. This enables you to configure a process so that it runs differently than other processes using the same property.

The Property Value field contains the value to which a property is set.

Note: As with property names, property values are also case-sensitive.

Finally, the Property Description field enables you enter free-form user-defined text describing the property and its values. ACI provides descriptions for pre-loaded properties delivered from ACI.

External Connections Configuration

The External Connections Configuration window or screen maintains external connection configuration data in the EXTCND. Using this window or screen, you can add, delete, or modify external connections to external entities (e.g., client website, ATM, C++ process, etc). ACI JSF-based applications perform all data communications to external entities using entries in the EXTCND. External entity-specific formatters are used to build and parse messages to each of the external entities.

Currently, the following protocols are supported to connect to an external entity. Additional protocols may be added in the future as required.

- Enterprise JavaBeans (EJB)
- Hypertext Transfer Protocol (HTTP)
- Java Message Service (JMS) - Asynchronous
- Java Message Service (JMS) - Synchronous
- Pathway (PATHSEND) protocol*
- Simple Mail Transfer Protocol (SMTP)
- Transmission Control Protocol/Internet Protocol (TCP/IP)
- Transmission Control Protocol/Internet Protocol (TCP/IP) - Client
- Transmission Control Protocol/Internet Protocol (TCP/IP) - Server

* The PATHSEND protocol enables JSF-based applications to send Pathway messages to Pathway servers on the HP NonStop platform. JSF-based applications that use the PATHSEND protocol must provide an adapter and message formatter that can handle Pathway messages. In addition, the following JSF Java archive (.jar) files are required: framework-nsk.jar and common-nsk.jar.

The HP NonStop platform requires HP NonStop Server for Java Version 4.0 (NSJ4) or later and revision AAC (or later) of HP's JToolkit 2.0 for NonStop servers. JToolkit includes a .jar file (tdmext.jar) that must be included in the JSF-based application's classpath. It also contains a library that must be bound into the JVM.

Several fields in the EXTCND must be configured in accordance with the protocol used for the connection. These dependencies are described following a description of each of the fields in the EXTCND under the [“Protocol Configuration Options”](#) topic below.

The External Connection ID field contains a user-defined logical name for an external connection. The Process ID field specifies the logical name for an executing Java Virtual Machine process that uses this connection. The value you enter in this field must match the value of the process.id property specified in a JVM runtime argument or in the configuration properties file when the process was started. For BASE24-es, the value in this field must also be defined in the Process Configuration data source (PROCD) before you can enter it here.

General Configuration Options

Several general configuration options in the EXTCND describe the external connection, specify the formatter class used to build and parse messages for the connection, and configure the number of connections, timeout, and retry processing for the external connection.

The Connection Description field contains an optional user-defined description of the external connection. The Message Format Class field specifies the name of the invoked formatter class that builds request messages to and parses response messages from the external entity.

Note: The name of the invoked formatter class must be defined in the classpath of the ACI application or located in a JAR that is in the classpath.

The Timeout Limit field defines the maximum number of seconds that can elapse from the time a request is sent to an external entity to the time a response is received back the external entity.

The Initial Connections and Maximum Connections fields are used only when one of the TCP/IP protocols is used. The Initial Connections field defines the initial number of socket connections to the defined external entity that the application creates at startup. If more connections are needed as transaction volume grows, the application uses the value in the Maximum Connections field to determine the maximum number of additional connections that can be created.

The Retry Attempts field specifies the maximum number of times the application process tries to establish a connection with the external entity if a message failure occurs. A message failure can occur at the system or application level.

Protocol Configuration Options

The protocol options that need to be configured for an external connection depends entirely on the type of protocol used for the connection. The Protocol Code field specifies the type of protocol to be used. The following paragraphs describe each of the options that can or must be configured for each possible protocol.

Note: All option values entered in the Protocol Options field must start, end, and be separated with a period (.). For example, if you enter the HDR2 option by itself in the Protocol Options field, you would type “.HDR2.”. If you enter both the HDR2 and NOREPLY option in the Protocol Options field, you would type “.HDR2.NOREPLY.”.

TCP/IP

When using the TCP/IP, TCP/IP - Client, or TCP/IP - Server protocols, you must configure one of the following options in the Protocol Options field: HDR2 or R2LF. You can also optionally enter either or both of the following options in the Protocol Options field: EOT and NOREPLY. The options specified in this field define how TCP/IP data streams are tied to messages.

The HDR2 option indicates that the TCP/IP stream is prefaced with a two-byte binary message length. Note that this is a requirement for connecting to NET24-XPNET and BASE24-es C++ servers using TCP/IP for pass-through message processing. The R2LF option indicates to read until a line feed character is encountered.

The optional EOT option indicates whether the ACI application removes the End-of-text (EOT) character from response messages received from the external entity. The optional NOREPLY option indicates that the ACI application does not expect to receive a response message in reply to a request message. Therefore, it does not store any context for the request message.

In addition to the protocol options, you must specify the destination IP address and port number and the maximum number of connections allowed. The ACI application connects to the specified IP address and port number for sending requests. The Max. Connections Allowed field indicates the maximum number of connections that can be created as workload increases. A value of 0 or less in this field indicates that an unlimited number of connections can be created and they are acquired as they are needed. Any value greater than zero is initialized at process startup.

HTTP

When using the HTTP protocol, you can optionally set the Protocol Options field to a value of “.SSL” to use the Secure Sockets Layer (SSL). You can also optionally set the NOREPLY option to indicate that the ACI application does not expect to receive responses from the external entity. The Visa 3-D Secure connection for the ACI Commerce Gateway product requires the SSL option.

In addition to the protocol options, you must specify the destination IP address and port number and the maximum number of connections allowed. The ACI application connects to the specified IP address and port number for sending requests.

JMS-ASYNC

When using the JMS - Asynchronous protocol, you can optionally enter the NOREPLY option to indicate that the ACI application does not expect to receive responses from the external entity.

When connecting to a BASE24-es Integrated Server process, two additional options apply—MDSHDR and NONMQJMSTARGET. The MDSHDR option indicates to prepend a Message Delivery Subsystem (MDS) header to request messages. This option is required for all request messages sent to a BASE24-es Integrated Server process. The NONMQJMSTARGET option is also required when connecting to a BASE24-es Integrated Server process using WebSphere MQ. This option indicates to not include the MQRFH version 2 header (MQRFH2) in messages sent to an WebSphere MQ queue for a non-JMS application, such as the BASE24-es Integrated Server process.

In addition to the above protocol options, you must specify whether the ACI application uses a JMS connection type of QUEUE or TOPIC. Use the QUEUE connection type if the ACI application connects to the external entity using a JMS or WebSphere MQ queue. Use the TOPIC connection type if the ACI application connects to the external entity using a JMS topic. A JMS topic identifies a *publish/subscribe* destination type for a JMS server. Currently, the TOPIC value is reserved for future use. If you are using a JMS or WebSphere MQ queue to connect, you must also specify the name of the destination queue in the Destination Queue Name field, and, optionally, the reply queue in the Reply Queue Name field. If you are using the NOREPLY option, you can leave the Reply Queue Name field empty.

You can have multiple JSF application processes share the same external connection configurations by entering “{PROCESSID}” in the Reply Queue Name field. The “{PROCESSID}” syntax indicates that a response from the external entity is to be delivered to the specific process that sent the message. It indicates that the process ID of the process, as specified in the process.id JVM runtime argument or configuration properties file, is substituted as the reply queue name. For example, you can use the “{PROCESSID}” syntax for BASE24-es ATM Device Handler process connections to the BASE24-es Integrated Server process. All response messages from the BASE24-es Integrated Server process must be sent to the originating BASE24-es ATM Device Handler process for performance reasons because the transaction message context is maintained in memory instead of being written to disk.

JMS-SYNC

When using the JMS - Synchronous protocol, there are no protocol options. You only need to specify the JMS connection type and destination queue name as described above for the JMS-Asynchronous protocol.

EJB

When using an Enterprise JavaBeans (EJB) protocol, you must configure the name of the bean to connect to. This protocol is currently reserved for future use.

SMTP

When using the SMTP protocol, no protocol configuration options are configured.

PATHSEND

When using the PATHSEND protocol, no protocol configuration options are configured.

PATHSEND Protocol Configuration

The configuration requirements for adding an external connection to the EXTCND using the PATHSEND protocol are provided in the following table. The first column of the table lists the applicable fields on the External Connections Configuration window and the second column provides guidelines on how to set each applicable field for the PATHSEND protocol.

Field	Description
External Connection ID	User-defined connection ID.
Connection Description	User-defined text description of the connection (e.g., "PATHSEND - Send a message to an NSK Pathway server.").
Message Format Class	The message formatter class provided by the JSF-based application.
Timeout Limit	The amount of time (in seconds) to wait before the request is timed out.
Initial Connections	Not applicable.
Retry Attempts	The number of times a failed request should be retried before failing the request.
Maximum Connections	Not applicable.
Protocol Code	PATHSEND
Protocol Options	. (No additional protocol options.)
Protocol Field 1	The PPD name of the Pathmon process for the Pathway application.
Protocol Field 2	The name of the server class within the Pathway application. For example, this could be "SERVER-NCP" for sending requests to the Network Control Point (NCP) process.

Listener Configuration

The Listener Configuration window or screen maintains Listener configuration data in the LSTND. Using this window or screen, you can add, delete, or modify queues for which the ACI JSF application is to listen for messages received from an external entity (e.g., client website, ATM, C++ process, etc.) or another ACI JSF-based application. ACI JSF-based applications perform all data communications from external entities using entries in the LSTND. External entity-specific formatters are used to translate messages from each of the external entities. JMS internal listener types are used for one JSF-based application to communicate with another JSF-based application. JMS internal listener types assume that the message is composed of a DataObject serialized in XML format.

Currently, the following protocols are supported to connect to an external entity. Additional protocols may be added in the future as required.

- Java Message Service (JMS) External Request
- JMS External Response
- JMS Internal Request
- JMS Internal Response
- Transmission Control Protocol/Internet Protocol (TCP/IP)
- Transmission Control Protocol/Internet Protocol (TCP/IP) - Client
- Transmission Control Protocol/Internet Protocol (TCP/IP) - Server

Note: The LSTND is not used for the HTTP protocol. HTTP is the recommended protocol for most ACI Commerce Gateway installations due to its ability to easily traverse firewalls.

Several fields in the LSTND must be configured in accordance with the protocol used for the queue connection. These dependencies are described following a description of each of the fields in the LSTND under the [“Protocol Configuration Options”](#) topic below.

The Listener ID field contains the logical name of the Listener process for the connection. Listener processes use the value of the ListenerID field as the queue name they are to listen on. The Process ID field contains the logical name of the JSF-based application process to receive messages from this Listener process. You can associate multiple JSF-based application processes with a single Listener process. The value you enter in this field must match the value of the process.id

property specified in a JVM runtime argument when the process was started. For BASE24-es, the value in this field must also be defined in the Process Configuration data source (PROCD) before you can enter it here.

General Configuration Options

Several general configuration options in the LSTND describe the external or internal connection, specify the formatter class, method, and adapter used to build and parse messages for the connection, and configure the number of connections for the external or internal connection.

The Maximum Connections field specifies the maximum number of socket connections that the application allows to be created by another entity that is attempting to communicate with it. This value is used when a TCP/IP protocol is configured in the Listener Type field.

The Message Format Class field specifies the name of the invoked formatter class that builds request messages to and parses response messages from the external entity. For connections from another JSF-based application (i.e., the Listener Type field is set to a JMS-INTERNAL value), the `com.aciworldwide.common.CommonXMLFormatter` is used to translate messages; thus, you do not need to configure a formatter class in this field.

The Message Format Method field specifies the message format method to be invoked by the message format class specified in the Message Format Class field before sending or after receiving a message.

The Message Adapter Class field specifies the class name of the application adapter that is invoked by a request after the message formatter is invoked.

Note: The names entered in the Message Format Class, Message Format Method, and Message Adapter Class fields must be defined in the classpath of the ACI JSF-based application.

Protocol Configuration Options

The protocol options that need to be configured for a Listener process depends entirely on the type of listener used for the connection. The Listener Type field specifies the type of protocol to be used. The following paragraphs describe each of the options that can or must be configured for each possible protocol. Note that all option values entered in the Protocol Field Options field must start, end, and be separated with a period (.). For example, if you enter the HDR2 option by itself in

the Protocol Field Options field, you would type “.HDR2.”. If you enter both the HDR2 and NOREPLY option in the Protocol Field Options field, you would type “.HDR2.NOREPLY.”.

TCP/IP

When using the TCP/IP, TCP/IP - Client, or TCP/IP - Server protocols, you must configure one of the following options in the Protocol Field Options field: HDR2 or R2STR. You can also optionally enter one of the following options in the Protocol Field Options field: NOREPLY, SSL, or CLIENTAUTH. The options specified in this field define how TCP/IP data streams are tied to messages.

The HDR2 option indicates that the TCP/IP stream is prefaced with a two-byte binary message length. Note that this is a requirement for connecting to NET24-XPNET and BASE24-es C++ servers using TCP/IP for pass-through message processing. The R2STR option indicates to read until a specified text string character is encountered in the message. You must enter this text string in the Termination String field.

The optional NOREPLY option indicates that the client does not expect to receive a response message from the JSF-based application in reply to a request message. Therefore, it does not store any context for the request message.

The optional .SSL option indicates to use the Secure Sockets Layer (SSL) in communications between the client and the JSF-based application. With this option, the client authenticates the JSF-based application before making a connection.

The optional CLIENTAUTH option indicates whether the JSF-based application enforces SSL client authentication. With this option set, the JSF-based application authenticates the client before accepting a connection. When using the CLIENTAUTH option, the JSF-based application must have a database of valid client machines previously loaded.

JMS External

When using the JMS External Request or JMS External Response protocols, the Protocol Field Options field is displayed, but it is not used. You do not need to configure any options for these protocols.

JMS Internal

You do not need to configure any options for the JMS Internal Request or JMS Internal Response protocols.

Process Configuration

The Process Configuration window or screen maintains process configuration data in the PROCD. The PROCD is used when a JSF-based application needs to broadcast messages to other similar processes in a clustered configuration to keep cached application data in synchronization. If two or more JSF-based application processes (e.g., the BASE24-es Java User Interface process and BASE24-es ATM Device Handler processes) access one or more of the same cached data sources, they must be configured in the PROCD so that the shared cached data remains synchronized.

The Process ID field contains the logical name of the JSF-based application and must match the value of the process.id property specified when the application was started.

The External Connection ID field specifies the key to an EXTCND row that defines the internal protocol mechanism used to send system configuration messages to processes configured in a cluster for an application. The value in this field must match a value in the External Connection ID field in the EXTCND.

The Process Description field is a user-defined text description of this JSF-based application process.

Note: On the ACI desktop user interface, process IDs must be configured in the PROCD before they are displayed in selectable lists on the following windows:

- Application Configuration
- Configuration Properties
- External Connection Configuration
- Listener Configuration
- System Monitor
- System Properties

For the ACI desktop user interface, you must enter the same value in the External Connection ID column that you enter in the Process ID column.

ACI Worldwide Inc.

Section 4

Administration

All ACI JSF-based applications provide facilities to perform the following administrative functions:

- View and clear objects cached in the Cache Manager within the executing Java Virtual Machine (JVM) process
- Stop and start TCP/IP sockets
- Reload application-level properties after making changes to the Application Configuration data source (APPLD) and view system properties
- Monitor database usage
- Monitor system resource usage
- Log and monitor events
- Trace JSF application processing

This section provides guidelines and procedures for each of the above functions.

Cache Administration

The Cache Management feature of the JSF allows fast access to objects that would otherwise take a long time to access. It involves keeping a copy of objects that are expensive to construct in cache memory after the immediate need for the object is over. The object can be expensive to construct for any number of reasons, such as requiring a lengthy computation or being *fetched* from a data source on disk.

When a data source is cached, the JSF-based application attempts to retrieve data from cache memory. If the row needed is not present in cache memory, the row is read from the data source on disk and is stored in cache memory. The row remains in the cache memory pool until the Cache Manager automatically cleans up cache pools on a regular basis such as nightly or even hourly (as configured in the appropriate Java server property) or you manually clear cache through the Cache Administration window or screen.

The Service Name field on the Cache Administration window or screen identifies a database service that accesses a data source that is cached. See the `db.audited.db-service-name` property description in appendices to this user guide for a list of all possible database service names that you can configure to be cached for the application.

The Object Key field on the Cache Administration window or screen contains the key to each cached object for each database service. Key values separated by a colon indicate two sets of comma-delimited values:

- Values before the colon are the key column values used to read the cached row.
- Values after the colon are the key column values that were previously used to read the cached row.

Note: An object key of `FetchAllObj` indicates that an application performed a Fetch All function on a database service that was configured for caching in a `db.cached.db-service-name` property. This object contains all database table rows retrieved from the data source.

The Objects cached field displays the total number of objects cached by the Cache Manager within the executing Java Virtual Machine (JVM) process.

When you click the Clear Cache button on the Cache Administration window or screen, all objects currently cached are removed from the cache memory pool.

TCP/IP Socket Administration

Using the Socket Administration window or screen, you can stop and start inbound and outbound TCP/IP sockets for both clients and servers. Separate tabs are used to display inbound client, outbound client, inbound server, and outbound server sockets.

Because client sockets are remote to the JSF-based application, they are identified by both a host or IP address and a port number. Server sockets are local to the JSF-based application and are identified with a port number only.

All outbound sockets are used for external TCP/IP connections configured in the EXTCND. All inbound sockets are used for TCP/IP connections configured in the LSTND.

Outbound connections configured with the TCP/IP or TCP/IP-CLIENT protocol in the EXTCND are displayed on the Client Outbound tab of the Socket Administration window, while outbound connections configured with the TCP/IP-SERVER protocol in the EXTCND are displayed on the Server Outbound tab.

Inbound connections configured with the TCP/IP or TCP/IP-CLIENT protocol in the LSTND are displayed on the Client Inbound tab of the Socket Administration window, while inbound connections configured with the TCP/IP-SERVER protocol in the LSTND are displayed on the Server Inbound tab.

Thus, the host or IP addresses and port numbers displayed on the Socket Administration window reflect all TCP/IP connections configured in the EXTCND and LSTND. If there are multiple connections in the EXTCND or LSTND for the same host or IP address and port number, you will see the same host or IP address and port number listed multiple times on the Socket Administration window, once for each configured connection.

For inbound sockets, the Socket Administration window or screen displays the number of sockets that were initiated remotely by a client or server for a connection. These are displayed in the Remote Connections column. In the server environment configured in the LSTND, multiple client sockets can be established on a given TCP/IP port. The number of remote connections is a *snapshot*, at the time you accessed the Socket Administration window or refreshed the view, of the number of these sockets that have been accepted on the port.

For outbound sockets, the Socket Administration window or screen displays the number of initiated TCP/IP connections that are free. These are displayed in the Free Connections column. The maximum number of connections (sockets) to allow for a host/IP address and port combination is configured in the Max.

Connections Allowed field. The number of free connections is a *snapshot*, at the time you accessed the Socket Administration window or refreshed the view, of the number of these sockets that are free (i.e., not currently in use by a thread of the JSF application). In the JSF, sockets (for each host/IP address and port combination) are pooled; allowing them to be shared among multiple threads.

For all socket connections, the Socket Administration window or screen displays the status of the socket, which can be Connected or Not Connected. You can stop a socket by changing the status from Connected to Not Connected and clicking Submit (▶). You can start a socket by changing the status from Not Connected to Connected and clicking Submit (▶).

The Total Connections field displays the total number of started TCP/IP connections. Note that if multiple connections are made to the same host or IP address and port, they are only counted once.

Property Administration

The JSF allows you to view all properties currently in use by a Java server process. On the Configuration Properties window, you can view all of the properties defined to a Java server process in either the config.properties file or the Application Configuration data source (APPLD). Any changes you make to properties in the APPLD from the Application Configuration window do not take effect in processing until you click the Reload button on the Configuration Properties window. When you click the Reload button, the Java server process reads the APPLD and reloads the property values into memory.

On the System Properties window, you can view all of the system-level properties currently defined to a Java server process.

To change properties in the config.properties file, you must use a text editor to change the property values in the file and then stop and restart all Java server processes that use the file.

To change system properties specified in the JVM runtime arguments, you must stop the Java server process and restart it with revised JVM runtime argument values.

To change system properties inherited from the operating system, refer to the vendor-specific operating system documentation.

Database Administration

The Database Administration window or screen enables you to view the Java DataBase Connectivity (JDBC) driver used to connect to the database, the number of clients currently using the driver, the user ID used to authenticate a connection to the database, the total number of connections, and the number of free connections. This window essentially provides a *snapshot* of the current database usage for any given Java server process. You can use this information to monitor database workload requirements and adjust database property values in the config.properties file as needed.

The JDBC data source driver is configured in the db.driver property. You can change the value of this property to access different types of data sources. The current value of the db.driver property in the config.properties file is displayed in the JDBC Driver Name field on the Database Administration window.

The Number of Clients field displays the current number of Java server process execution threads that are accessing the database through the JDBC driver. This number is dynamic as a thread removes a connection from the pool to perform a database access operation and then returns it to the pool when done. As work loads increase, this number will probably also increase.

The Data Source Name field displays the data source connection information required by a specific JDBC driver for a specific database. This is the data source URI specification that allows the JDBC driver to access the correct database and is configured in the db.datasource property in the config.properties file. Refer to the [db.datasource](#) property description in section 5 for more information.

The User ID field displays the user ID that Java server processes use to authenticate a connection to the database. This value is configured in the db.user.id property in the config.properties file.

The Total Connections field displays the current number of active connections from the Java server process to the database, including any static connections. Static connections are configured in the db.static.connections property in the config.properties file and remain active throughout the life of the Java server process. Static connections allow all the expense of connecting to the database to be incurred only during process startup.

Note: The number of current connections increases as work load increases and execution threads require connections that have not been established. The Java server process dynamically allocates more connections up to the maximum number configured in the db.max.connections property in the config.properties file.

Finally, the Free Connections field displays the current number of inactive connections available for the Java server process from the connection pool. This number is calculated by subtracting the number of current active connections from the maximum number of connections configured in the `db.max.connections` property in the `config.properties` file.

System Monitoring

The System Monitoring window enables you to view the current use of all JSF system resources by a Java server process from a single window. From this window, you can view a summary of the following:

- Total number of active and free database connections and threads of execution (clients) currently accessing the database.
- Total number of TCP/IP socket connections and free TCP/IP socket connections as well as the total number of clients currently connected to the Java server process. The free (available) connections is calculated by subtracting the number of current connections from the maximum connections configured in the EXTCND and LSTND.
- Java config.properties file, log4j configuration file for generating event messages, log4j configuration file for tracing, and a flag indicating whether trace is currently enabled for the Java server process (i.e., the value of the trace.enabled property).
- The total number of objects currently cached by the Java server process.
- The total number of bytes in memory currently used by the Java server process and available in the operating system.
- The number of current active execution threads for the Java server process.

Because the System Monitoring window provides a summary of all JSF system resource usage, it also contains hyperlinks to the following windows for a more detailed view of system usage by the Java server process:

- Database Administration window
- Socket Administration window
- Configuration Properties window
- Cache Administration window

Event Administration

Events generated by JSF applications are logged according to the log4j configuration in the EventConfig.xml file specified by the event.log4j.config.url or event.config properties. The event.config property allows you to use substitution variables when specifying the location of the EventConfig.xml file in the config.properties file. A substitution variable is identified by a starting value of “\${” and a closing value of “}”. The value between the braces is used as a property key to search for a value to substitute from the JVM system properties, and if not found, from the entries in the config.properties file or Application Configuration data source (APPLD). This same functionality is available inside the EventConfig.xml file for specifying file locations. Thus, to specify a process-specific EventConfig.xml file located in a /config directory under the current directory, you could specify:

```
event.config=${user.dir}${file.separator}${process.id}_EventConfig.xml
```

Each of the substitution variables specifies a system, configuration (config.properties), or application (APPLD) property. The JSF substitutes the values of these properties to resolve the file location.

Also, within the EventConfig.xml file you could specify events to go to a process-specific file as shown in the following example.

```
<appender name="EVENT" class="com.aciworldwide.framework.log.
DailyRollingFileAppenderExt">
  <param name="File" value="${user.dir}${file.separator}${process.id}_Event.
log"/>
  <!-- Solaris Example: <param name="File" value="/logs/Events.log"/> -->
  <param name="DatePattern" value="'.'yyyy-MM-dd"/>
  <layout class="org.apache.log4j.PatternLayout">
    <param name="ConversionPattern" value="%d{ISO8601} %-5p %c [%t] %x -
%m%n"/>
  </layout>
</appender>
```

The EventConfig.xml file specifies the Appender and Layout components to be used for event messages generated by JSF-based Java server processes. The Appender component specifies the output location for event messages while the Layout component specifies the format of event messages. The example EventConfig.xml file shown below contains different appenders for several of the

platforms on which JSF-based ACI applications are deployed. The appropriate appender aggregates need to be uncommented and the values enclosed in braces {} must be edited for site-specific values during installation.

The appender configuration for an IBM Mainframe zSeries operating system platform is denoted by IBMZ comments and requires a site-specific value for the {ibmz_port} variable.

The appender configuration for an HP NonStop platform is denoted by EMS comments and requires a site-specific value for the {collector} variable.

The appender configuration for a Unix platform is denoted by UNIX comments and requires site-specific values for the {jms_url} and {jms_queue} variables.

Additional aggregates and variables may need to be uncommented or defined as required for your site.

A sample EventConfig.xml file is shown below.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE log4j:configuration SYSTEM "log4j.dtd">
<log4j:configuration debug="false">

  <!-- Appender for the EMS log.-->
  <!-- Comment out the element if not needed and remove EVENTFILE from category
element below -->
<!--EMS
  <appender name="EMSAPPENDER" class="com.aciworldwide.framework.log.
EMSAppender">
    <param name="Collector" value="{collector}"/>
    <layout class="org.apache.log4j.PatternLayout">
      <param name="ConversionPattern" value="%d{ISO8601} %-5p %c [%t] %x - %m%n"/>
    </layout>
  </appender>
EMS-->

<!-- IBMZ
  <appender name="IBMZAPPENDER" class="com.aciworldwide.framework.log.
ACITCPAppender">
    <param name="Port" value="{ibmz_port}"/>
    <layout class="com.aciworldwide.framework.log.ACISidistLayout">
      </layout>
    </appender>
IBMZ-->

<!--UNIX
```

```

    <appender name="UNIXAPPENDER" class="com.aciworldwide.framework.log.
JMSMQAppender">
        <param name="ProviderUrl" value="{jms_url}"/>
        <param name="QueueConnectionFactoryBindingName"
value="JmsQueueConnectionFactory"/>
        <param name="QueueName" value="{jms_queue}"/>
        <param name="InitialContextFactory" value="com.sun.jndi.fscontext.
RefFSContextFactory"/>
        <layout class="org.apache.log4j.PatternLayout">
            <param name="ConversionPattern" value="%d{ISO8601} %-5p %c [%t] %x -
%m%n"/>
        </layout>
    </appender>
UNIX-->

<!-- Appender for the system console (i.e., System.out) -->
<appender name="SYSOUT" class="org.apache.log4j.ConsoleAppender">
    <param name="Target" value="System.out"/>
    <layout class="org.apache.log4j.PatternLayout">
        <param name="ConversionPattern" value="%d{ISO8601} %-5p %c [%t] %x -
%m%n"/>
    </layout>
</appender>

<!-- Appender for the system error console (i.e., System.err) -->
<appender name="SYSERR" class="org.apache.log4j.ConsoleAppender">
    <param name="Target" value="System.err"/>
    <layout class="org.apache.log4j.PatternLayout">
        <param name="ConversionPattern" value="%d{ISO8601} %-5p %c [%t] %x -
%m%n"/>
    </layout>
</appender>

<!-- Appender for the Error log files. -->
<appender name="EVENT" class="com.aciworldwide.framework.log.
DailyRollingFileAppenderExt">
    <!--
        For the following line, change {root} to the location of where the
ESWebEvents log file
        should be placed.

        DOS specifier - change {root} to drive: or drive:/{other directory} to
                        be prefixed to value below (Examples - C:/ESLogs, C:/
ESWeb/ESLogs)

        UNIX specifier - change {root} to root location to be prefixed to value
                        below (Examples - db01/ESLogs, db01/ESWeb/ESLogs)
    -->
    <param name="File" value="{root}/ESWebEvents.log"/>
    <param name="DatePattern" value="'. 'yyyy-MM-dd"/>
    <layout class="org.apache.log4j.PatternLayout">

```

```
<param name="ConversionPattern" value="%d{ISO8601} %-5p %c [%t] %x -
%m%n"/>
</layout>
</appender>

<!-- Appender for the CRITICAL Events files. -->
<appender name="CRITICAL_EVENTS" class="com.aciworldwide.framework.log.
DailyRollingFileAppenderExt">
  <param name="Threshold" value="FATAL"/>
  <!--
    For the following line, change {root} to the location of where the
    ESWebCriticalEvents log file
    should be placed.

    DOS specifier - change {root} to drive: or drive:/{other directory} to
    be prefixed to value below (Examples - C:/ESLogs, C:/
    ESWeb/ESLogs)

    UNIX specifier - change {root} to root location to be prefixed to value
    below (Examples - db01/ESLogs, db01/ESWeb/ESLogs)
  -->
  <param name="File" value="{root}/ESWebCriticalEvents.log"/>
  <param name="DatePattern" value="'. 'yyyy-MM-dd"/>
  <layout class="org.apache.log4j.PatternLayout">
    <param name="ConversionPattern" value="%d{ISO8601} %-5p %c [%t] %x -
    %m%n"/>
  </layout>
</appender>

<appender name="NULL" class="com.aciworldwide.framework.log.NullAppender">
  <layout class="org.apache.log4j.SimpleLayout"/>
</appender>

<!-- The category used by Error Manager. All event messages are logged under
this category. -->
<!-- ErrorManager automatically prepends this category name to all requests. -
->
<!-- This example illustrates sending CRITICAL events to a different
destination. In this case, -->
<!-- it is to a different file; but it could be, for example, an SMTPAppender
(to send an E-mail alert). -->

<category name="event" additivity="false">
  <priority value="info"/>
<!--EMS
  <appender-ref ref="EMSAPPENDER"/>
EMS-->
<!--IBMZ
  <appender-ref ref="IBMZAPPENDER"/>
IBMZ-->
<!--UNIX
  <appender-ref ref="UNIXAPPENDER"/>
```

```

UNIX-->
    <appender-ref ref="EVENT"/>
    <appender-ref ref="SYSOUT"/>
    <appender-ref ref="CRITICAL_EVENTS"/>
</category>

<!-- The required root category. Because the 'event' category has it's-->
<!-- additivity flag set to false, nothing should use this category. -->
<root>
    <priority value="debug"/>
    <appender-ref ref="SYSOUT"/>
</root>
</log4j:configuration>

```

Event Message Documentation

Event messages that can be generated by JSF applications, such as BASE24-es ATM Device Handler and IFX Manager processes and the BASE24-es Java User Interface servlets, are documented on the Event Message and Event Message Documentation tabs of the Integrated Server Event Configuration and Management window of the ACI desktop. JSF-specific messages are identified with subsystem IDs in the range of 3000–3065, while event messages generated by application-specific components in BASE24-es ATM Device Handler or IFX Manager processes and the BASE24-es Java User Interface servlets are identified with subsystem IDs in the range of 5500–5999.

Note: JSF-based applications do not use tokens in event messages. Therefore, you cannot use token IDs (e.g., <1>, <2>, etc.) for text substitution in event messages generated by JSF-based applications. JSF-based applications use braces ({}) instead of tokens to denote text substitution in event messages. Brace-based text substitution is hard-coded in JSF-based application code.

Additional References

You may find the following additional references useful in implementing log4j:

- Short introduction to log4j (<http://jakarta.apache.org/log4j/docs/manual.html>)
- Quick start to using log4j (<http://www.vipan.com/htdocs/log4jhelp.html>)
- Available appenders (<http://jakarta.apache.org/log4j/docs/api/index.html>)

Trace Administration

The JSF provides two types of tracing—normal and JDAL.

Note: The Trace facility was designed as a diagnostic tool for development and test environments and not for general use in a production environment. ACI strongly recommends that it not be used in a production environment due to the overhead and impacts on performance.

Normal Tracing

The normal tracing of XML messages sent and received by a Java server process is enabled with two application-level properties—`trace.enabled` and `trace.dataobject.enabled`. These properties are configured in the Application Configuration data source (APPLD). Note that database tracing is included when performing a normal trace.

The `trace.enabled` property indicates whether normal tracing is performed. A value of `true` for this property enables normal tracing while a value of `false` disables it. The `trace.dataobject.enabled` property indicates whether trace messages are enabled for data object output from the Java server process when message tracing is enabled. When this property is set to `true`, the `MessageSchema` class dumps traces of the contents of the primary data object during request message processing and response message building.

Examples of normal trace messages, including database trace messages, are shown below.

```
2004-08-30 12:25:44,012 trace.cache.put.com.aciworldwide.framework.CacheManager
com.aciworldwide.framework.CacheManager.put(Unknown Source) •mainÛ - [04.1.0 -
Build 20040712-1016] Service: EXTRCNCT, category: DATABASE, key:
selextrcnctprocessid:1:JDH1 written to cache, a new service map entry was added
2004-08-30 12:25:44,013 trace.database.content.EXTRCNCT.com.aciworldwide.
common.DBService com.aciworldwide.common.DBService.traceFetch(Unknown Source)
•mainÛ - [04.1.0 - Build 20040712-1016] Fetch OK, Service Name: EXTRCNCT,
Statement ID: selextrcnctprocessid
Statement: SELECT ExtrCnctID, PrcsID, DscrTx, MssgFrmtClasTx, PrtcCd,
PrtcOptnList, PrtcDfnd1Tx, PrtcDfnd2Tx, PrtcDfnd3Tx, TimeOutNum, RtryAtmpNum,
InitConnNum, MaxConnNum, LastUpdtTm FROM extrcnct WHERE PrcsID = ?
Key Field 1: process_id_str Value: JDH1
2004-08-30 12:25:44,015 trace.database.content.EXTRCNCT.com.aciworldwide.
common.DBService com.aciworldwide.common.DBService.traceFetch(Unknown Source)
•mainÛ - [04.1.0 - Build 20040712-1016] Fetch Result row #1:
```

```

Field: external_connection_id_str Value: ATMSRPLY
Field: process_id_str Value: JDH1
Field: description_str Value: queue for sending out to atm
Field: message_format_class_str Value: com.aciworldwide.base24.es.common.
formatter.ResponseFormatterController
Field: protocol_code_str Value: JMS-ASYNC
Field: protocol_option_list_str Value: .MDSHDR.NONMQJMSTARGET.
Field: protocol_field_1_str Value: QUEUE
Field: protocol_field_2_str Value: ATMSRPLY
Field: protocol_field_3_str Value: {TERMINID}
Field: timeout_int Value: 300
Field: retry_attempts_int Value: 3
Field: initial_connections_int Value: 1
Field: maximum_connections_int Value: 1
Field: last_update_time Value: 2004-04-15 13:30:11.438

```

JDAL Tracing

The ACI-written Java Database Access Layer (JDAL) (type 2) and Java Database Access Layer 2 (JDAL2) (type 3) database drivers allow you perform tracing of JDAL database functions. The `jdal.trace` system property indicates whether JDAL tracing is performed. A value of `true` for this property enables JDAL tracing while a value of `false` disables it. The `jdal.trace.method` system property indicates whether JDAL methods are traced when JDAL tracing is enabled. A value of `true` for this property enables JDAL method tracing while a value of `false` disables it.

Both of these properties are system properties specified in JVM runtime arguments. To ensure that JDAL tracing is disabled in a production environment, both of these properties should be set to `false` as shown below:

```

-Djdal.trace=false
-Djdal.trace.method=false

```

Examples of JDAL trace messages are shown below.

```

2004-08-30 12:25:43.993[main] JDAL (Trace) Sending SIDAL String: <GETNEXT>=<stmt
id="1718264953" typ="SELECT"> </stmt>
2004-08-30 12:25:43.998[main] JDAL (Trace) Receiving SIDAL String: Result =
ExtrCnctID='ESAUTH',PrclsID='JDH1
',DscrTx='ConnectiontotheBASE24-esISO93Authorizer
',MssgFrmtClasTx='com.aciworldwide.base24.es.common.formatter.iso8583.es.
FormatterController
',PrtcCd='JMS-ASYNC',PrtcOptnList='.
MDSHDR.NONMQJMSTARGET.
',PrtcDfnd1Tx='QUEUE

```

```
' ,PrtcDfnd2Tx='ATMISO
' ,PrtcDfnd3Tx='{PROCESSID}
' ,TimeOutNum='300' ,RtryAtmpNum='3' ,InitConnNum='1' ,MaxConnNum='1' ,LastUpdtTm='13
3013280529380000';
2004-08-30 12:25:43.999[main] JDAL (Trace) Statement: <stmt id="1718264953"
typ="SELECT"> </stmt> Error Code: jdal_ok Result More value: eof
2004-08-30 12:25:43.999[main] JDAL (Trace) Col (0) ExtrCnctID = ESAUTH
2004-08-30 12:25:43.999[main] JDAL (Trace) Col (1) PrcsID = JDH1
2004-08-30 12:25:44.000[main] JDAL (Trace) Col (2) DscrTx = Connection to the
BASE24-es ISO93Authorizer
2004-08-30 12:25:44.000[main] JDAL (Trace) Col (3) MssgFrmtClasTx = com.
aciworldwide.base24.es.common.formatter.iso8583.es.FormatterController
2004-08-30 12:25:44.001[main] JDAL (Trace) Col (4) PrtcCd = JMS-ASYNC
2004-08-30 12:25:44.004[main] JDAL (Trace) Col (5) PrtcOptnList = .MDSHDR.
NONMQJMSTARGET.
2004-08-30 12:25:44.004[main] JDAL (Trace) Col (6) PrtcDfnd1Tx=QUEUE
2004-08-30 12:25:44.005[main] JDAL (Trace) Col (7) PrtcDfnd2Tx=ATMISO
2004-08-30 12:25:44.005[main] JDAL (Trace) Col (8) PrtcDfnd3Tx='{PROCESSID}
2004-08-30 12:25:44.006[main] JDAL (Trace) Col (9) TimeOutNum = 300
2004-08-30 12:25:44.007[main] JDAL (Trace) Col (10) RtryAtmpNum = 3
2004-08-30 12:25:44.007[main] JDAL (Trace) Col (11) InitConnNum = 1
2004-08-30 12:25:44.008[main] JDAL (Trace) Col (12) MaxConnNum = 1
2004-08-30 12:25:44.009[main] JDAL (Trace) Col (13) LastUpdtTm =
133013280529380000
```

Trace Output Configuration

All trace messages are written to the logging location defined in the log4j configuration in the TraceConfig.xml file specified by the trace.log4j.config.url or trace.config properties. The trace.config property allows you to use substitution variables when specifying the location of the TraceConfig.xml file in the config.properties file. A substitution variable is identified by a starting value of “\${” and a closing value of “}”. The value between the braces is used as a property key to search for a value to substitute from the JVM system properties, and if not found, from the entries in the config.properties file. This same functionality is available inside the TraceConfig.xml file for specifying file locations. Thus, to specify a process-specific TraceConfig.xml file located in a /config directory under the current directory, you could specify:

```
event.config=${user.dir}${file.separator}${process.id}_TraceConfig.xml
```

Each of the substitution variables specifies a system property. The JSF substitutes the values of these properties to resolve the file location.

Also, within the TraceConfig.xml file you could specify trace data to go to a process-specific file as shown in the following example.

```
<appender name="TRACE" class="org.apache.log4j.FileAppender">
  <param name="File" value="${user.dir}${file.separator}${process.id}_
Trace.log"/>
  <param name="Append" value="false"/>
  <layout class="org.apache.log4j.PatternLayout">
    <param name="ConversionPattern" value="%d{ISO8601} %l [%t] %x -
%m%n"/>
  </layout>
```

The TraceConfig.xml file specifies the Appender and Layout components to be used for trace messages generated by JSF-based Java server processes. The Appender component specifies the output location for trace messages while the Layout component specifies the format of trace messages. You can uncomment the Appender aggregate to be used for your system.

A sample TraceConfig.xml file is shown below. The File and DatePattern params specify the path and name of the trace output file. For this sample, the path and name would be: C:/CommerceFramework/Logs/trace.yyyy-MM-dd.log, where yyyy-MM-dd is the date the trace was started.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE log4j:configuration SYSTEM "log4j.dtd">
<log4j:configuration debug="false">
  <!-- Appender for the system console for trace messages -->
  <appender name="SYSOUT" class="org.apache.log4j.ConsoleAppender">
    <param name="Target" value="System.out"/>
    <layout class="org.apache.log4j.PatternLayout">
      <param name="ConversionPattern" value="%d{ISO8601} %c %l [%t] %x -
%m%n"/>
    </layout>
  </appender>

  <!-- Appender for trace files for messages. -->
  <appender name="TRACE" class="com.aciworldwide.framework.log.
DailyRollingFileAppenderExt">
    <param name="File" value="C:/CommerceFramework/Logs/trace.log"/>
    <!-- Solaris Example: <param name="File" value="/opt/CommerceFramework/Logs/
trace.log"/> -->
    <param name="DatePattern" value="'. 'yyyy-MM-dd"/>
    <layout class="org.apache.log4j.PatternLayout">
      <param name="ConversionPattern" value="%d{ISO8601} %c %l [%t] %x -
%m%n"/>
    </layout>
  </appender>
```

```
<appender name="NULL" class="com.aciworldwide.framework.log.NullAppender">
  <layout class="org.apache.log4j.SimpleLayout"/>
</appender>

<!-- The category used by Trace Manager. All trace messages are logged under
this category. -->
<!-- TraceManager automatically prepends this category name to all requests.
For TraceManager, -->
<!-- the priority should be left at 'debug'. TraceManager always logs
messages with this priority. -->
<!-- In this example, trace messages are sent to both a trace file (identified
by an appender named -->
<!-- TRACE, and to the system error console. To only send message to the
trace file, simply remove -->
<!-- the line referring to SYSOUT -->
<category name="trace" additivity="false">
  <priority value="debug"/>
  <appender-ref ref="TRACE"/>
  <appender-ref ref="SYSOUT"/>
</category>

<!-- The following two categories illustrate one method to enable selective
tracing. Only categories whose -->
<!-- name starts with 'trace.test' will be logged to the trace file and the
system console. -->
<!-- This is accomplished by setting the priority of category named 'trace'
to 'fatal'. Note that this -->
<!-- works only for tracing because TraceManager always uses the 'debug'
priority. -->
<!-- <category name="trace.test" additivity="false">
  <priority value="debug"/>
  <appender-ref ref="TRACE"/>
  <appender-ref ref="SYSOUT"/>
</category>
<category name="trace" additivity="false">
  <priority value="fatal"/>
  <appender-ref ref="TRACE"/>
  <appender-ref ref="SYSOUT"/>
</category> -->
<!-- The required root category. Because the 'trace' category has it's-->
<!-- additivity flag set to false, nothing should use this category. -->
<root>
  <priority value="debug"/>
  <appender-ref ref="SYSOUT"/>
</root>
</log4j:configuration>
```

Section 5

Configuration Properties

Java servers built using the JSF are configured using properties configured at the system level, in a configuration properties file read at startup, or in the Application Configuration data source. This section describes each of the properties common to ACI applications built on the JSF. Properties specific to an ACI application built on the JSF are described in an appendix to this user guide or in an application-specific manual.

Java server properties are defined for ACI applications when the application is installed. While most properties can remain set to their installed values, some may require adjustment after installation to meet changing transaction processing requirements.

System-Level Properties

This subsection describes the minimum system properties required for Java server processes built on the JSF, as well as optional system properties. Application-specific system properties are described in the application-specific appendices of the user guide. Some system properties are set at runtime while others are propagated from the JVM on which the Java server process is run or from the user logged onto the JVM. You can view all the system-level properties for a Java server process on the System Properties window or screen.

Required System Properties

The following system properties are required for JSF-based applications.

config.filename — The fully-qualified location of the config.properties file (e.g., /user/B24es_cur/ESWeb/ESConfig/uaf2/P6Q^GATE/config.properties). This is a runtime system property that is either set with the -D flag argument when a Java server process is started as shown in the following example, or defaults to a file named “config.properties”.

```
JAVA -Dconfig.filename=/user/B24es_cur/ESWeb/ESConfig/uaf2/P6Q^GATE/config.properties
```

file.encoding — The character encoding set used for the default locale (e.g., ISO8859-1).

file.separator — The platform-dependent name or file separator character (e.g., / on UNIX). ACI recommends that you set this property to / since it is valid on a variety of hardware platforms (e.g., Unix, HP NonStop, Windows, etc).

process.id — The symbolic name assigned to a Java server process (e.g., P6Q^ESWEB01). This is a runtime system property that is either set with the -D flag argument when a Java server process is started or is set in the config.properties file. If present in both places, the value of the -D flag argument takes precedence. This property is used by several components and services in the JSF. The value of this property must match the symbolic name configured for the Java server process on the Process Configuration window.

Required:	Yes
Example Entry:	JAVA -Dprocess.id=JDH2
Usage:	CommonXMLFormatter DBConnectionManager ExternalConnectionComponent FrameworkAdmin JMSMessageRouterService MessageRouterService MessageService EventLogAppender LogManager TimerTaskManager JMSUtils

System Property Configuration

The `file.encoding` and `file.separator` system properties are set by the JVM. Several options are available for configuring the process ID and configuration filename system properties as discussed below.

For a JSF application that is not running as a Web server, the process ID and configuration filename properties are set as follows. The process ID can be set in a `-D` option JVM runtime argument or it can be set inside the configuration properties file. If it is set in both locations, the `-D` argument value takes precedence. The configuration filename property must either be set when a Java server process is started using the `-D` option JVM runtime argument (`-Dconfig.filename`) or it defaults to a file named “`config.properties`”.

For Web-based JSF applications, the process ID and configuration filename servlet initialization parameters may be retrieved from the `web.xml` file instead.

Non-Web Application

When the configuration filename property is specified by the `-Dconfig.filename` argument, the following rules apply for resolving the path to the configuration file. You can specify an absolute path (e.g., `c:/config/myconfig.properties`) or a relative path (e.g., `../config/myconfig.properties`) to the configuration file. If you specify a relative path, the path is resolved based on the current working directory of the JVM. You can also use the following shorthand notations in the argument value to specify the JVM’s classpath to search for the configuration file or the home directory of the user who started the JVM.

- For the classpath, you can use “{classpath}/” or “\${classpath}/” to represent the classpath in the argument value (e.g., “{classpath}/myconfig.properties” or “\${classpath}/myconfig.properties”). Note that the argument value must start with “{classpath}/” or “\${classpath}/” for this shorthand substitution notation.
- For the home directory, you can use “{user.home}/” or “\${user.home}/” to represent the home directory of the user who started the JVM (e.g., “{user.home}/myconfig.properties” or “\${user.home}/myconfig.properties”). In a Windows environment, both of these examples are expanded to “C:/Documents and Settings//myconfig.properties”. Note that the argument value must start with “{user.home}/” or “\${user.home}/” for this shorthand substitution notation.

When the configuration filename property is not specified by the -Dconfig.filename argument, the JSF uses a default file name of “config.properties” and searches for this file in the following order:

1. The current working directory.
2. The classpath.
3. The user’s home directory.

Web Application

For JSF applications running as Web servers, the process ID and configuration properties filename can be retrieved from the process.id and config.filename servlet initialization parameters, respectively, in the web.xml file. The following is an example of the servlet initialization parameters for the InitializationHTTPServlet.

```
<servlet>
  <servlet-name>InitializationHTTPServlet</servlet-name>
  <display-name>Initialization</display-name>
  <servlet-class>com.aciworldwide.framework.InitializationHTTPServlet</
servlet-class>
  <init-param>
    <param-name>process.id</param-name>
    <param-value>MyProcessId</param-value>
  </init-param>
  <init-param>
    <param-name>config.filename</param-name>
    <param-value>/WEB-INF/config/myconfig.properties</param-value>
  </init-param>
```

```
<load-on-startup>100</load-on-startup>
```

If the value of the `config.filename` parameter begins with a slash character (“/”), the file location is assumed to be absolute (relative to the Web application’s context root. If the value of the `config.filename` parameter does not begin with a slash character, the JSF searches for the file in the following locations in the order specified.

1. `/WEB-INF/conf`
2. `/WEB-INF/config`
3. `/WEB-INF`
4. Use the servlet’s classloader

If the `config.filename` parameter is not specified, the JSF searches for a file named “`config.properties`” in the locations and order specified above. If the JSF still cannot find the configuration file, it searches for the file using the “[Non-Web Application](#)” method described above.

Optional System Properties

The following system properties are optional for JSF-based applications.

jdal.trace — A flag indicating whether the ACI-written driver, Java Database Access Layer (JDAL) performs database tracing. Valid values are as follows:

`true` = Yes, JDAL performs database tracing.
`false` = No, JDAL does not perform database tracing. Default.

Note: The Trace facility was designed as a diagnostic tool for development and test environments and not for general use in a production environment. ACI strongly recommends that it not be used in a production environment due to the overhead and impacts on performance. Therefore, you should only enter this property in a `-D` JVM runtime argument when instructed by ACI for problem resolution.

jdal.trace.method — A flag indicating whether the JDAL driver performs method tracing when JDAL tracing is enabled. When this property is set to true, trace method signatures through the JSF JDAL driver are included in the trace output for problem resolution. Valid values are as follows:

true = Yes, JDAL performs method tracing.

false = No, JDAL does not perform method tracing. Default.

Note: The Trace facility was designed as a diagnostic tool for development and test environments and not for general use in a production environment. ACI strongly recommends that it not be used in a production environment due to the overhead and impacts on performance. Therefore, you should only enter this property in a -D JVM runtime argument when instructed by ACI for problem resolution.

Configuration Properties

Properties defined in the config.properties file are read by a Java server process when it initializes. A minimum of four configuration properties are required to bootstrap a Java server process built on the JSF—`db.user.id`, `db.user.password`, `event.log4j.config.url`, and `trace.log4j.config.url`. Several database properties (i.e., properties that start with “db.”) used to access the application database must also be defined in the config.properties file. These configuration properties, along with any other properties required to initialize an application process, must be defined in the config.properties file specified at runtime in the config.filename property for a Java server process. You can view the properties configured in the config.properties file, along with the properties configured in the Application Configuration data source, on the Configuration Properties window. You must edit the config.properties file to change the values of configuration properties in this file. Modified property values in the config.properties file do not take effect until after you stop and restart the Java server process.

Password Encryption Utility

The ConfigPropertyBuilder.class command window utility enables you to encrypt the password value for the db.user.password property. Note that the database password must be encrypted before the ACI application can access the database. After you have entered the db.user.id and db.user.password properties in the config.properties file, enter the following command at the Unix or DOS prompt:

Syntax

```
java -cp CLASSPATH ConfigPropertyBuilder directory/config.properties
```

CLASSPATH is a classpath that includes the framework-tools.jar, framework-core.jar, and bouncycastle-1.20.jar. The ConfigPropertyBuilder.class is distributed in the framework-tools.jar.

directory is the qualified name of the directory in which the config.properties file is located.

When you run the utility, it prompts you for both the db.user.id and db.user.password property values as shown below. You must enter both values correctly for the correct value to be generated.

Enter db.userid:
db-user-value

Enter db.password:
db-password-value

The utility creates a new config.properties file with an encrypted value for the db.user.password property. It also renames the original config.properties file to config.properties.xx where xx starts at 01 and is increased by one with each execution of the utility.

The utility also displays the encrypted and decrypted values of the database password as shown below.

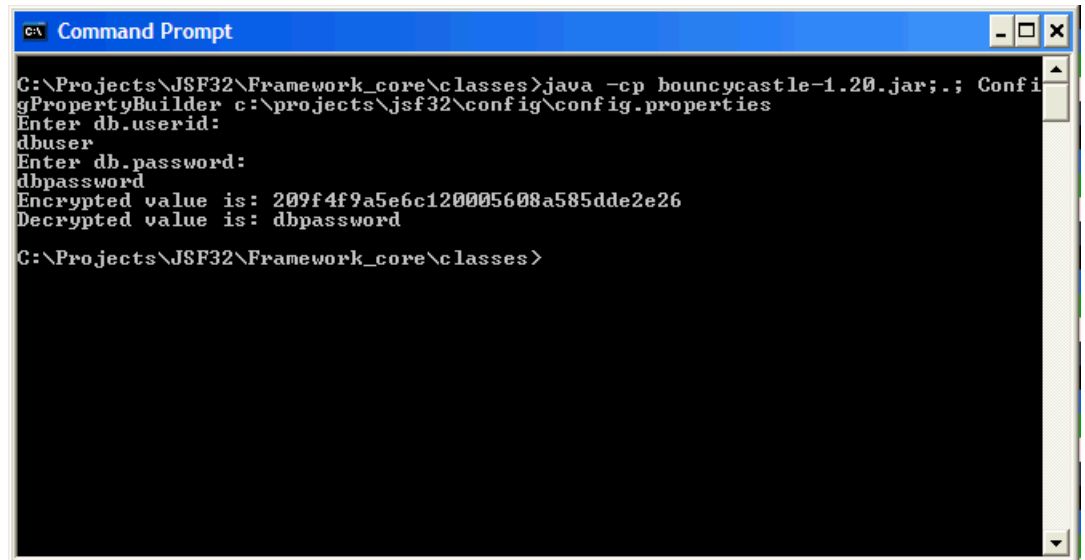
Encrypted value is: *encrypted-value*
Decrypted value is: *unencrypted-value*

After the password value is encrypted, it will be much longer than the original value and will contain both alphabetic and numeric characters.

Example Command. The following is an example command for running the ConfigPropertyBuilder.class command window utility from a DOS prompt. The command was run from a specific directory to simplify the class path needed and the config.properties file location was fully-qualified.

Example Command Syntax

```
C:/Projects/JSF32/Framework_core/classes>java -cp bouncycastle-1.20.jar;. ConfigPropertyBuilder c:/projects/jsf32/config/config.properties
```



```
C:\> Command Prompt

C:\Projects\JSF32\Framework_core\classes>java -cp bouncycastle-1.20.jar;.; ConfigPropertyBuilder c:\projects\jsf32\config\config.properties
Enter db.userid:
dbuser
Enter db.password:
dbpassword
Encrypted value is: 209f4f9a5e6c120005608a585dde2e26
Decrypted value is: dbpassword

C:\Projects\JSF32\Framework_core\classes>
```

Application Configuration Properties

Most properties, other than those required to bootstrap a Java server process, are configured in the Application Configuration data source (APPLD).^{*} The advantage of placing these properties in the APPLD is that you can change property values in the APPLD without stopping and restarting the Java server process. Properties in the APPLD are maintained from the Application Configuration window. When you add, modify, or delete properties in the APPLD, the changes do not take effect until after you reload the APPLD into cache memory by selecting a process ID on the Configuration Properties window and click the Reload button. The Configuration Properties window displays all APPLD and config.properties file properties for a Java server process.

- ^{*} Some JSF-based applications, such as BASE24-es ATM Device Handler processes, use additional properties files. Some of the properties used by BASE24-es ATM Device Handler processes are defined in ATM Bundle and device property files.

JSF Properties

The following properties are generic to the JSF. Any application-specific properties are either described in an application-specific appendix to this user guide or in an application-specific manual.

baos.manager.init.count — The number of `ByteArrayOutputStream` objects initially created by the Byte Array Output Stream Manager. Byte array output streams are pooled to avoid the cost of initializing the internal `byte[]` each time one is created. If this property is not configured, it defaults to 4.

Required: No
Example Entry: `baos.manager.init.count=2`
Usage: Byte Array Output Stream Manager

baos.manager.init.stream.size — The initial size of the `ByteArrayOutputStream` objects to create. If this property is not configured, it defaults to approximately 4K.

Required: No
Example Entry: `baos.manager.init.stream.size=2024`
Usage: Byte Array Output Stream Manager

baos.manager.max.pool.size — The maximum number of `ByteArrayOutputStream` objects that can be pooled. If this property is not configured, it defaults to 10.

Required: No
Example Entry: `baos.manager.max.pool.size=12`
Usage: Byte Array Output Stream Manager

cleanup.userauditlog.retention.days — The number of days to leave audit rows in the User Audit Log data source (UALOGD) before they are removed. The default value is 30 days.

The Payment Card Industry (PCI) Data Security Standard, which is used as the framework for Visa's Cardholder Information Security Program (CISP), states that you should retain audit trail history "for a period that is consistent with its effective use, as well as legal regulations." An audit history usually covers a period of at

least one year, with a minimum of three months available online. The PCI Data Security Standard also states that you should “promptly back-up audit trail files to a centralized log server or media that is difficult to alter.”

Required: No
Example Entry: `cleanup.userauditlog.retention.days=15`
Usage: User Audit Log Cleanup component

cleanup.userauditlog.cleanup.starttime — The time of day, in hh:mm format, the User Audit Log Cleanup component starts removing old rows from the User Audit Log data source (UALOGD). The default value is 23:59 (midnight).

Required: No
Example Entry: `cleanup.userauditlog.cleanup.starttime=01:00`
Usage: User Audit Log Cleanup component

cleanup.userauditlog.timeperiod.hours — The number of hours between runs if the User Audit Log Cleanup component is to remove old audit rows from the User Audit Log data source (UALOGD) more frequently than every twenty four hours. The default value is 24 hours.

Required: No
Example Entry: `cleanup.userauditlog.timeperiod.hours=12`
Usage: User Audit Log Cleanup component

cleanup.database.cache.interval.hours — The number of hours to leave data rows in cache memory before they are removed. The default value is 24 hours.

Required: No
Example Entry: `cleanup.database.cache.interval.hours=48`
Usage: Database Cache Cleanup component

cleanup.dbcache.cleanup.starttime — The time of day, in hh:mm format, the Database Cache Cleanup component starts removing old data rows from cache memory. The default value is 23:59 (midnight).

Required: No
Example Entry: `cleanup.dbcache.cleanup.starttime=23:15`
Usage: Database Cache Cleanup component

cleanup.dbcache.timeperiod.hours — The number of hours between runs if the Database Cache Cleanup component is to remove old data rows from cache memory more frequently than every twenty four hours. The default value is 24 hours.

Required: No
Example Entry: cleanup.dbcache.timeperiod.hours=12
Usage: Database Cache Cleanup component

context.manager.connection.id — When the context.type property is set to MEM-OUT-OF-PROCESS, this property specifies the external connection ID of an entry in the External Connection data source (EXTCND) that is used to send the context message to the Context Manager process. The value of this property must match a value configured in the EXTCND.

Required: Yes, if the context.type property is set to MEM-OUT-OF-PROCESS
Example Entry: context.manager.connection.id=JSRCTXMGR
Usage: ContextManager

context.retention.period — The number of days, from the start of the current day, message context is retained in the Message Context data source (MSGCNTX) before it is deleted by the Context Manager process. This property is used only if the context.type property is set to a value of DISK.

Required: Yes, if the context.type property is set to DISK
Example Entry: context.retention.period=3
Usage: Message Context Component

context.type — Indicates the type of context management the Context Manager performs. Valid values are as follows:

DISK	= The transaction context is maintained in the Message Context data source (MSGCNTX) on disk. This allows for response messages to be returned to any process that is configured to listen on the response queue.
------	---

- MEM-IN-PROCESS** = The transaction context is maintained in the memory of the process generating the context. This means that response messages must be returned to the process that originated the message.
- MEM-OUT-OF-PROCESS** = The transaction context is maintained in the memory of a separate process configured as a Context Manager. This allows for response messages to be returned to any process that is configured to listen on the response queue.

Note: For performance reasons, ACI recommends that this property be set to a value of MEM-IN-PROCESS for BASE24-es ATM Device Handler processes.

Required: Yes
 Example Entry: context.type=MEM-IN-PROCESS
 Usage: ContextManager

crypto.jce.prng.algname — The name of the SHA-1 Pseudo-Random Number Generation (SHA1PRNG) algorithm to use if the provider specified by the crypto.jce.prng.provider property uses a name other than the default value of SHA1PRNG.

Required: No
 Example Entry: crypto.jce.prng.algname=CRYPTOKI
 Usage: JCE Provider

crypto.jce.prng.provider — The name of the SHA-1 Pseudo-Random Number Generation (SHA1PRNG) provider.

Required: No
 Example Entry: crypto.jce.prng.provider=au.com.eracom.crypto.provider.ERACOMProvider
 Usage: JCE Provider

crypto.jce.aescipher.algname — The name of the Advanced Encryption Standard (AES) cipher algorithm to use if the provider specified by the crypto.jce.aescipher.provider property uses a name other than the default value of AES.

Required: No
Example Entry: `crypto.jce.aescipher.algname=Rijndael`
Usage: JCE Provider

crypto.jce.aescipher.provider — The name of the Advanced Encryption Standard (AES) cipher algorithm provider.

Required: No
Example Entry: `crypto.jce.aescipher.provider=sun.security.provider.Sun`
Usage: JCE Provider

crypto.jce.descipher.algname — The name of the Data Encryption Standard (DES) algorithm to use if the provider specified by the `crypto.jce.descipher.provider` property uses a name other than the default value of DES/CBC/PKCS5Padding.

Required: No
Example Entry: `crypto.jce.descipher.algname=DES`
Usage: JCE Provider

crypto.jce.descipher.provider — The name of the Data Encryption Standard (DES) algorithm provider.

Required: No
Example Entry: `crypto.jce.descipher.provider=com.sun.crypto.provider.SunJCE`
Usage: JCE Provider

crypto.jce.desedecipher.algname — The name of the Triple Data Encryption Standard (3DES) algorithm to use if the provider specified by the `crypto.jce.desedecipher.provider` property uses a name other than the default value of DESede.

Required: No
Example Entry: `crypto.jce.desedecipher.algname=DESede`
Usage: JCE Provider

crypto.jce.desedecipher.provider — The name of the Triple Data Encryption Standard (3DES) algorithm provider.

Required: No
Example Entry: `crypto.jce.desedecipher.provider=au.com.eracom.crypto.provider.ERACOMProvider`
Usage: JCE Provider

crypto.jce.cast5cipher.algname — The name of the block cipher developed by Carlisle Adams and Stafford Tavares (CAST5) to use if the provider specified by the `crypto.jce.cast5cipher.provider` property uses a name other than the default value of CAST5.

Required: No
Example Entry: `crypto.jce.cast5cipher.algname=DESede`
Usage: JCE Provider

crypto.jce.cast5cipher.provider — The name of the CAST5 block cipher algorithm provider.

Required: No
Example Entry: `crypto.jce.cast5cipher.provider=au.com.eracom.crypto.provider.ERACOMProvider`
Usage: JCE Provider

crypto.jce.sha1.algname — The name of the Secure Hash Algorithm (SHA-1) to use if the provider specified by the `crypto.jce.sha1.provider` property uses a name other than the default value of SHA.

Required: No
Example Entry: `crypto.jce.sha1.algname=SHA`
Usage: JCE Provider

crypto.jce.sha1.provider — The name of the SHA-1 hash algorithm provider.

Required: No
Example Entry: `crypto.jce.sha1.provider=au.com.eracom.crypto.provider.ERACOMProvider`
Usage: JCE Provider

crypto.jce.md5.algname — The name of the MD5 algorithm to use if the provider specified by the crypto.jce.md5.provider property uses a name other than the default value of MD5. The MD5 algorithm is used to create a message digest of input data and is used to verify data integrity.

Required: No
Example Entry: crypto.jce.md5.algname=MD5
Usage: JCE Provider

crypto.jce.md5.provider — The name of the MD5 algorithm provider.

Required: No
Example Entry: crypto.jce.md5.provider=au.com.eracom.crypto.provider.ERACOMProvider
Usage: JCE Provider

crypto.jce.hmacsha1.algname — The name of the Hash-based Message Authentication Code (HMAC) Secure Hash Algorithm (SHA-1) to use if the provider specified by the crypto.jce.hmacsha1.provider property uses a name other than the default value of HMACSHA1.

Required: No
Example Entry: crypto.jce.hmacsha1.algname=HMACSHA1
Usage: JCE Provider

crypto.jce.hmacsha1.provider — The name of the HMAC SHA-1 hash algorithm provider.

Required: No
Example Entry: crypto.jce.hmacsha1.provider=au.com.eracom.crypto.provider.ERACOMProvider
Usage: JCE Provider

crypto.jce.rsa.algname — The name of the Rivest, Shamir, and Adleman (RSA) cipher algorithm to use if the provider specified by the crypto.jce.rsa.provider property uses a name other than the default value of RSA.

Required: No
Example Entry: crypto.jce.rsa.algname=RSA
Usage: JCE Provider

crypto.jce.rsa.provider — The name of the RSA cipher algorithm provider.

Required: No
Example Entry: `crypto.jce.rsa.provider=au.com.eracom.crypto.provider.ERACOMProvider`
Usage: JCE Provider

crypto.jce.sha1rsasig.algname — The name of the SHA-1 with RSA digital signature algorithm to use if the provider specified by the `crypto.jce.sha1rsasig.provider` property uses a name other than the default value of `SHA1withRSA`.

Required: No
Example Entry: `crypto.jce.sha1rsasig.algname=SHA1withRSA`
Usage: JCE Provider

crypto.jce.sha1rsasig.provider — The name of the SHA-1 with RSA digital signature algorithm provider.

Required: No
Example Entry: `crypto.jce.sha1rsasig.provider=au.com.eracom.crypto.provider.ERACOMProvider`
Usage: JCE Provider

crypto.jce.dbcipher.algname — The name of the Triple Data Encryption Standard (3DES)/Electronic Code Book (ECB)/Public Key Cryptographic Standards (PKCS) 5 Padding algorithm to use if the provider specified by the `crypto.jce.dbcipher.provider` property uses a name other than the default value of `DESede/ECB/PKCS5Padding`.

Required: No
Example Entry: `crypto.jce.dbcipher.algname=DESede/ECB/PKCS5Padding`
Usage: JCE Provider

crypto.jce.dbcipher.provider — The name of the `DESede/ECB/PKCS5Padding` algorithm provider.

Required: No
Example Entry: `crypto.jce.dbcipher.provider=sun.security.provider.Sun`
Usage: JCE Provider

crypto.pool.ceiling — The maximum number of objects that can be pooled for a particular combination of cryptographic object and key. This indicates the maximum number of concurrent accesses needed for a cryptographic object and ensures that no more than the specified number of keys per key type, algorithm, or alias are stored in the pool. The default value for this property is 5.

If you are using an ACI application that accesses a hardware security device from a vendor other than Eracom using the JCE, you should leave this property set to its default value or set it to a value less than the default.

If you are using an ACI application that accesses an Eracom hardware security device using the JCE, you can use the following calculation to determine the appropriate value for this property.

KeyCount	= (number of DES keys + number of 3DES keys + number of HMAC keys) × 2 (modes of operation (encrypt/decrypt))
NumConnections	= 65534 × ~90% = 59000 (Allow approximately 10% padding for overages)
crypto.pool.ceiling value	= NumConnections ÷ KeyCount (Round down for the final result)

In the above calculation, 65534 is the maximum number of *sessions* or connections available to an Eracom security device. When some ACI applications, such as the ACI Access Control Server (ACS), use a large number of keys in a concurrent manner, the ACS may use all of these connections. The ceiling specified by this property prevents this from occurring.

Required:	No
Example Entry:	crypto.pool.ceiling=5
Usage:	JCECryptoPool

crypto.provider — The name of the class used to provide cryptographic services (i.e., key and certificate management).

Required:	Yes, if using the Certificate Manager or Cryptography class
Example Entry:	crypto.provider=com.aciworldwide.framework.crypto.JCEProvider
Usage:	Certificate Manager Cryptography

db.alt.datasource.name.db-service-name — The name of an alternate data source for the database service indicated by *db-service-name*. The value of this property must match a user-defined data source name configured in the Alternate Data Source data source (DATASRC). This property enables you to maintain specified data sources in a data format that is different from the main application database. For example, if your BASE24-es database is an SQL database, you could maintain specified data sources in flat JDAL files by configuring those alternate JDAL data sources in the DATASRC and specifying them in this property. When the Database Service attempts to access a data source, it first looks to see whether a `db.alt.datasource.name.db-service-name` property is configured for the database service that accesses that data source. If so, it accesses the alternate data source specified by the property. If not, it accesses the data source in the usual manner.

Required: No
Example Entry: `db.alt.datasource.name.inst=Institution`
Usage: DBService

db.audit.db-service-name — Indicates whether user auditing is performed for a particular database service, represented by *db-service-name*. The lowercase *db-service-name* should be set to an actual database service as specified in the service.gen file. There can be multiple occurrences of this property, with one for each database service requiring auditing. Valid values are as follows:

true = Yes, user auditing is performed for this database service.
false = No, user auditing is not performed for this database service.

Note: Depending on your release of the JSF, you can use either the `db.audit.db-service-name` property or the `db.audited.db-service name` property to define whether auditing is performed for a particular database service. With the `db.audit.db-service-name` property, full auditing is always performed when this property and the `db.audited` property are set to true. This property is used for auditing in release 3.0.2 of the JSF. With the `db.audited.db-service-name` property available starting in release 4.1 of the JSF, you can specify whether full or medium (header only) auditing is performed when the `db.audited` property is set to true.

Required: No
Example Entry: `db.audited.extrcnct=true`
Usage: Database Component

db.audited — A flag indicating whether any ACI desktop user interface activity is to be audited. If this property is set to a value of true, database actions (i.e., delete, insert, update, and view) are audited as defined for each database service in the `db.audited.db-service-name` or `db.audit.db-service-name` properties. Valid values are as follows:

true = Yes, user interface activities are audited.
false = No, user interface activities are not audited.

db.audited.db-service-name — Indicates the type of user auditing performed for a particular database service, represented by *db-service-name*. The lowercase *db-service-name* should be set to an actual database service as specified in the `service.gen` file. There can be multiple occurrences of this property, with one for each database service requiring auditing. Valid values are as follows:

none = No auditing is performed for this database service.
medium = Medium auditing is performed for this database service. For this value, only header information is logged in auditing records.
full = Full auditing is performed for this database service. For this value, header information as well as detail information about the before and after images of the affected record are logged.

If this property is not defined for a JSF database service, it defaults to a value of none unless BASE24-es is installed. When BASE24-es is installed, all of these properties default to a value of full.

db-service-name values generic to the JSF are listed in the following table.

Database Service Name	Data Source	ACI Desktop Window
applcnfg	Application Configuration data source (APPLD)	Application Configuration window
datasrc	Alternate Data Source data source (DATASRC)	Not Applicable
extrcnct	External Connections data source (EXTCND)	External Connection Configuration window
listener	Listener data source (LSTND)	Listener Configuration window

Database Service Name	Data Source	ACI Desktop Window
process	Process data source (PROCD)	Process Configuration window

Refer to the application-specific appendices of this user guide or an application-specific manual for a complete list of database services and the data sources accessed by an application.

Note: Depending on your release of the JSF, you can use either the `db.audit.db-service-name` property or the `db.audited.db-service name` property to define whether auditing is performed for a particular database service. With the `db.audit.db-service-name` property, full auditing is always performed when the `db.audit.db-service-name` property and the `db.audited` property are set to true. This property is used for auditing in release 3.0.2 of the JSF. With the `db.audited.db-service-name` property available starting in release 4.1 of the JSF, you can specify whether full or medium (header only) auditing is performed when the `db.audited` property is set to true.

Required: No
Example Entry: `db.audited.applcnfg=full`
Usage: Database Component

db.broadcast — A flag indicating whether to send broadcast messages to other JVMs when performing one of the following functions: clearing the cached database entries for all services, clearing the cached database entries for a specific service, or reloading the application configuration properties for the process into cache memory. Valid values are as follows:

true = Yes, send broadcast messages for certain database caching operations.
false = No, do not send broadcast messages for certain database caching operations.

Required: No
Example Entry: `db.broadcast=true`
Usage: Framework Administration

db.cached.db-service-name — Indicates whether the data source indicated by the database service name, represented by *db-service-name*, is accessed from cache memory or from disk. Valid values are as follows:

true = Yes, perform caching for this database service.

false = No, do not perform caching for this database service.

This property indicates whether the database access performed on the specified service uses the database service caching mechanism provided by the CacheManager. For all SQL queries, except the `fetchbyDataObject` method, the database services cache is searched first for the data. If the data is not found, the cache is updated with the data retrieved from the data source. Thereafter, it is retrieved from cache until the timer for the `cleanup.database.cache.interval.hours` property expires or the database service adds, updates or deletes a row of the data source or the Java server process is stopped and restarted, at which time, cache memory is flushed. This functionality should be used for tables that are updated infrequently, or read frequently and contain a small number of rows. The lowercase *db-service-name* is set to an actual database service as specified in the `service.gen` file. You can configure multiple occurrences of this property, with one for each database service requiring caching support.

db-service-name values generic to the JSF are listed in the following table.

Database Service Name	Data Source	ACI Desktop Window
applcnfg	Application Configuration data source (APPLD)	Application Configuration window
datasrc	Alternate Data Source data source (DATASRC)	Not Applicable
extrcnct	External Connections data source (EXTCND)	External Connection Configuration window
listener	Listener data source (LSTND)	Listener Configuration window
process	Process data source (PROCD)	Process Configuration window

Refer to the application-specific appendices of this user guide or an application-specific manual for a complete list of database services and the data sources accessed by an application.

Required: Yes
 Example Entry: db.cached.applcnfg=true
 Usage: Cache Manager

db.connection.wait.time — The number of milliseconds the Database Connection Manager waits for another thread to free a database connection. This property value is used when the maximum number of connections has been reached as specified in the db.max.connections property.

Required: Yes
 Example Entry: db.connection.wait.time=10
 Usage: DBConnectionManager

db.datasource — The data source URI specification that allows the JDBC driver to access the correct database. You can typically find the configuration requirements for this property in the vendor documentation for the JDBC driver. JDBC driver requirements may include information such as the host or IP address where database resides, the port to connect to for the database, the name of database on the host, and other JDBC driver settings.

Required: Yes
 Example Entries: BASE24-es ATM Device Handler:
 db.datasource=jdbc:JTurbo://172.16.1.53:1433/
 DeviceHandlerv32/sp=false

 BASE24-es Java User Interface Servlets:
 db.datasource=jdbc:JDAL:-MDBV=CSV:-AI=ES:-STRT=/G/
 moat/ui6sdata

 ACI Commerce Gateway:
 db.datasource=jdbc:JTurbo://127.0.0.1:1433/ACIDB/sp=false
 (MS SQL Server):
 db.datasource=jdbc:JTurbo://127.0.0.1:1433/ACIDB/sp=false
 (Oracle 8i):
 db.datasource=jdbc:oracle:thin:@127.0.0.1:1521:ACIDB
 (IBM DB2-UDB):
 db.datasource=jdbc:db2:ACIDB

ACI Access Control Server:

db.datasource=jdbc:JTurbo://127.0.0.1:1433/ACISC/sp=false

(MS SQL Server):

db.datasource=jdbc:JTurbo://127.0.0.1:1433/ACISC/sp=false

(Oracle 8i):

db.datasource=jdbc:oracle:thin:@127.0.0.1:1521:ACISC

(IBM DB2-UDB):

db.datasource=jdbc:db2:ACISC

Usage: DBConnectionManager

db.driver — The class name of the JDBC driver to use for all database connections. Enter the entire package name as it is found in the JAR containing the driver. Refer to the vendor documentation for the JDBC driver for supporting information.

Required: Yes

Example Entries: MS SQL Server (JTurbo):
db.driver=com.ashna.jturbo.driver.Driver

Oracle 8i:

db.driver=oracle.jdbc.driver.OracleDriver

IBM DB2-UDB:

db.driver=COM.ibm.db2.jdbc.app.DB2Driver

JDAL:

db.driver=com.aciworldwide.jdbcdriver.jdal.JDALDriver

Usage: DBConnectionManager

db.max.connections — The maximum number of concurrent database connections that the Database Connection Manager (DBConnectionManager) can have open at one time.

Required: Yes

Example Entry: db.max.connections=50

Usage: DBConnectionManager

db.static.connections — The number of connections the Database Connection Manager acquires when the Java server process is started. These connections remain active throughout the life of the process.

Required: Yes
Example Entry: db.static.connections=2
Usage: DBConnectionManager

db.table.prefix — The prefix, if one exists, to prepend to all data source table names in the application database. This allows you to use multiple databases with the same underlying data source names prepended with a unique identifier for each database, thereby making the full data source names in each database unique across all of the application's databases.

Required: No
Example Entry: db.table.prefix=Prod_
Usage: Database

db.type — The type of database being used in the current JVM executing environment. Valid values are as follows:

DB2 = IBM relational database
JDAL = ACI's Java Data Access Layer database
JDAL2 = ACI's Java Data Access Layer 2 database
MSSQL = Microsoft SQL Server database
ORACLE = Oracle database
SQLMX = HP NonStop SQL/MX database

Required: Yes
Example Entry: db.type=MSSQL
Usage: Database Service

db.useisolationlevel — A flag indicating whether the database services automatically manipulate connection transaction isolation levels. This property is set to true for Microsoft SQL Server and IBM DB2 databases. This property is set to false for Oracle and HP NonStop SML/MP databases. When this property is set to true, database access is automatically set to reduce database contention on queries where reading through locks is valid. Valid values are as follows:

true = Yes, automatically manipulate connection transaction isolation levels.
false = No, do not automatically manipulate connection transaction isolation levels.

Required: No
Example Entry: db.useisolationlevel=true
Usage: Database Services

db.user.id — The JDBC user ID needed to authenticate a connection to the database. This user ID is used to securely access the underlying database and is typically established by a database or system administrator. ACI applications use this user ID to access the database.

Required: Yes
Example Entry: db.user.id=myuser
Usage: DBConnectionManager

db.user.password — The JDBC password needed to authenticate a connection to the database. This user ID is used to securely access the underlying database and is typically established by a database or system administrator. For security purposes, you can encrypt the password stored in the config.properties file using the [password encryption utility](#) provided by ACI.

Required: Yes
Example Entry: db.user.password=mypswd
Usage: DBConnectionManager

event.log4j.config.url — The fully-qualified URI location of the log4j configuration file (EventConfig.xml) for logging event messages.

Required: Yes
Example Entries:

event.log4j.config.url=file:///C:/src/jdh/config/BASE24-Common/
EventConfig.xml

event.log4j.config.url=file:///C:/SecureCommerce/CommerceGateway/
Config/EventConfig.xml

event.log4j.config.url=http://localhost:8080/config/EventConfig.xml

Usage: EventManager

event.sockets.encoding — The character encoding set (e.g., ISO8859-1) used for event logging with the ACI Transmission Control Protocol (TCP) Appender. If this property is not set, the default system encoding is assumed.

Required: No
 Example Entry: event.sockets.encoding=ISO8859-1
 Usage: ACI TCP Appender

event.sockets.host — The name of the host to send event messages to when using the ACI TCP Appender to log event messages. If this property is not configured, it defaults to localhost.

Required: No
 Example Entry: event.sockets.host=remotehost
 Usage: ACI TCP Appender

event.sockets.port — The TCP port number to connect to, acting as a client, when using the ACI TCP Appender to log event messages.

Required: Yes, if using the ACI TCP Appender
 Example Entry: event.sockets.port=8004
 Usage: ACI TCP Appender

initialization.function.*n*.classname — Specifies the fully-qualified Java package and class name to be started at initialization by the Initialization Manager, where *n* is a sequential number starting at 0 and increased by 1 for each class to be initialized.

Note: If initialization functions are not sequentially numbered starting at 0 and increased by 1 for each function, the Java server process stops initializing with the last function correctly numbered. For example, if functions are numbered 0, 1, 3, etc. instead of 0, 1, 2, etc., the Java server process stops initializing with function 1 because the next function expected is function 2.

Required: No
 Example Entries:

initialization.function.0.classname=es.server.framework.
 TransactionMapManager

initialization.function.1.classname=es.server.common.
 DataValidationManager

initialization.function.2.classname=es.server.formatter.
 MessageSchemaManager

Usage: InitializationManager

initialization.function.*n*.methodname — Specifies the fully-qualified initialization method to be called at initialization by the Initialization Manager, where *n* is a sequential number starting at 0 and increased by 1 for each initialization method to be called. This property is used in conjunction with the `initialization.function.n.classname` property to invoke a specific method for the associated class at initialization. For example, the `initialization.function.1.methodname` property would invoke the specified method for the class identified by the `initialization.function.1.classname` property.

Note: If initialization functions are not sequentially numbered starting at 0 and increased by 1 for each function, the Java server process stops calling the last method correctly numbered. For example, if functions are numbered 0, 1, 3, etc. instead of 0, 1, 2, etc., the Java server process stops initializing with function 1 because the next function expected is function 2.

Required: No

Example Entries:

`initialization.function.0.methodname=init`

`initialization.function.1.methodname=init`

`initialization.function.2.methodname=init`

Usage: InitializationManager

keymanager.provider — The pathname of the provider for key management services.

Required: Yes

Example Entry: `keymanager.provider = com.aciworldwide.framework.crypto.
KeyStoreService`

Usage: KeyManager

keystore.filename — The path name of the data source containing the Java keystore. Public and private keys, as well as certificates are stored in the Java keystore. For keystores maintained in a hardware security device, this property must be set to null. See the [keystore.type](#) property for more details.

Required: Yes

Example Entry: `keystore.filename=C://SecureCommerce//ACS//Config//
Keystore//keystore.bin`

Usage: Key Store Service

keystore.hardware — A flag indicating whether the keystore is maintained within a hardware security device or in software. Valid values are as follows:

true = Yes, the keystore is maintained within a hardware security device.
false = No, the keystore is not maintained within a hardware security device. It is maintained in software.

The value of this property must be set in accordance with the value of the [keystore.type](#) property.

Required: Yes
Example Entry: keystore.filename=false
Usage: Key Store Service

keystore.password — The password used to access the Java keystore. Public and private keys, as well as certificates are stored in the Java keystore. Therefore, the value of this property should be treated with the upmost confidentiality. This property is currently used only for Eracom JCE functionality.

Required: Yes, for Eracom JCE functionality.
Example Entry: keystore.password=1234
Usage: Eracom JCE Provider
Key Store Service

keystore.provider — The name of the Java Keystore provider used by the Key Store Service. The JSF allows you to use any hardware or software Java keystore provider. The value of this property must be set in accordance with the value of the [keystore.type](#) property. The following are currently supported values for the keystore.provider property:

SUNJCE = Software-based Java Cryptography Extension (JCE) keystore provided by Sun.
IBMJCE = Software-based JCE keystore provided by IBM.
nCipher.KM = nCipher keystore.
ERACOM = Hardware keystore provided by Eracom.

Required: Yes
Example Entry: keystore.provider=SunJCE
Usage: Key Store Service

keystore.type — Indicates the type of keystore provided. The value depends on the keystore provider and the configuration used. Valid values are as follows:

CRYPTOKI = Keystore provided by nCipherKM.
 JCEKS = Software-based Java Cryptography Extension (JCE) keystore provided by Sun or IBM.
 nCipher.sworld = Hardware keystore provided by Eracom.

The keystore.provider, keystore.hardware, and keystore.filename properties must be set in accordance with the value in the keystore.type property as shown in the following table.

keystore.type	keystore.provider	keystore.hardware	keystore.filename
CRYPTOKI	nCipherKM	false	<i>file-pathname</i>
JCEKS	SUNJCE, IBMJCE	false	<i>file-pathname</i>
nCipher.sworld	ERACOM	true	null

Required: Yes
 Example Entry: keystore.type = JCEKS
 Usage: Key Store Service

mail.smtp.host — The name of the Simple Mail Transfer Protocol (SMTP) mail server. This property, along with all the other mail.smtp properties, must be configured in the config.properties file to support the sending of SMTP e-mail messages to an SMTP host system.

Required: Yes, if using SMTP
 Example Entry: mail.smtp.host=lng001.tsa.com
 Usage: SMTP Process

mail.smtp.from.addr — The e-mail address to be used as the from address in e-mails sent to an SMTP host system. This property, along with all the other mail.smtp properties, must be configured in the config.properties file to support the sending of SMTP e-mail messages to an SMTP host system.

Required: Yes, if using SMTP
 Example Entry: mail.smtp.from.addr=SMTPTester@aciworldwide.com
 Usage: SMTP Process

mail.smtp.auth.required — A flag indicating whether the receiving SMTP mail server requires the user specified in the mail.smtp.auth.user property to be authenticated before accepting messages from the user. Valid values are as follows:

true = Yes, authenticate the user.
false = No, do not authenticate the user.

This property, along with all the other mail.smtp properties, must be configured in the config.properties file to support the sending of SMTP e-mail messages to an SMTP host system.

Required: Yes, if using SMTP
Example Entry: mail.smtp.auth.required=false
Usage: SMTP Process

mail.smtp.auth.user — The user name to use for authentication if the SMTP mail server requires authentication and the mail.smtp.auth.required property is set to true. This property, along with all the other mail.smtp properties, must be configured in the config.properties file to support the sending of SMTP e-mail messages to an SMTP host system.

Required: Yes, if using SMTP
Example Entry: mail.smtp.auth.user=SMTPTester
Usage: SMTP Process

mail.smtp.auth.password — The password to use for authentication of the user specified in the mail.smtp.auth.user property if the SMTP mail server requires authentication and the mail.smtp.auth.required property is set to true. This property, along with all the other mail.smtp properties, must be configured in the config.properties file to support the sending of SMTP e-mail messages to an SMTP host system.

Required: Yes, if using SMTP
Example Entry: mail.smtp.auth.password=Mypassword1
Usage: SMTP Process

mail.smtp.max.retries — The maximum number of times to retry establishing a connection to the SMTP host when sending a message. This property, along with all the other mail.smtp properties, must be configured in the config.properties file to support the sending of SMTP e-mail messages to an SMTP host system.

Required: Yes, if using SMTP
Example Entry: mail.smtp.max.retries=3
Usage: SMTP Process

message.environment — Indicates the type of messaging environment used for the JSF-based application. Valid values are as follows:

DIRECT = Direct messaging between processes.
JMS = Messaging through the Java Message Service (JMS)
EJB = Messaging through Enterprise JavaBeans (EJB). Reserved for future use.

Required: Yes
Example Entry: message.environment=JMS
Usage: CommonHTTPServlet
CommonXMLServlet
CommonJSPPage

message.jms.queue.externalconnectionid — The external connection ID configured in the External Connection data source (EXTCND) for JMS. This property is required if the message.environment property is set to JMS. The value of this property must match an external connection ID value in the EXTCND for the JMS service.

Required: Yes, if the message.environment property is set to JMS
Example Entries: message.jms.queue.externalconnectionid=NCR-JMS
message.jms.queue.externalconnectionid=DIEBOLD-JMS
message.jms.queue.externalconnectionid=COMMERCEGATEWAY
Usage: CommonHTTPServlet
CommonXMLServlet
CommonJSPPage

message.jms.Connection.DisableOnStartup — A flag indicating whether the JMS Message Router Service's capability to start JMS connections at startup is disabled. Disabling JMS connections at startup enables the application to determine when to start accepting messages from the middleware component they connect to. Valid values are as follows:

true = Yes, disable the capability to start JMS connections at startup.
false = No, do not disable the capability to start JMS connections at startup.

Required: No
 Example Entry: message.jms.Connection.DisableOnStartup=true
 Usage: JMS Message Router Service

message.jms.queuesFromJNDI — A flag indicating whether the ACI application defines the queue name or the queue name is retrieved from a Java Naming and Directory Interface (JNDI) server. If this property is set to true, it allows the JNDI server to manage the queue to queue name mapping. If this property is set to false, the ACI application defines the actual queue name. This property is only used when using JMS for messaging. A setting of true is preferred. Valid values are as follows:

true = Yes, the queue name is retrieved from a JNDI server.
 false = No, the queue name is not retrieved from a JNDI server. The application must define the queue name.

Required: No
 Example Entry: message.jms.queuesFromJNDI=true
 Usage: JMS Listener Manager
 JMS Connection Manager
 JMS Session Manager

message.jms.QueueConnectionFactory — The name of the queue connection factory used to automatically create queues for point-to-point messaging.

Required: Yes, if using a JMS messaging provider such as WebSphere MQ or NET24-XPNET.
 Example Entry: message.jms.QueueConnectionFactory=JmsQueueConnectionFactory
 Usage: JMS Connection Manager

message.jms.jndi.initialcontextfactory — The name of the initial context factory for the JNDI service provider specified by the message.jms.jndi.providerurl property. The context factory creates an initial context instance for reaching all other parts of the namespace of the underlying naming/directory service.

Required: Yes, if using a JMS messaging provider such as WebSphere MQ or NET24-XPNET.

Example Entries: `message.jms.jndi.initialcontextfactory=com.sun.enterprise.naming.SerialInitContextFactory`
`message.jms.jndi.initialcontextfactory=com.sun.jndi.ldap.LdapCtxFactory`
(WebSphere MQ):
`message.jms.jndi.initialcontextfactory=com.sun.jndi.fscontext.RefFSContextFactory`
(Open JMS):
`message.jms.jndi.initialcontextfactory=org.exolab.jms.jndi.rmi.RmiJndiInitialContextFactory`
`message.jms.jndi.initialcontextfactory=progress.message.jclient.QueueConnectionFactory`

Usage: JMS Listener Manager
JMS Connection Manager
JMS Queue Manager

message.jms.jndi.providerurl — The URL location of the Java Naming and Directory Interface (JNDI) processing provider (e.g., LDAP, DNS, CORBA, etc.).

Required: Yes, if using a JMS messaging provider such as WebSphere MQ or NET24-XPNET.

Example Entries: `message.jms.jndi.providerurl=t3://localhost:7001`
(Open JMS):
`message.jms.jndi.providerurl=rmi://localhost:1099/JndiServer`
(WebSphere MQ):
`message.jms.jndi.providerurl=file:C:/Program files/ibm/MQSeries/Java/JNDI-Directory`

Usage: JMS Listener Manager
JMS Connection Manager
JMS Queue Manager

message.jms.use.nsk.threads — A flag indicating whether to use HP NonStop message threading. This property must be set to a value of true if the JSF application is installed on an HP NonStop platform. This property is not used when the JSF is installed on an SMP platform. Valid values are as follows:

true = Yes, use HP NonStop message threading.
false = No, do not use HP NonStop message threading.

Required: Yes, if the JSF is installed on an HP NonStop platform
 Example Entry: message.jsm.use.nsk.threads=true
 Usage: Message Listener Implementation

message.property.handling — A code indicating how the application handles messages to and from other applications. Valid values are as follows:

JMS = The application communicates through Java Message Service (JMS) to other applications using JMS. JMS send/receive properties are used.
 MDS = The application communicates through JMS to a BASE24-es application that requires the BASE24-es proprietary message header. Message Delivery Subsystem (MDS) header properties are used.
 Default.

Required: Yes
 Example Entry: message.property.handling=JMS
 Usage: Message Properties

sax2.parser.init.count — The number of Simple API for XML (SAX2)-compliant XML parsers to be initially created by the InitializationManager at process startup. It is more efficient to have a pool of parser instances available for clients rather than having a client create a new one as needed. If this property is not configured, it defaults to 4.

Required: No
 Example Entry: sax2.parser.init.count=5
 Usage: SAX2 Parser Manager

SSL.client.keystore — The pathname of the data source containing the keystore for Secure Sockets Layer (SSL) client security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.keyStore system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
 Example Entry: SSL.client.keystore = SSLKeyStore
 Usage: HTTPSConnectionFactorySunJSSE

SSL.client.keystore.password — The password required to access the SSL keystore for client security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.keyStorePassword system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
Example Entry: SSL.server.keystore.password = password
Usage: HTTPSConnectionFactorySunJSSE

SSL.client.truststore — The pathname of the data source containing the truststore for Secure Sockets Layer (SSL) client security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.trustStore system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

A *truststore* is a keystore that is used when making decisions about who to trust. If you receive some data from an entity that you already trust, and if you can verify that the entity is the one it claims to be, then you can assume that the data really came from that entity.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
Example Entry: SSL.server.truststore = SSLTrustStore
Usage: HTTPSConnectionFactorySunJSSE

SSL.client.truststore.password — The password required to access the SSL truststore for client security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.trustStorePassword system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
Example Entry: SSL.server.truststore.password = password
Usage: JSSEManager

SSL.debug — A flag indicating whether to log trace messages for Secure Sockets Layer (SSL) processing to the standard out location (STDOUT). Valid values are as follows:

all = Yes, log trace messages for Sun SSL processing.
 true = Yes, log trace messages for IBM SSL processing.
 false = No, do not log trace messages for SSL processing. Default

Note: The Trace facility was designed as a diagnostic tool for development and test environments and not for general use in a production environment. ACI strongly recommends that it not be used in a production environment due to the overhead and impacts on performance.

Required: No
 Example Entry: SSL.debug= false
 Usage: JSSEManager
 HTTPSConnectionFactorySunJSSE

SSL.protocol.handler.pkgs — The pathname of the handler's implementation package name to be added to the list of packages searched by the Java URL class. This is a system property.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
 Example Entry: java.protocol.handler.pkgs=com.sun.net.ssl.internal.www.protocol myApp
 Usage: HTTPSConnectionFactorySunJSSE

SSL.server.keystore — The pathname of the data source containing the keystore for Secure Sockets Layer (SSL) server security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.keyStore system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
 Example Entry: SSL.server.keystore = SSLKeyStore
 Usage: JSSEManager

SSL.server.keystore.password — The password required to access the SSL keystore for server security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.keyStorePassword system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
Example Entry: SSL.server.keystore.password = password
Usage: JSSEManager

SSL.server.truststore — The pathname of the data source containing the truststore for Secure Sockets Layer (SSL) server security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.trustStore system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

A *truststore* is a keystore that is used when making decisions about who to trust. If you receive some data from an entity that you already trust, and if you can verify that the entity is the one it claims to be, then you can assume that the data really came from that entity.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
Example Entry: SSL.server.truststore = SSLTrustStore
Usage: JSSEManager

SSL.server.truststore.password — The password required to access the SSL truststore for server security. The value in this property is used to initialize the JSSE environment by loading the value of the property in the config.properties file to the javax.net.ssl.trustStorePassword system property for the JVM. Refer to the *Java Secure Socket Extension (JSSE) 1.0.2 API User's Guide* for more information.

Required: Yes, if an external connection configured in the External Connections data source (EXTCND) requires HTTP/S.
Example Entry: SSL.server.truststore.password = password
Usage: JSSEManager

timertask.manager.processid — The process ID of the Timer process that is to run all cleanup tasks for the executing JVM. The functions to be executed are specified in `timertask.function.n.classname` properties.

Required: Yes, if cleanup tasks are required.
Example Entries: `timertask.manager.processid=P6S^ESWEB01`
`timertask.manager.processid=TMRTASKS`
Usage: Timer Task Manager

timertask.function.n.classname — The name of a class to be executed by the Timer process, where *n* is a number starting at 0 and increased by 1 for each class to be executed. Each class designates cleanup tasks to be performed for the JSF application.

Required: Yes, if cleanup tasks are required.
Example Entries: `timertask.function.0.classname=com.aciworldwide.common.timertask.UserAuditLogCleanup`
`timertask.function.1.classname=com.aciworldwide.common.timertask.DatabaseCacheCleanup`
(BASE24-es ATM Device Handler process):
`timertask.function.0.classname=com.aciworldwide.base24.es.dh.common.timer.timertask.ReversalSAFTimerTask`
`timertask.function.1.classname=com.aciworldwide.base24.es.dh.common.timer.timertask.StatementDataCleanupTimerTask`
Usage: Timer Task Manager

trace.enabled — A flag indicating whether trace messages are enabled for components or services executing in the Java server process. Message tracing is configured in the configuration file specified by the `trace.log4j.config.url` property. Valid values are as follows:

true = Yes, message tracing is enabled.
false = No, message tracing is not enabled.

Note: The Trace facility was designed as a diagnostic tool for development and test environments and not for general use in a production environment. ACI strongly recommends that it not be used in a production environment due to the overhead and impacts on performance.

Required: Yes
Example Entry: `trace.enabled=false`
Usage: TraceManager

trace.log4j.config.url — The fully-qualified URI location of the log4j configuration file for logging trace messages.

Required: Yes
Example Entries: `trace.log4j.config.url=file:///C:/src/jdh/config/BASE24-Common/TraceConfig.xml`
`trace.log4j.config.url=http://localhost:8080/config/TraceConfig.xml`
Usage: TraceManager

website.accountformat — The format of account numbers entered at the web site. All characters, other than the number sign (#) character, are for formatting the entered account number. The # character represents a character of the entered account number. For example, if the characters of the entered account number are 1234567890123456 and this property is set to a value of #####-#####-#####, the account number would be formatted as 1234-5678-9012-3456. If this property is not configured, it defaults to a value of #####-#####-#####.

Required: No
Example Entry: `website.accountformat=#####-#####-#####`
Usage: WebsiteConfig

website.phoneformat — The format of phone numbers entered at the web site. All characters, other than the number sign (#) character, are for formatting the entered phone number. The # character represents a character of the entered phone number. For example, if the characters of the entered phone number are 4023907600 and this property is set to a value of (###) ###-#####, the phone number would be formatted as (402) 390-7600. If this property is not configured, it defaults to a value of (###) ###-#####.

Required: No
Example Entry: `website.phoneformat=(###) ###-#####`
Usage: WebsiteConfig

website.character.encoding — The internal character encoding set (e.g., ISO8859-1) used for all data entered from JSP screens on the website. The default value is ISO8859-1. The value of this property should match the value in the i18n.jsf file located in the website's web-inf directory.

Required: No
Example Entry: `website.character.encoding=ISO8859-1`
Usage: `CommonJSPPage`

website.datestyle — A code indicating the format in which dates are displayed on a browser user interface pages. The value of this property is used by the Java `DateFormat` class and is set for the process so all browser pages display dates in the same way. The `DateFormat` class provides many class methods for obtaining default date/time formatters based on the default or a given locale and a number of formatting styles. The formatting styles for the class include `FULL`, `LONG`, `MEDIUM`, and `SHORT`. Valid values of this property are as follows:

- 0 = `FULL` displays the full date, including the day of the week, month, day, year, and calendar as in Wednesday, April 14, 2004 AD
- 1 = `LONG` displays the month, day, and year as in April 14, 2004
- 2 = `MEDIUM` displays the month (abbreviated), day, and year as in Apr 14, 2004
- 3 = `SHORT` displays the month, day, and year numerically as in 04.14.04

Required: No
Example Entry: `website.datestyle=1`
Usage: `AdminPage`
`AppPage`
`WebsiteConfig`

website.defaultlanguage — A user-defined language code used to access a row from the Language data source (`LANGUAG`) to obtain a locale value (see the `website.defaultlocale` property description below). The locale value specifies the default language to be used for the JSF-based application. If this property is not configured, it defaults to a value of `USA`.

Required: No
Example Entry: `website.defaultlanguage=USA`
Usage: `WebsiteConfig`

website.defaultlocale — The default locale to use for localizing the JSF-based application. For the JSF, the locale is a string value comprised of the ISO language code and ISO country code, where the locale string value is formatted as follows:

ll_CC

where:

ll is the two-byte alpha language indicator from ISO-639 in lower-case and *CC* is the two-byte alpha country code from ISO-3166 in upper-case.

Note: The language indicator and country code are separated by an underbar (_).

The combination of language indicator and country code allows you to differentiate between different usages of a particular language in different countries. For example, a language code of `en_US` specifies the English language as used in the United States while a code of `en_GB` specifies the English language as used in Great Britain; a language code of `es_ES` specifies the Spanish language as used in Spain while a code of `es_MX` specifies the Spanish language as used in Mexico.

Required: Yes, if used with a Web browser-based user interface.

Example Entry: `website.defaultlocale=en_US`

Usage: AdminPage
AdminUser
CommonJSPPages
CommonXMLFormatter
HTTPServletBase
Language
WebsiteConfig
XMLServletBase

website.domain — The default domain name for a website application.

Required: No

Example Entry: `website.domain=aciworldwide.com`

Usage: WebsiteConfig

website.inactivetimeout — The number of seconds that can elapse without activity before a web connection is considered to have timed out. For example, this property could be used to identify ATMs that are not active because of communication problems. This property should be set to a value greater than the longest anticipated disconnect period (e.g., for a dial-up ATM). This property is only used with a Web browser-based user interface. If this property is not configured, it defaults to 20 seconds.

Required: No

Example Entry: `website.inactivetimeout=15`

Usage: WebsiteConfig

website.secureSSL — A flag indicating whether the application uses Secure Sockets Layer (SSL) security for web connections. Valid values are as follows:

true = Yes, SSL security is used.
false = No, SSL security is not used.

Required: No
Example Entry: website.secureSSL=false
Usage: WebsiteConfig

website.timestyle — A code indicating the format in which times are displayed on a browser user interface pages. The value of this property is used by the Java DateFormat class and is set for the process so all browser pages display times in the same way. The DateFormat class provides many class methods for obtaining default date/time formatters based on the default or a given locale and a number of formatting styles. The formatting styles for the class include FULL, LONG, MEDIUM, and SHORT. Valid values of this property are as follows:

0 = FULL displays the hour, minute, second, am/pm indicator, and time zone as in 3:30:42pm CST
1 = LONG displays the hour, minute, second, and am/pm indicator as in 3:30:32pm
2 = MEDIUM displays the hour, minute, and am/pm indicator as in 3:30pm
3 = SHORT displays just the hour and minute as in 3:30

Required: No
Example Entry: website.timestyle=1
Usage: AdminPage
AppPage
WebsiteConfig

website.xml.menu.map.filename.*n* — The name of a Transaction Menu Map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each filename specified. Transaction Menu Map XML files specify schemas used to manage browser user interface menu items. These schemas provide input for all possible menu items that can be presented on the browser user interface. Some menu items may not be displayed for a specific user due to the User Security (USEC) application permission definitions for the user. These schema files also allow you to customize the menu layout. For example, you can customize the schema to change the menu layout, such as the order in which menu items are presented, and to change the tasks associated with individual menu items. This property should be defined in the initialization section of the config.properties file

because the Menu Map Manager reads it at initialization to preload menu handling details into memory. The data is held in memory for as long as the application is running.

Note: The JSFMenuMap.xml file contains general information that is distributed with all JSF-based applications using a browser user interface.

Required: Yes, for Web browser-based applications.
Example Entries: website.xml.menu.map.filename.0=JSFMenuMap.xml
website.xml.menu.map.filename.1=ESMenuMap.xml
website.xml.menu.map.filename.1=PMMMenuMap.xml
Usage: MenuMapManager
WebsiteConfig

xml.builder.use.indent — A flag indicating whether the Builder class builds XML messages with indentation and new line characters. These formatted XML messages are easier to read and are useful in a testing or development environment. ACI recommends that you do not use indented XML messages in a production system. Valid values are as follows:

true = Yes, use indentation when building XML documents.
false = No, do not use indentation when building XML documents. Default.

Required: No
Example Entry: xml.builder.use.indent=false
Usage: Builder

xmltranmap.filename.*n* — The name of a Transaction Map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each filename specified. Transaction Map XML files map symbolic keys (such as a message ID) into formatter classes to parse the message and a command classes to process the transaction. This property should be defined in the initialization section of the config.properties file because the Transaction Map Manager reads it at initialization to preload message handling details into memory. The data is held in memory for as long as the application is running.

Required: Application-dependent
Example Entries: BASE24-es ATM Diebold 911/912 and NCR NDC+ Device Handler processes:
xmltranmap.filename.0=C:/ECommerce/Version041/config/
DHCommonTranMap.xml

xmltranmap.filename.1=C:/ECommerce/Version041/config/
DieboldDHTranMap.xml

xmltranmap.filename.2=C:/ECommerce/Version041/config/
BaseDHSurchargeTranMap.xml

xmltranmap.filename.3=C:/ECommerce/Version041/config/
NCRDHTranMap.xml

BASE24-es ATM IFX Manager process:

xmltranmap.filename.0=/user/fnbo/ESDH041/config/
DHBase/DHCommonTranMap.xml

xmltranmap.filename.1=/user/fnbo/ESDH041/config/IFX/
IFXFrameworkTranMap.xml

xmltranmap.filename.2=/user/fnbo/ESDH041/config/IFX/
IFXATMTranMap.xml

xmltranmap.filename.3=/user/fnbo/ESDH041/config/
DHBase/ClassicDHTranMap.xml

Usage: TransactionMapManager

xmltransmetamap.filename.*n* — The fully-qualified name of a Transaction Meta Map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each filename specified. Transaction Meta Map XML files define a series of actions required to process a transaction. You can think of them as work flow descriptors. This property should be defined in the initialization section of the config.properties file because the Transaction Meta Map Manager reads it at initialization to preload message handling details into memory. The data is held in memory for as long as the application is running.

Required: Application-dependent

Example Entries:

BASE24-es ATM Diebold 911/912 and NCR NDC+ Device Handler processes:

xmltransmetamap.filename.0=C:/ECommerce/Version041/
config/DHCommonTransMetaMap.xml

xmltransmetamap.filename.1=C:/ECommerce/Version041/
config/DieboldDHTransMetaMap.xml

xmltransmetamap.filename.2=C:/ECommerce/Version041/
config/BaseDHSurchargeTransMetaMap.xml

xmltransmetamap.filename.3=C:/ECommerce/Version041/
config/DieboldDHSurchargeTransMetaMap.xml

xmltransmetamap.filename.4=C:/ECommerce/Version041/
config/NCRDHTransMetaMap.xml

xmltransmetamap.filename.5=C:/ECommerce/Version041/
config/NCRDHSurchargeTransMetaMap.xml

BASE24-es ATM IFX Manager process:

xmltransmetamap.filename.0=/user/fnbo/ESDH041/config/
DHBase/DHCommonTransMetaMap.xml

xmltransmetamap.filename.1=/user/fnbo/ESDH041/config/
IFX/IFXFrameworkTransMetaMap.xml

xmltransmetamap.filename.2=/user/fnbo/ESDH041/config/
IFX/IFXATMTransMetaMap.xml

xmltransmetamap.filename.3=/user/fnbo/ESDH041/config/
DHBase/ClassicDHTransMetaMap.xml

Usage: TransMetaMapManager

xmltransmetamap.trace.paths — A flag indicating whether the Transaction Meta Map Manager traces path states and writes trace messages to the file specified by the xmltransmetamap.trace.paths.filename property. Valid values are as follows:

true = Yes, trace path state information.

false = No, do not trace path state information.

Required: No

Example Entry: xmltransmetamap.trace.paths=false

Usage: TransMetaMapManager

xmltransmetamap.trace.paths.filename — The name of a file to which path states traced by the Transaction Meta Map Manager are written. You can then analyze the file output to ensure that transactions are being processed as expected.

Required: No

Example Entry: xmltransmetamap.trace.paths.filename=C://src//jdh//config//
TraceState

Usage: TransMetaMapManager

xmltransmetamap.trace.paths.errorconditions — A flag indicating whether the Transaction Meta Map Manager includes error conditions when tracing transaction path states. If so, error conditions are written to the file specified by the xmltransmetamap.trace.paths.filename property. Valid values are as follows:

true = Yes, include error conditions when tracing path state information.

false = No, do not include error conditions when tracing path state information.

Required:	No
Example Entry:	xmltransmetamap.trace.paths.errorconditions=false
Usage:	TransMetaMapManager

Appendix A

BASE24-es User Interface

This appendix describes third-party JARs and JSF properties specific to the BASE24-es Java User Interface servlets for the ACI desktop user interface.

Third-Pary JARs

The BASE24-es Java User Interface servlets use the following third-party JARs from the Sun XML Datatypes Library package to support the data validation mechanisms employed by the BASE24-es UI Java Server environment.

- relaxngDatatype
- xsdlib

System Properties

The following system properties are specific to the BASE24-es Java User Interface servlets.

cjsaf.enabled — A flag indicating whether contingency support is enabled for the BASE24-es Java User Interface servlets. If this property is set to true, the BASE24-es Java User Interface servlets writes messages it receives for requests to insert, delete, or update data in a data source maintained by a Java server to the Java User Interface Contingency Store and Forward data source (UIJSFD) after the requested changes are successfully made to the local data source.

Note that configuration changes to the following data sources are not written to the UIJSFD because they are system specific:

- Application Configuration data source (APPLD)
- External Connections data source (EXTCND)
- Listener data source (LSTND)
- Process data source (PROCD)

Valid values are as follows:

true = Yes, contingency support is enabled.
false = No, contingency support is not enabled. Default.

Required: No

cjsaf.sleep — The amount of time, in seconds, the Java User Interface Server Contingency SAF process sleeps between processing runs of data in the Java User Interface Contingency Store and Forward data source (UIJSFD).

Required: Yes, if the cjsaf.enabled property is set to true

cjsaf.retry — The maximum number of times the Java User Interface Server Contingency SAF process attempts to send a SAF message to another BASE24-es Java User Interface servlets process (Web server) before logging the SAF message in a warning event message and deleting the SAF record from the data source.

Required: Yes, if the `cjsaf.enabled` property is set to true

root.config — Identifies the root location of configuration information for User Interface Framework (UIF) support in the BASE24-es Java User Interface servlets instance.

Required: Yes

Example Entry C:/ESWeb/ESConfig

SCH — A flag indicating whether the BASE24-es Java User Interface servlets run in channel header support mode. Valid values are as follows:

true = Yes, the BASE24-es Java User Interface servlets run in Channel header support mode.

false = No, the BASE24-es Java User Interface servlets do not run in channel header support mode.

When this property is set to true, inbound request messages and responses to those requests will be processed with or without channel header support from the client. Channel header support is a feature of the User Interface Framework (UIF). If this property is set to false, channel header support is turned off.

Note: Support of this feature from the ACI desktop user interface client requires that the configuration file of `../ES/desktop/DesktopLauncher.lax` includes the system property setting of `-DSCH=true`.

Required: Yes

Application Properties

The following application properties configured in the config.properties file or the APPLD that are specific to the BASE24-es Java User Interface servlets.

config.home — Specifies the home location of the configuration files used by the Java User Interface servlets. The value of this property is used by components and services to resolve relative URIs to absolute URIs or file specifications. The value of this property must be in one of the following formats and must end in a forward slash (/):

```
config.home=file:///D:/config/
```

Required: Yes

Example Entries:

```
config.home=/user/B24es_044/ESWeb/ESConfig/
```

```
config.home=file:///C:/user/B24es044/ESWeb/ESConfig/
```

contingency.saf.extconn — Identifies the external HTTP connection to which the BASE24-es Java User Interface servlets sends messages for database contingency processing.

Required: Yes, if you are using database contingency

Example Entry CONTINGENCY

dataValidation.schema.ESXMLDataTypes — Identifies the relative Universal Resource Identifier (URI) location of the data type schema definition file. The value of the config.home property is used to resolve the relative URI to an absolute location.

Required: Yes

Example Entry: schemas/lib/ESXMLDataTypes.xsd

db.audited.db-service-name — Indicates the type of user auditing performed for a particular database service, represented by *db-service-name*, when the db.audited property is set to true. Valid values are as follows:

- none = No auditing is performed for this database service.
- medium = Medium auditing is performed for this database service. For this value, only header information is logged in auditing records.
- full = Full auditing is performed for this database service. For this value, header information as well as detail information about the before and after images of the affected record are logged.

If this property is not defined for a database service, it defaults to a value of full.

Required: No
Example Entry: db.audited.applcnfg=full
Usage: Database Component

The following table lists all Java database service names in the BASE24-es system as well as the data sources and ACI desktop user interface windows associated with each database service. If a database service for a data source is not listed in this table, the data source is maintained by a C++ server. For these servers, auditing is controlled by the UI_AUDIT_LEVEL environment attribute rather than a property. All audit records are written to the User Audit Log data source, regardless of whether the data source is maintained by a Java or C++ server.

Note: All non-database activity on the ACI desktop user interface (e.g., user logon, logoff selected, logoff, password verification, password change, password expiration, session expired, session timeout, and audit report generation) is always audited.

Database Service Name	Data Source	ACI Desktop Window
applcnfg*	Application Configuration data source (APPLD)	Application Configuration window
atds1	ATM Terminal Data Static data source—general data (EATS1D)	Location and Processing tabs of the ATM Channel Configuration window
atmchannelpreviousdaytotalsservice	Previous Day ATM Terminal Data Dynamic data source—general data (EAPDTD)	Previous Days Totals tab of the ATM Channel Configuration window

Database Service Name	Data Source	ACI Desktop Window
atmdynamicdata1service	ATM Terminal Data Dynamic data source— general data (EATD1D)	Hopper and Terminal Activity tabs of the ATM Channel Configuration window, ATM Channel Management windows, and the ATM Channel Administration window
atmdynamicdata2service	ATM Terminal Data Dynamic data source— scratch pad (EATD2D)	Not applicable
atmloadprofiles-service	ATM Configuration Load data source (ATMCND)	ATM Load Profile window
atmreceiptreqtypeservice	Receipt Required data source (RCPTRD)	ATM Receipt Required Type Configuration
atmterminalloaddataservice	ATM Terminal Load data source (EATDLD)	Not applicable
banknetconfigurationservice	Banknet Configuration data source (BNETCD)	Banknet Values and Message Management tabs of the Banknet ISO Interface Configuration window
banknetinstitutionidservice	Banknet Institution Identification data source (BNIID)	Banknet Acquirers tab of the Banknet ISO Interface Configuration window
banknetmerchantprogramservice	Banknet Merchant Program data source (BNTMPD)	Merchant Programs tab of the Banknet ISO Interface Configuration window
cardaccountservice	Card Account data source (CRDAD)	Accounts tab of the Card Management window
cardservice	Card data source (CRDD)	General, Status, and Limits tabs of the Card Management window

Database Service Name	Data Source	ACI Desktop Window
configprofile	ATM Device Handler Configuration data source (ATDCND)	ATM Channel Profile Configuration window
contextcfgservice	Context Configuration data source (CTXCD)	Context Profile Configuration
contingencyinterfaceservice	Contingency Interface Configuration data source (CTGYD)	Contingency Interface Configuration window
currencyconvrateservice	Currency Conversion Rate data source (CCRD)	Currency Conversion Rate Configuration window
derivationkeyservice*	Derivation Key data source (DRVKD)	DUKPT Derivation Key Profile Configuration window
environservice*	Environment data source (ENVMTD)	Environment Configuration window
eurooptedcurrencyservice	Euro Opted Currency data source (EOCD)	Euro Opted Currency Configuration window
extrcnct*	External Connections data source (EXTCND)	External Connection Configuration window
filepartctlgservice*	File Partition Catalog data source (FPCTGD)	Catalog tab of the File Partition Configuration window
filepartfieldsservice*	File Partition Fields data source (FPFLDD)	Fields tab of the File Partition Configuration window
filepartlistservice*	File Partition List data source (FPLSTD)	Catalog tab of the File Partition Configuration window
fileupdaterouterservice	File Update Router data source (FURD)	File Update Router Configuration window

Database Service Name	Data Source	ACI Desktop Window
hopperprofile	Hopper Profile data source (HPRPFD)	ATM Hopper Profile Configuration window
ifxdataservice	IFX data source (IFXDTD)	Not applicable
imtatmtranmappingsservice	IMT Processing Code to ATM Transaction Code data source (IMAPD)	Internal Message Translation (IMT) Transaction Mapping - ATM window
imtpostranmappingsservice	IMT Processing Code to POS Transaction Code data source (IMPPD)	Internal Message Translation (IMT) Transaction Mapping - POS window
imtservice	IMT Interface data source (IMTIFD)	IMT Values tab of the IMT Interface Configuration window
institutionconfigservice	Institution data source (INSTD)	Institution Configuration window
interfacemanagerservice	Interface Manager data source (IFMGRD)	Various tabs of all Interface Configuration windows
interfacesafservice	Interface SAF data source (IFSAFD)	Store-and-Forward (SAF) tab of all Interface Configuration windows
interfaceservice	Interface data source (INTFD)	Various tabs of all Interface Configuration windows
interfacestationsservice	Interface Station data source (IFSTAD)	Station(s) tab of all Interface Configuration windows
interfaceterminalservice	Interface Terminal data source (IFTRMD)	Interface Terminal Configuration window
limitservice	Limits data source (LMTD)	Limit Profile Management window

Database Service Name	Data Source	ACI Desktop Window
linkedcurrencyservice	Linked Currency data source (LNKCD)	Linked Currency Configuration window
listener [*]	Listener data source (LSTND)	Listener Configuration window
mdsinterfaceservice	MasterCard Debit Switch Interface data source (MDSIFD)	MDS tab of the MasterCard Debit Switch (MDS) Interface Configuration window
operatorid	Operator ID data source (OIF)	ATM Diebold Channel Operator ID Configuration window
optionalreceiptprofile	Optional Receipts Profile data source (OR)	ATM Optional Receipt Profile window
positivebalanceservice	Positive Balance data source (PBALD)	Positive Balance Management window
preauthholdservice	Preauthorization Hold data source (PREAUD)	Preauthorization Hold Management window
prefixonuservice	Prefix data source (PRFXD)	On-Us Prefix Configuration window
process [*]	Process data source (PROCD)	Process Configuration window
receiptformatservice	Receipt data source (RCPTD)	ATM Receipt Format Profile Configuration window
secdevcfgservice	Security Device Configuration data source (SECDCD)	Security Device Configuration window
spdh_acd_service	SPDH Action Code Description data source (SPIAD)	Alternate Descriptions tab of the SPDH Device Profile Configuration window

Database Service Name	Data Source	ACI Desktop Window
spdh_message_service	SPDH Message data source (SPIMD)	Message Configuration tab of the SPDH Device Profile Configuration window
spdh_opt_resp_service	SPDH Optional Response data source (SPIOD)	Optional Response Data tab of the SPDH Device Profile Configuration window
spdh_tcd_service	SPDH Transaction Code Description data source (SPITD)	Alternate Descriptions tab of the SPDH Device Profile Configuration window
starinstitutionservice	Star Institution data source (SINSTD)	Star Institution Configuration window
starnetworkrelservice	Star Network Identifier/Instrument Type Relation data source (STRNID)	Star Network Identifier/Instrument Type Relationship Configuration window
surchargeservice	Surcharge data source (SURCHD)	Surcharge Profile Configuration window
systemprefixservice	System Prefix data source (SPRFXD)	System Prefix Configuration window
uicontingencysafservice	Java User Interface Store and Forward data source (UIJSFD)	Not applicable
usageservice	Usages data source (USGD)	Usages Management window
visainterfaceservice	Visa Interface data source (VISIFD)	Visa Processing tab of the VisaNet ISO Interface Configuration window

* Available for BASE24 customers using the Transaction Security Services process.

db.cached.db-service-name — Indicates whether the data source indicated by the database service name, represented by *db-service-name*, is accessed from cache memory or from disk. Valid values are as follows:

true = Yes, perform caching for this database service.
false = No, do not perform caching for this database service. Default.

Refer to the [db.audited.db-service-name](#) property description for a complete list of database services for the Java User Interface servlets and the data sources they access.

If this property is not defined for a database service, it defaults to a value of false.

Note: The caching mechanism of the JSF is completely independent from, and should not be confused with, external memory tables used in online transaction processing (OLTP).

Required: No
Example Entry: db.cached.applcnfg=false
Usage: Cache Manager

db.upshiftdata — Indicates whether the Java application supports the automatic upshifting of string data. Upshifting of string data is handled at the application level when you are using Enscribe for VSAM data sources. In this case, you should set this property to true. If you are using an SQL database, the database may handle upshifting requirements, in which case, this property can be set to false. Valid values are as follows:

true = Yes, the Java application automatically upshifts string data.
false = No, the Java application does not automatically upshift string data.

Required: Yes
Example Entry: db.upshiftdata=false

db.sqlmp — A flag indicating whether the database used in the current JVM executing environment is NonStop SQL/MP. Valid values are as follows:

true = Yes, the database is NonStop SQL/MP.
false = No, the database is not NonStop SQL/MP.

Required: No
Example Entry: db.sqlmp=true

elmtmap.jar.aci.es-elmtmaps — The name of the JAR file containing BASE24-es element to data validation map properties files.

Required: Yes
Example Entry: jars/es-elmtmaps.jar
Usage: Data Validation Manager

extract.userauditlog.interval.days — The number of days to schedule between automatic extracts of the previous day's audit data from User Audit Log data source (UALOGD) to an XML file. If this property is not configured, it defaults to 1 day, which indicates that the automatic extract is performed daily. The `extract.userauditlog.starttime` property indicates the time of day when the automatic extract is started. Scheduled extracts always extract all of the previous day's user audit data to the XML file.

A `timertask.function.n.classname` property must be configured for User Audit Log XML Extract before this property is used in processing. If a `timertask.function.n.classname` property is not configured for User Audit Log XML Extract, automatic extracts of the previous day's audit data are not performed.

Required: No
Example Entry: 7
Usage: User Audit Log XML Extract

extract.userauditlog.starttime — The time of day, in hh:mm format, at which automatic extracts of the previous day's audit data from User Audit Log data source (UALOGD) to an XML file are started. If this property is not configured, it defaults to :30, which indicates that the automatic extract starts at 12:30 am. The `extract.userauditlog.interval.days` property indicates the number of days to schedule between automatic extracts.

A `timertask.function.n.classname` property must be configured for User Audit Log XML Extract before this property is used in processing. If a `timertask.function.n.classname` property is not configured for User Audit Log XML Extract, automatic extracts of the previous day's audit data are not performed.

Required: No
Example Entry: 01:30 (1:30 am)
Usage: User Audit Log XML Extract

filter.field.task-id.n — Indicates field filter support for specific task IDs, where *task-id* is the task ID associated with a specific ACI desktop user interface window or framework and *n* is a sequential number starting with 0 and increased by 1 for multiple filters associated with the same task ID. Task IDs are described in the *ACI Desktop User Interface Manual*.

Note: Filter properties are created and used for internal ACI development. Do not alter these properties at all after your system is installed.

This property guides the Java process to map incoming data values from logical elements in a message to a field ID understood by the filtering implementation configured for the USEC application. USEC filtering is configured on the Filters tab of the User Security Role Configuration window. Values for this property are entered in the following three-part format, with each part separated by a semicolon:

Identifier;field ID;logical message element

where:

Identifier is a code indicating whether the property value is to be used only for a filter, only for USEC maintenance, or for both a filter and for USEC maintenance. Valid values are:

F = Filter only
M = USEC maintenance only
B = Both filter and USEC maintenance

Field ID is the name of the field as known by the USEC application. Currently, filtering is supported for the following fields: InstitutionID, Issuer, and Profile.

Logical message element is the logical element in the message that contains the data.

Filtering support also requires a filter class specification in the appropriate transaction map XML file.

Required: No

Example Entries:

filter.field.InstitutionConfiguration.0=B;InstitutionID;instdInstitutionID

filter.field.InstitutionConfiguration.1=M;Issuer;instdInstitutionID

initialization.function.*n*.classname — Specifies the fully-qualified Java package and class name to be started at initialization by the Initialization Manager, where *n* is a sequential number starting at 0 and increased by 1 for each class to be initialized. These properties should be defined in the config.properties file.

Note: If initialization functions are not sequentially numbered starting at 0 and increased by 1 for each function, the Java server process will stop initializing with the last function correctly numbered. For example, if functions were numbered 0, 1, 3, etc. instead of 0, 1, 2, etc., the Java server process would stop initializing with function 1 because the next function expected would be function 2.

Required: Yes

Example Entries: initialization.function.0.classname=es.server.framework.
TransactionMapManager
initialization.function.1.classname=es.server.common.
DataValidatorManager
initialization.function.2.classname=es.server.formatter.xml.
MessageSchemaManager
initialization.function.3.classname=usec.common.
USECManager

Usage: InitializationManager

LogChannelTimeouts — A flag indicating whether the UAUD application generates error event messages when an audit record it sends to the Java User Interface Servlets for logging to the User Audit Log data source (UALOGD) is not responded to within the allotted time limit. Valid values are as follows:

true = Yes, generate error event messages for late responses to UAUD messages. Default

false = No, do not generate error event messages for late responses to UAUD messages.

Required: No

Example Entries: LogChannelTimeouts=false

Usage: UAUD application

ptc.connectionID — Identifies the unformatted connection ID to use for straight pass-through processing of XML request and response messages. The value of this property must match the name of an external connection ID in the EXTCND that uses a null formatter class.

The Java User Interface servlets support a pass-through component that allows external entities to have work combined within workflow guidelines of other BASE24-es-based components. Additionally, for customization, the pass-through component class can be extended to provide enhanced data handling requirements.

Required: No

Example Entry: `ptc.connectionID=BASE24-es/C++XMLSERVER`

ptcf.connectionID — Identifies the formatted connection ID to use for pass-through processing of XML request and response messages. The value of this property must match the name of an external connection ID in the EXTCND that uses a valid formatter class.

The Java User Interface servlets support a pass-through component that allows external entities to have work combined within workflow guidelines of other BASE24-es-based components. The pass-through component allows request messages to be formatted and response messages to be parsed to and from external entities when the original XML request is not in the required format for those entities. Additionally, for customization, the pass-thru component class can be extended to provide enhanced data handling requirements.

Required: No

Example Entry: `ptc.connectionID=BASE24-es/C++XMLSERVER
FORMATTED`

report.userauditlog.loc — The name of the directory where the user audit extract file is created. For manually started extracts, the user audit extract file name is specified in the Extract file name field on the Audit Extract window on the ACI desktop user interface. For automatic extracts, a default filename of `AudExt_yymmdd_hhmmss.xml` is used for the extract file where *yymmdd* is the date and *hhmmss* is the time of the extract. This property defaults to a value of “{report location},” which must be changed to a valid location after installation.

Note that if the specified directory does not already exist, the audit extract file is not created.

Required: Yes, if using user auditing

Example Entries: `/user/B24es_052/ESWeb/ESConfig/uaf2/P6Q^GATE/reports/
c:/ECommerce/Version52/reports/
///ecommerce/version52/reports/`

timertask.function.*n*.classname — The name of a class to be executed by the UI Timer process, where *n* is a number starting at 0 and increased by 1 for each class to be executed. Each class designates cleanup tasks or the automatic user audit log extract task to be performed for the JSF application. These properties should be defined in the config.properties file. Note that the property for the Java User Interface Contingency SAF process is only required if the cjsaf.enabled system property is set to true.

Required: Yes

Example Entries: timertask.function.0.classname=com.aciworldwide.common.
timertask.UserAuditLogCleanup
timertask.function.1.classname=es.server.timertask.
ContingencySAFProcess
timertask.function.2.classname=com.aciworldwide.common.
timertask.UserAuditLogXMLeXtract

Usage: Timer Task Manager

timertask.manager.processid — The process ID of the BASE24-es Java User Interface servlets (Web Server) process designated to run timed tasks only. This process can be referred to as the UI Timer process, which performs the functions specified in the timertask.function.*n*.classname properties.

Required: Yes

Example Entry: timertask.manager.processid=JUITIMR1

Usage: Timer Task Manager

trace.dataobject.enabled — A flag indicating whether trace messages are enabled for data object output from the Java server process. Tracing of messages is configured within the TraceConfig.xml file that is specified by the trace.log4j.config.url property. When this property is set to true, the MessageSchema class dumps traces of the contents of the primary data object during request message processing and response message building.

Note: The Trace facility was designed as a diagnostic tool for development and test environments and not for general use in a production environment. ACI strongly recommends that it not be used in a production environment due to the overhead and impacts on performance.

Required: No

Example Entry: trace.dataobject.enabled=false

Usage: Primary data object

tranmap.jar.aci.tranmap-name — The relative URI location of a Transaction Map XML JAR file, where *tranmap-name* is the name of the XML JAR file without the .jar file extension. A separate property is required for every Transaction Map XML JAR file used. To meet performance requirements, the TransactionMapManager preloads all information contained in this configuration JAR file into individual TransactionMapField objects, with each object representing an <entry> from the configuration file. The value of the config.home property is used to resolve the relative URI location to an absolute URI or file specification.

Required: Yes

Example Entries

tranmap.jar.aci.atm-transmaps=jars/atm-transmaps.jar

tranmap.jar.aci.es-transmaps=jars/es-transmaps.jar

Usage: Transaction Map Manager

usec.host — The name of the remote host (name or IP address) where the USEC application is executing. The USEC Interface (USECIF) component uses this property value to communicate with the USEC application.

Required: Yes

Example Entries: usec.host=usecprod1
usec.host=1.160.10.240

Usage: USEC Interface component

usec.port — The remote port number through which the USEC Interface (USECIF) component communicates with the USEC application.

Required: Yes

Example Entry: usec.port=4672

Usage: USEC Interface component

xml.msgschema.jar.aci.schema-name — The relative URI location of an XML Schema Definition (XSD) JAR file, where *schema-name* is the name of the JAR file without the .jar file extension. A separate property is required for every XSD JAR file used. The MessageSchemaManager uses the specified JAR file at initialization to locate and preload request and response message schema definitions. The value of the config.home property is used to resolve the relative URI location to an absolute URI or file specification.

Required: Yes

Example Entries:

xml.msgschema.jar.aci.atm-schemas=jars/atm-schemas.jar

xml.msgschema.jar.aci.es-schemas=jars/es-schemas.jar

Usage: Message Schema Manager

Sample Config.properties File

The following is an example of the config.properties file used for the BASE24-es Java User Interface servlets. This file may need to be edited during installation for site-specific information.

```
#
# Number of days to leave rows in the auditlog.
# Default is thirty days.
#
cleanup.userauditlog.retention.days=30

#
# Time of day to start the cleanup task.
# Default starttime is 23:59 "midnight"
#
cleanup.userauditlog.cleanup.starttime=23:59

#
# Time period between runs of the cleanup process if it should be run more
# frequently than every twenty four hours.
# Default 24 hours.
#
cleanup.userauditlog.timeperiod.hours=24

#
# Dataload jars
#
dataload.jar.aci.atm-dataloads=jars/atm-dataloads.jar
dataload.jar.aci.es-dataloads=jars/es-dataloads.jar

#
# Database properties
#
db.max.connections=10

#
# {user id} - user ID to use for applications to access the database
# {user password} - user password for the user ID above accessing the database
#
db.user.id={user id}
db.user.password={user password}

#
# {driver specification} - should be set according to JDBC driver requirements
# and
# may include information such as:
#
```

```
#          - host or IP address where database resides
#          - port to connect to for database
#          - name of database on the host
#          - other JDBC driver settings
#
db.datasource=jdbc:{driver specification}

#
# db.static.connections defines the number of connections to acquire to the
# database
# when the jvm starts.  These connections will stay active through the life of
# the
# executing JVM.
#
db.static.connections=2

#
# db.connection.wait.time defines the amount of time, in milliseconds, to wait
# for
# another thread to free a Database connection in the Database connection
# manager.
# This is used when the maximum number of connections have been acquired as
# specified
# in the db.max.connections property.
#
db.connection.wait.time=1000

#
# db.useisolationlevel indicates that the database services will automatically
# manipulate
# connection transaction isolation levels.  This should be set to true for
# Microsoft
# SQL Server and IBM's DB2 databases.  It should be set to false for Oracle and
# Tandem SML/MP
# databases.  When set to true database access will automatically be set to
# reduce
# database contention on queries where reading through locks is valid.
#
db.useisolationlevel=true

#
# db.driver specifies the class name of the JDBC driver to use for all database
# connections.
# The whole package name as it is found in the jar file containing the driver
# should be specified.
#
# {driver package} - should be set to the Java package specification of the
# JDBC driver being used
#
db.driver={driver package}
db.sqlmp=false
db.type=JDAL
```

```
#
# Error Manager properties. If the filename contains the characters
# "YYYYMMDD" in its name this will cause the ErrorManager to roll log
# files on a daily basis. If these characters are not found in the name
# the same log file will remain open.
#
# LogManager properties.
# The Log4J config file location for trace and event
#
# DOS specifier - change {root} to drive: or drive:/{other directory} to
#                  be prefixed to value below (Example - C: or C:/ESWeb)
#
# UNIX specifier - change {root} to root location to be prefixed to value
#                  below (Example - db01, db01/ESWeb)
#
# {root} setting should be same as that for TraceConfig.xml below.
#
event.log4j.config.url=file:/// {root}/ESConfig/EventConfig.xml

#
# Gens jars.
#
gens.jar.aci.atm-gens=jars/atm-gens.jar
gens.jar.aci.es-gens=jars/es-gens.jar

#
# Initialization Manager Properties.
#
initialization.function.0.classname=es.server.framework.TransactionMapManager
initialization.function.1.classname=es.server.common.DataValidatorManager
initialization.function.2.classname=es.server.formatter.xml.MessageSchemaManager
initialization.function.3.classname=usec.common.USECManager
#initialization.function.4.classname=com.aciworldwide.common.
MessageRouterService

#
# Messaging properties.
#
#
# Valid values for message.environment are:
#   DIRECT
#   JMS
#
#message.environment=JMS
#
#   OPENJMS
#
#message.jms.jndi.initialcontextfactory=org.exolab.jms.jndi.rmi.
RmiJndiInitialContextFactory
#message.jms.jndi.providerurl=rmi://{jms host}:{jms port}/JndiServer
#message.jms.QueueConnectionFactory=JmsQueueConnectionFactory
```

```
#message.jms.TopicConnectionFactory=JmsTopicConnectionFactory
#
#  XPNET with embedded JNDI
#
#message.jms.jndi.initialcontextfactory=com.aci.jms.jndi.embedded.
EmbeddedXpnetInitialContextFactory
#message.jms.jndi.providerurl=jms://dummy
#message.jms.QueueConnectionFactory=JmsQueueConnectionFactory
#message.jms.TopicConnectionFactory=JmsTopicConnectionFactory

#
#  The following MUST be set to true if running on the HP NSK platform
#  due to how it handles threading.
#
#message.jms.use.nsk.threads=false

#
#  The process id of the one process that is to run
#  the cleanup tasks.
#
timertask.manager.processid={timer process id}

#
#  Timer Task Manager Properties.
#  For every entry increment the function counter.  Each class must implement
the
#  TimerTaskInterface.
#
timertask.function.0.classname=com.aciworldwide.common.timertask.
UserAuditLogCleanup
#timertask.function.1.classname=es.server.timertask.ContingencySAFProcess
timertask.function.1.classname=com.aciworldwide.common.timertask.
UserAuditLogXMLExtract

#
#  Trace processing properties.
#
#  DOS specifier - change {root} to drive: or drive:/{other directory} to
#                  be prefixed to value below (Examples - C:, C:/ESWeb)
#
#  UNIX specifier - change {root} to root location to be prefixed to value
#                  below (Examples - db01, db01/ESWeb)
#
#  {root} setting should be same as that for EventConfig.xml above.
#
trace.log4j.config.url=file:/// {root} /ESConfig/TraceConfig.xml
trace.enabled=false

#
#  BASE24-es Properties.
#
#  DOS specifier - change {root} to drive: or drive:/{other directory} to
```

```
#           be prefixed to value below (Examples - C:, C:/ESWeb)
#
#  UNIX specifier - change {root} to root location to be prefixed to value
#                   below (Examples - db01, db01/ESWeb)
#
#  {root} setting should be same as that for EventConfig.xml and TraceConfig.xml
#  above.
#
config.home={root}/ESConfig/

#
#  Identifies remote host and port where USEC environment is executing.
#
usec.host={host}
usec.port={port}
```

Appendix B

BASE24-es ATM Device Handler and IFX Manager Processes

This appendix describes the JSF properties specific to BASE24-es ATM Device Handler or IFX Manager processes.

Properties

The following application properties configured in the config.properties file or the APPLD are specific to the BASE24-es ATM Device Handler or IFX Manager processes. Some of the properties defined depend on the platform on which BASE24-es is installed.

In some property names, the following values indicate whether a property is specific to a device type or contains a default value.

01 = NCR NDC+
02 = Diebold 911/912
03 = IFX
Dflt = Default

ATMCHNLADMNPTC.connectionID — Identifies the queue on which channel administrative requests are to be placed. The value of this property must match an external ID configured in the External Connections data source (EXTCND).

Required: Yes
Example Entry: ATM_CHANNEL_ADMIN
Usage: BASE24-es Java User Interface servlets

ATMCHANNEL.connectionID.01 — Identifies the name of the connection ID to be used from the External Connections data source (EXTCND) for ATM terminals using the NCR NDC+ message format. Messages other than administrative messages from the BASE24-es ATM Device Handler process are sent to this connection.

Required: Yes, if using NCR NDC+ terminals.
Example Entry: NCR-JMS
Usage: BASE24-es Java User Interface servlets

ATMCHANNEL.connectionID.02 — Identifies the name of the connection ID to be used from the External Connections data source (EXTCND) for ATM terminals that use the Diebold 911 or 912 message format. Messages other than administrative messages from the BASE24-es ATM Device Handler process are sent to this connection.

Required: Yes, if using Diebold 911/912 terminals.
Example Entry: DIEBOLD-JMS
Usage: BASE24-es Java User Interface servlets

ATMCHANNEL.connectionID.03 — Identifies the name of the connection ID to be used from the External Connections data source (EXTCND) for IFX terminals. Messages other than administrative messages from the BASE24-es ATM IFX Manager process are sent to this connection.

Required: Yes, if using IFX terminals.
Example Entry: IFX-JMS
Usage: BASE24-es Java User Interface servlets

ATMRELOAD.connectionID — Identifies the name of the connection ID to be used from the External Connections data source (EXTCND) for requests to reload XML configuration documents into memory.

Required: Yes
Example Entry: ATM-RELOAD
Usage: BASE24-es Java User Interface servlets

atmcutover.clear.atd totals.on.eopp — A flag indicating whether the ATM Cutover process clears ATM hopper totals and transaction amounts and counts from the BASE24-es ATM Terminal Data Dynamic data source—general data source (EATD1D) during terminal cutover processing. Valid values are as follows:

true = Yes, the ATM Cutover process clears totals from the EATD1D during terminal cutover processing.
false = No, the ATM Cutover process does not clear totals from the EATD1D during terminal cutover processing. Default.

Required: No
Example Entry: false
Usage: ATM Cutover process

atm.dh.save.previous.day.totals — A flag indicating whether the previous day's totals for an ATM terminal are written to the Previous Day ATM Terminal Data Dynamic data source—general data (EAPDTD) when you enter a balancing transaction for the ATM or only during forced terminal cutover processing by the End-of-Period Processing (EOPP) process.

When this property is set to a value of false, the Previous Day Activity tab is disabled on the ATM Channel Configuration window. When this property is set to a value of true, this tab is enabled. Valid values are as follows:

true = Yes, previous day totals are written to the EAPDTD before terminal cutover processing.

false = No, previous day totals are not written to the EAPDTD. Default.

Required: No

Usage: ATM Cutover process

cutover.saf.max.rowcount — The maximum number of rows the Cutover SAF process can retrieve from the BASE24-es ATM Cutover SAF data source (COSAFD) in a single fetch operation. If this property is not configured, it defaults to 10.

Required: No

Example Entry: 5

Usage: Cutover SAF Timer Task

cutover.saf.msg.retries — The maximum number of times the Cutover SAF process attempts to send a cutover message to the authorizer for logging. If the cutover message cannot be successfully sent after the maximum number of retries is reached, it is written to the Exception Log data source (EXLOGD). If this property is not configured, it defaults to a value of 2.

Required: No

Example Entry: 3

Usage: Cutover SAF Timer Task

cutover.saf.send.interval — The number of seconds the Cutover SAF process waits between sending cutover record messages to the authorizer for logging. A value of 0 indicates that there is no delay between the sending of messages. If this property is not configured, it defaults to a value of .25 seconds.

Required: No

Example Entry: 1

Usage: Cutover SAF Timer Task

cutover.saf.start.delay.minutes — The number of minutes to delay the start of cutover SAF processing after the Cutover SAF process is started. If this property is not configured, it defaults to a value of 15 minutes.

Required: No
Example Entry: 1
Usage: Cutover SAF Timer Task

cutover.saf.run.delay.minutes — The number of minutes the Cutover SAF process waits between searching the BASE24-es ATM Cutover SAF data source (COSAFD) for cutover rows to be sent to the authorizer for logging. If this property is not configured, it defaults to a value of 15 minutes.

Required: No
Example Entry: 20
Usage: Cutover SAF Timer Task

dataValidation.schema.ATMXMLDataTypes — Identifies the relative URI location of BASE24-es ATM Device Handler data type schema definition files. More than one may exist.

Required: Yes
Example Entry: schemas/lib/ATMXMLDataTypes.xsd

db.cached.db-service-name — Indicates whether the data source indicated by the database service name, represented by *db-service-name*, is accessed from cache memory or from disk. Valid values are as follows:

true = Yes, this data source is accessed from cache memory.
false = No, this data source is not accessed from cache memory.

db-service-name values used by the BASE24-es ATM Device Handler or IFX Manager processes are shown in the following table.

ACI recommends that all of these data sources be cached.

Database Service Name	Data Source	ACI Desktop Window
acqtxnal	Acquirer Transactions Allowed data source (ACQTXD)	Acquirer Transactions Allowed Configuration window
atds1	BASE24-es ATM Terminal Data Static data source—general data (EATS1D)	Location and Processing tabs of the ATM Channel Configuration window
atmcnfg	ATM Terminal Data Dynamic data source—general data (EATD1D)	Hopper and Terminal Activity tabs of the ATM Channel Configuration window, ATM Channel Management windows, and the ATM Channel Administration window
configprofile	ATM Device Handler Configuration data source (ATDCND)	ATM Channel Profile Configuration window
hopperprofile	Hopper Profile data source (HPRPFD)	ATM Hopper Profile Configuration window
optionalreceiptprofile	Optional Receipts data source (OR)	ATM Optional Receipt Profile window
receiptrqrd	Receipt Required data source (RCPTRD)	ATM Receipt Required Type Configuration
receipt	Receipt data source (RCPTD)	ATM Receipt Format Profile Configuration window
operid	Operator ID data source (OIF)	ATM Diebold Channel Operator ID Configuration window
inst	Institution data source (INSTD)	Institution Configuration window

Database Service Name	Data Source	ACI Desktop Window
extrcnct	External Connections data source (EXTCND)	External Connection Configuration window
listener	Listener data source (LSTND)	Listener Configuration window

devicehandler.atmlog.cleanup.timeperiod.hours — The number of hours to delay between runs of the ATM Native Message Log Cleanup task to delete old ATM native messages from the ATM Native Message Log data source (ATLGD). If this property is not configured, it defaults to 24 hours.

Required: No
 Example Entry: 48
 Usage: ATM Native Message Log Cleanup Timer Task

devicehandler.atmlog.cleanup.starttime — The time of day, in hh:mm format, the ATM Native Message Log Cleanup timer task is to run. If this property is not configured, it defaults to a value of 23:59.

Required: No
 Example Entry: 01:30
 Usage: ATM Native Message Log Cleanup Timer Task

devicehandler.atmlog.cleanup.retention.days — The number of days statement data rows are to be retained in the ATM Native Message Log data source (ATLGD). Rows older than this retention period are deleted when the ATM Native Message Log Cleanup timer task is run. If this property is not configured, it defaults to 7 days.

Required: No
 Example Entry: 2
 Usage: ATM Native Message Log Cleanup Timer Task

devicehandler.balance.enquiry.reversal — A flag indicating whether balance inquiry transactions are reversed. Valid values are as follows:

true = Yes, balance inquiry transactions are reversed.
 false = No, balance inquiry transactions are not reversed. Default.

Required: No
Example Entry: true
Usage: All ATM device types supporting balance inquiry reversal commands

devicehandler.dkm.key.change.interval — The maximum time allowed (in hours) between PIN and MAC key changes for an ATM terminal. The default value is 0, which indicates that a key change interval is not used. The difference between the current time and the DKM key change timestamp for a terminal can not exceed the value specified by this property without a key change being performed. When a key change is performed, due to a dynamic key change limit or this interval being exceeded, the DKM key change timestamp for the terminal is updated. Valid values for this property are 0–32767. The default value is 0, which indicates that a key change interval is not used.

Required: No
Example Entry: 144
Usage: All ATM device types

devicehandler.dkm.mac.error.limit — The maximum number of consecutive MAC errors for an ATM, before both the MAC key and PIN key must be changed in the ATM. If this property is not configured, it defaults to a value of 3. A value of 0 for this property indicates that a consecutive MAC error limit is not used. When a dynamic key management limit or interval is exceeded, the Device Handler process initiates a download of a new MAC key and a new PIN key to the terminal, resets all dynamic key management counters for the terminal to 0, and updates the key change timestamp for the terminal.

Required: No
Example Entry: 4
Usage: All ATM device types

devicehandler.dkm.mac.transaction.limit — The maximum number of transactions for an ATM for which the MAC is authenticated or generated, before both the MAC key and PIN key must be changed in the ATM. If this property is not configured, it defaults to 100 transactions. A value of 0 for this property indicates that a MAC transaction limit is not used. When a dynamic key management limit or interval is exceeded, the Device Handler process initiates a download of a new MAC key and a new PIN key to the terminal, resets all dynamic key management counters for the terminal to 0, and updates the key change timestamp for the terminal.

Required: No
Example Entry: 200
Usage: All ATM device types

devicehandler.dkm.pin.error.limit — The maximum number of PIN errors for an ATM, before both the MAC key and PIN key must be changed in the ATM. If this property is not configured, it defaults to a value of 3. A value of 0 for this property indicates that a PIN error limit is not used. When a dynamic key management limit or interval is exceeded, the Device Handler process initiates a download of a new MAC key and a new PIN key to the terminal, resets all dynamic key management counters for the terminal to 0, and updates the key change timestamp for the terminal.

Required: No
Example Entry: 4
Usage: All ATM device types

devicehandler.dkm.pin.transaction.limit — The maximum number of transactions for an ATM for which the PIN is verified, before both the MAC key and PIN key must be changed in the ATM. If this property is not configured, it defaults to a value of 1000 transactions. A value of 0 for this property indicates that a PIN transaction limit is not used. When a dynamic key management limit or interval is exceeded, the Device Handler process initiates a download of a new MAC key and a new PIN key to the terminal, resets all dynamic key management counters for the terminal to 0, and updates the key change timestamp for the terminal.

Required: No
Example Entry: 2000
Usage: All ATM device types

devicehandler.maximum.statement.data.chunk.size — Specifies how many bytes of data the Device Handler process can store in a single row in the Statement Print data source (STMTD). The maximum size is 4000 bytes. The actual size is determined by the configured size of the StmtData element in meta data. The configured size may be less than 4000 bytes.

Required: No
Usage: All ATM device types supporting statement print transactions

devicehandler.max.rowcount.inactivity — The maximum number of rows the Transaction Inactivity Timer task can retrieve from the BASE24-es ATM Terminal Data Dynamic data source—general data (EATD1D) in a single fetch operation. If this property is not configured, it defaults to 10.

Required: No
Usage: Transaction Inactivity Timer Task

devicehandler.max.rowcount.load.inactivity — The maximum number of rows the Load Inactivity Timer task can retrieve from the BASE24-es ATM Terminal Data Dynamic data source—general data (EATD1D) in a single fetch operation. If this property is not configured, it defaults to 10.

Required: No
Usage: Load Inactivity Timer Task

devicehandler.message.logging — A flag indicating whether the Device Handler process logs external messages to the ATM Native Message Log data source (ATLGD). Valid values are as follows:

true = Yes, external messages are logged to the ATLGD.
false = No, external messages are not logged to the ATLGD. Default.

Required: No
Usage: All ATM device types

devicehandler.run.delay.minutes.inactivity — The number of minutes the Transaction Inactivity Timer task waits between searching the BASE24-es ATM Terminal Data Dynamic data source—general data (EATD1D) for ATM terminals that have exceeded their transaction inactivity timer limit. The ATM Device Handler process generates event messages and resets the transaction inactivity timer when a transaction has not been initiated from the terminal within the specified time interval. If this property is not configured, it defaults to 5 minutes.

Required: No
Example Entry: 10
Usage: Transaction Inactivity Timer Task

devicehandler.run.delay.minutes.load.inactivity — The number of minutes the Load Inactivity Timer task waits between searching the BASE24-es ATM Terminal Data Dynamic data source—general data (EATD1D) for ATM terminals that have exceeded their transaction load inactivity timer limit. The

ATM Device Handler process generates event messages and resets the load inactivity timer when a transaction has not been initiated from the terminal within the specified time interval. If this property is not configured, it defaults to 5 minutes.

Required: No
Example Entry: 10
Usage: Load Inactivity Timer Task

devicehandler.start.delay.minutes.inactivity — The number of minutes to delay the start of transaction inactivity timer processing after the BASE24-es ATM Device Handler process is started. If this property is not configured, it defaults to 5 minutes.

Required: No
Example Entry: 1
Usage: Transaction Inactivity Timer Task

devicehandler.start.delay.minutes.load.inactivity — The number of minutes to delay the start of load inactivity timer processing after the BASE24-es ATM Device Handler process is started.

Required: No
Example Entry: 1
Usage: Load Inactivity Timer Task

devicehandler.tss.external.connection — The ID of the external connection to use for sending requests to the Transaction Security Services process. The value of this property must match an external connection ID in the External Connection Configuration data source (EXTCND).

Required: Yes
Example Entry: ATM_TSS
Usage: All ATM device types

devicehandler.phonecard.amount.value.currency-code.n — An amount to displayed on the phone card purchase transaction selection screen at the ATM. *Currency-code* specifies the currency in which the amount is displayed and *n* is a number from 1 to 4 that indicates whether the amount is the first, second, third, or

fourth choice displayed. You must define `devicehandler.phonecard.amount.value.currency-code.n` properties for each currency in which amounts can be displayed on the phone card purchase transaction selection screen at the ATM.

Required: Yes, for phone card purchase transactions

Example Entries: `devicehandler.phonecard.amount.value.826.1=165`
`devicehandler.phonecard.amount.value.826.2=440`
`devicehandler.phonecard.amount.value.826.3=825`
`devicehandler.phonecard.amount.value.826.4=1375`
`devicehandler.phonecard.amount.value.840.1=300`
`devicehandler.phonecard.amount.value.840.2=800`
`devicehandler.phonecard.amount.value.840.3=1500`
`devicehandler.phonecard.amount.value.840.4=2500`

Usage: All ATM device types

devicehandler.phonecard.minutes.value.n — The number of minutes to be purchased in a phone card purchase transaction, where *n* indicates the choice (first, second, third, or fourth), this value is associated with on the phone card purchase transaction selection screen at the ATM.

Required: Yes, for phone card purchase transactions

Example Entries: `devicehandler.phonecard.minutes.value.1=10`
`devicehandler.phonecard.minutes.value.2=30`
`devicehandler.phonecard.minutes.value.3=50`
`devicehandler.phonecard.minutes.value.4=120`

Usage: All ATM device types

diebold.override.operator.id.access — A flag indicating whether the BASE24-es Diebold 911/912 Device Handler process checks the Diebold Operator ID data source (OIF) to determine access to the ATM chest door. This property must be set to true for the host-controlled solenoid-activated lock feature. Valid values are as follows:

true = Yes, use the OIF to determine access to the ATM chest door.
false = No, do not use the OIF to determine access to the ATM chest door.
Default.

Required: Yes, if using the host-controlled solenoid-activated lock feature for the BASE24-es Diebold 911/912 Device Handler process

Usage: Diebold 911/912 terminals

elmtmap.jar.aci.atm-elmtmaps — The name of the JAR file containing BASE24-es ATM Device Handler element to data validation map properties files.

Required: Yes

Example Entry: jars/atm-elmtmaps.jar

ifx.formatter.ifx.ns.is.default.ns — A flag indicating whether the IFX namespace is the default namespace of XML output documents. XML namespaces provide a simple method for qualifying element and attribute names used in Extensible Markup Language documents by associating them with namespaces identified by URI references. Valid values are as follows:

true = Yes, the IFX namespace is the default namespace of XML output documents. Default.

false = No, the IFX namespace is not the default namespace of XML output documents.

Required: Yes, for the BASE24-es IFX Manager process

Example Entry: true

Usage: IFX terminals

ifx.message.support.filename.n — Specifies the name of an IFX message support map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each file to be initialized. IFX message support map XML files identify the IFX service wrappers supported by the IFX Manager process and the messages within each wrapper that are supported. The IFX Manager process uses this file two ways—to validate IFX documents received and to list the messages supported in a Service Profile Inquiry response. This property is defined in the Application Configuration data source (APPLD). The Message Support Map Manager reads these files into memory at initialization. The data is held in memory for as long as the application is running.

Required: Yes, for IFX Manager process.

Example Entries: ifx.message.support.filename.0=/user/fnbo/ESDH041/config/IFX/IFXFrameworkMessageSupportMap.xml
ifx.message.support.filename.1=/user/fnbo/ESDH041/config/IFX/IFXATMMessageSupportMap.xml

Usage: Message Support Map Manager

ifx.status.code.map.n — Specifies the name of an IFX status code map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each file to be initialized. IFX status code map XML files map IFX status codes to a severity

(Info, Warn, or Error) and an optional description. Status codes are provided in the response to an IFX device. The `StatusCode` and `Severity` are required elements (as they are required to be sent by the IFX schema), while the `StatusDesc` element is optional. These files should be configured in the Application Configuration data source (APPLD). The IFX Status Code Map Manager reads them at initialization to preload status code mapping details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for IFX Manager process.
Example Entries: `ifx.status.code.map.0=/user/fnbo/ESDH041/config/IFX/IFXStatusCodeMap.xml`
Usage: IFX Status Code Map Manager

initialization.function.*n*.classname — Specifies the fully-qualified Java package and class name to be started at initialization by the Initialization Manager, where *n* is a sequential number starting at 0 and increased by 1 for each class to be initialized. These properties should be defined in the `config.properties` file.

Note: If initialization functions are not sequentially numbered starting at 0 and increased by 1 for each function, the Java server process stops initializing with the last function correctly numbered. For example, if functions are numbered 0, 1, 3, etc. instead of 0, 1, 2, etc., the Java server process stops initializing with function 1 because the next function expected is function 2.

Required: Yes
Example Entries:

Diebold 911/912 and NCR NDC+ Device Handler processes:

```
initialization.function.0.classname=com.aciworldwide.  
framework.TransactionMapManager  
initialization.function.1.classname=com.aciworldwide.  
common.TransMetaMapManager  
initialization.function.2.classname=com.aciworldwide.base24.  
es.dh.common.config.OpCodeMapManager  
initialization.function.3.classname=com.aciworldwide.base24.  
es.dh.common.config.CurrencySymbolMapManager  
initialization.function.4.classname=com.aciworldwide.base24.  
es.dh.common.config.ReceiptTemplateService  
initialization.function.5.classname=com.aciworldwide.base24.  
es.dh.common.config.ReceiptRequiredFlagsService  
initialization.function.6.classname=com.aciworldwide.base24.  
es.dh.common.config.StateMapManager
```

initialization.function.7.classname=com.aciworldwide.base24.
es.dh.common.config.ReceiptMapManager
initialization.function.8.classname=com.aciworldwide.base24.
es.dh.common.config.LocaleMapManager
initialization.function.9.classname=com.aciworldwide.base24.
es.dh.common.config.DispenseAlgorithmMapManager
initialization.function.10.classname=com.aciworldwide.
base24.es.common.config.ISO93MessageMapManager
initialization.function.11.classname=com.aciworldwide.
base24.es.dh.common.config.PrintTagConversionMap
Manager
initialization.function.12.classname=com.aciworldwide.
base24.es.dh.common.config.PrivateUseDataMapManager
initialization.function.13.classname=com.aciworldwide.
base24.es.dh.common.config.AuthMapManager
initialization.function.14.classname=com.aciworldwide.
base24.es.common.config.MessageClassifierMapManager
initialization.function.15.classname=com.aciworldwide.
common.MessageRouterService

IFX Manager process:

initialization.function.0.classname=com.aciworldwide.
framework.TransactionMapManager
initialization.function.1.classname=com.aciworldwide.
common.TransMetaMapManager
initialization.function.2.classname=com.aciworldwide.base24.
es.ifx.framework.config.ActionCodeStatusCodeMapManager
initialization.function.3.classname=com.aciworldwide.base24.
es.ifx.framework.config.ReversalReasonCodeMapManager
initialization.function.4.classname=com.aciworldwide.base24.
es.ifx.framework.config.MessageSupportMapManager
initialization.function.5.classname=com.aciworldwide.base24.
es.dh.common.config.TokenMapManager
initialization.function.6.classname=com.aciworldwide.base24.
es.dh.common.config.TokenRequiredMapManager
initialization.function.7.classname=com.aciworldwide.base24.
es.dh.common.config.AuthMapManager
initialization.function.8.classname=com.aciworldwide.base24.
es.common.config.MessageClassifierMapManager
initialization.function.9.classname=com.aciworldwide.base24.
ifx.jaxb.JAXBContextManager

initialization.function.10.classname=com.aciworldwide.
 base24.es.dh.common.config.classic.LCONFManager
 initialization.function.11.classname=com.aciworldwide.
 base24.es.ifx.framework.config.IFXStatusCodeMapManager
 initialization.function.12.classname=com.aciworldwide.
 common.MessageRouterService
 Usage: InitializationManager

jaxb.context.n — The Java API for XML binding (JAXB) context path for IFX messages.

Required: Yes, for the BASE24-es IFX Manager process
 Example Entry: com.aciworldwide.base24.ifx.schema.v1_4_0:com.
 aciworldwide.base24.ifx.schema.v1_4_0.ACImods
 Usage: IFX terminals

message.jms.TopicConnectionFactory — The name of the topic connection factory used to automatically create topic connection objects with a publish/subscribe JMS provider.

Required: No
 Example Entry: message.jms.TopicConnectionFactory=JmsTopicConnection
 Factory
 Usage: JMS Connection Manager

ncr.mac.field.select.data.classname — The pathname of the class used to select fields to be included in a message authentication code (MAC) for the BASE24-es NCR NDC+ Device Handler process. This property is required only if you are performing message authentication between BASE24-es and NCR NDC+ terminals.

Note: Message authentication is not currently supported.

Required: Yes, if performing message authentication between
 BASE24-es and NCR NDC+ terminals
 Example Entry: com.aciworldwide.base24.es.dh.ncr.valueobjects.
 DefaultMacFieldSelectionData
 Usage: NCR NDC+ terminals

reversal.saf.max.rowcount — The maximum number of rows the Reversal SAF process can retrieve from the BASE24-es ATM Reversal SAF data source (RS) in a single fetch operation. If this property is not configured, it defaults to 10.

Required: No
Example Entry: 5
Usage: Reversal SAF Timer Task

reversal.saf.run.delay.minutes — The number of minutes the Reversal SAF process waits between searching the BASE24-es ATM Reversal SAF data source (RS) for reversal rows to be sent to the authorizer for processing. If this property is not configured, it defaults to 15 minutes.

Required: No
Example Entry: 20
Usage: Reversal SAF Timer Task

reversal.saf.start.delay.minutes — The number of minutes to delay the start of reversal SAF processing after the Reversal SAF process is started. If this property is not configured, it defaults to 15 minutes.

Required: No
Example Entry: 1
Usage: Reversal SAF Timer Task

reversal.saf.msg.retries — The maximum number of times the Cutover SAF process attempts to send a reversal message to the authorizer for processing. If the reversal message cannot be successfully sent after the maximum number of retries is reached, it is written to the Exception Log data source (EXLOGD). If this property is not configured, it defaults to a value of 2.

Required: No
Example Entry: 3
Usage: Reversal SAF Timer Task

reversal.saf.send.interval — The number of seconds the Reversal SAF process waits between sending reversal messages to the authorizer for processing. A value of 0 indicates that there is no delay between the sending of messages. If this property is not configured, it defaults to a value of .25 seconds.

Required: No
Example Entry: 1
Usage: Reversal SAF Timer Task

statementdata.cleanup.timeperiod.hours — The number of hours to delay between runs of the Statement Data Cleanup task to delete old statement data from the Statement Print data source (STMTD). If this property is not configured, it defaults to 24 hours.

Required: No
Example Entry: 48
Usage: Statement Data Cleanup Timer Task

statementdata.cleanup.starttime — The time of day, in hh:mm format, the Statement Data Cleanup timer task is to run. If this property is not configured, it defaults to a value of 23:59.

Required: No
Example Entry: 01:30
Usage: Statement Data Cleanup Timer Task

statementdata.cleanup.retention.days — The number of days statement data rows are to be retained in the Statement Print data source (STMTD). Rows older than this retention period are deleted when the Statement Data Cleanup timer task is run. If this property is not configured, it defaults to 1 day.

Required: No
Example Entry: 2
Usage: Statement Data Cleanup Timer Task

timertask.function.n.classname — The name of a class to be executed by the ATM Timer process, where *n* is a number starting at 0 and increased by 1 for each class to be executed. Each class designates cleanup tasks to be performed for the JSF application. These properties should be defined in the config.properties file.

Required: Yes

Example Entries: `timertask.function.0.classname=com.aciworldwide.base24.es.dh.common.timer.timertask.ReversalSAFTimerTask`
`timertask.function.1.classname=com.aciworldwide.base24.es.dh.common.timer.timertask.CutoverSAFTimerTask`
`timertask.function.2.classname=com.aciworldwide.base24.es.dh.common.timer.timertask.InactivityTimerTask`
`timertask.function.3.classname=com.aciworldwide.base24.es.dh.common.timer.timertask.LoadInactivityTimerTask`

Usage: Timer Task Manager

xmlactioncodestatuscodemap.filename. — The name of the Action Code to Status Code Map XML file that maps ISO and internal action codes to IFX status codes. This property should be defined in the Application Configuration data source (APPLD). The Action Code to Status Code Map Manager reads this file into memory at initialization. The data is held in memory for as long as the application is running.

Required: Yes, for an IFX Manager process.

Example Entries: `xmlactioncodestatuscodemap.filename=/user/fnbo/ESDH041/config/IFX/ActionCodeStatusCodeMap.xml`

Usage: Action Code to Status Code Map Manager

xmlauthmap.filename — The name of the Authorization Destination Map XML file that indicates the authorization destination to which the BASE24-es ATM Device Handler processes send transactions for specified transaction types and institution IDs. The transaction type and institution ID can be wildcarded. Currently, the BASE24-es ATM Device Handler process supports authorization destination values of AUTH_ES and AUTH_CLASSIC, which correspond to conditions in the appropriate Transaction Meta Map file. Conditions for AUTH_ES are defined in the Base DH Transaction Meta Map file while conditions for AUTH_CLASSIC are defined in the Classic DH Transaction Meta Map file. This property should be defined in the Application Configuration data source (APPLD). The Authorization Map Manager reads it at initialization to preload message routing details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for all types of ATM Device Handler processes.

Example Entry: `xmlauthmap.filename=C:/ECommerce/Version041/config/AuthMap.xml`

Usage: Authorization Map Manager

xmlcurrencysymbolmap.filename — The name of the Currency Symbol Map XML file that defines currency symbols for ATM statements and receipts by device type. This allows you to configure multiple currency symbols. This property should be defined in the Application Configuration data source (APPLD). The Currency Symbol Map Manager reads it at initialization to preload currency symbol details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for Diebold 911/912 and NCR NDC+ Device Handler processes. This property is not used by the IFX Manager process.

Example Entry: xmlcurrencysymbolmap.filename=C:/ECommerce/
Version041/config/CurrencySymbolMap.xml

Usage: Currency Symbol Map Manager

xmldispensealgorithmmap.filename — The name of the Dispense Algorithm Map XML file that defines a mapping between a dispense algorithm code and the class that implements the dispense algorithm. Currently, only the least bills method is supported. This property should be defined in the Application Configuration data source (APPLD). The Dispense Algorithm Map Manager reads it at initialization to preload dispense algorithm handling details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for Diebold 911/912 and NCR NDC+ Device Handler processes. This property is not used by the IFX Manager process.

Example Entry: xmldispensealgorithmmap.filename=C:/ECommerce/
Version041/config/DispenseAlgorithmMap.xml

Usage: Dispense Algorithm Map Manager

xmlisomessagemap.filename — The name of the ISO Message Map XML file that defines a list of classes to format individual bits (data elements) in the ISO message sent to the Integrated Server process for custom software modifications (CSMs). If needed for CSMs, this property should be configured in the Application Configuration data source (APPLD). The ISO 93 Message Map Manager reads it at initialization to preload ISO message handling details into memory. The data is held in memory for as long as the application is running.

Required:	No, unless needed for CSMs for any ATM Device Handler process.
Example Entry:	xmlisomessagemap.filename.0=C:/ECommerce/Version041/config/ISOMessageMap.xml
Usage:	ISO 93 Message Map Manager

xmllocalemap.filename — The name of the Locale Map XML file that maps language codes (received in the last byte of the eight-byte operations code buffer) to locale strings for specified terminal types and load profiles. The ATM terminal provides the language code in the eighth byte of the operations code buffer. Locale strings indicate the language in which the host should format text in a response (e.g., for a statement print transaction) and are used to access the appropriate ATM Bundler properties file. This property should be configured in the Application Configuration data source (APPLD). The Locale Map Manager reads it at initialization to preload language handling details into memory. The data is held in memory for as long as the application is running.

Required:	Yes, for Diebold 911/912 and NCR NDC+ Device Handler processes. This property is not used by the IFX Manager process.
Example Entry:	xmllocalemap.filename=C:/ECommerce/Version041/config/LocaleMap.xml
Usage:	Locale Map Manager

xmlmessageclassifiermap.filename — The name of the Message Classifier Map XML file that specifies the mapping between the first digit of the message type of a response message to the formatter controller class for that type of message. This property should be defined in the Application Configuration data source (APPLD). The Message Classifier Map Manager reads this file into memory at initialization. The data is held in memory for as long as the application is running.

Required:	Yes, for all types of ATM Device Handler processes.
Example Entry:	xmlmessageclassifiermap.filename=C:/ECommerce/Version041/config/MessageClassifierMap.xml
Usage:	Message Classifier Map Manager

xmlopcodemap.filename.n — The name of an Operation Code Map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each filename specified. Operation Code Map XML files translate the first five bytes of the eight-character ATM operations code buffer received from the ATM into a

BASE24-es processing code and pre-processing steps. This property should be defined in the Application Configuration data source (APPLD). The Operation Code Map Manager reads it at initialization to preload operations code translation details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for Diebold 911/912 and NCR NDC+ Device Handler processes. This property is not used by the IFX Manager process.

Example Entries: `xmlopcode.map.filename.0=C:/ECommerce/Version041/config/DieboldOpCodeMap.xml`
`xmlopcode.map.filename.1=C:/ECommerce/Version041/config/NCROpCodeMap.xml`

Usage: Operation Code Map Manager

xmlprinttagconversionmap.filename.*n* — The name of a Print Tag Conversion Map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each filename specified. Print Tag Conversion Map XML files define print tags for printer control sequences such as line breaks, form feeds, etc., as well as space and text compression. This property should be defined in the Application Configuration data source (APPLD). The Print Tag Conversion Map Manager reads it at initialization to printer control handling details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for Diebold 911/912 and NCR NDC+ Device Handler processes. This property is not used by the IFX Manager process.

Example Entries: `xmlprinttagconversionmap.filename.0=C:/ECommerce/Version041/config/DieboldPrintTagConversionMap.xml`
`xmlprinttagconversionmap.filename.1=C:/ECommerce/Version041/config/NCRPrintTagConversionMap.xml`

Usage: Print Tag Conversion Map Manager

xmlprivateusedatamap.filename — The name of the Private Use Field Formatter Map XML file that defines tags for private use fields (e.g., data elements P-62 and P-63) in the ISO message sent between the BASE24-es ATM Device Handler process and the Integrated Server process. Any tags defined in a private use data element for CSMs (e.g., S-127) must be defined in this file. This property should be defined in the Application Configuration data source (APPLD). The Private Use Data Map Manager reads it at initialization to preload ISO message handling details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for all ATM Device Handler processes.
 Example Entry: xmlprivateusedatamap.filename.0=C:/ECommerce/
 Version041/config/PrivateUseFieldFormatterMap.xml
 Usage: Private Use Data Map Manager

xmlreasoncodemap.filename — The name of the Reason Code Map XML file that maps IFX reversal reason codes to ISO reason codes. This property should be configured in the Application Configuration data source (APPLD). The Reversal Reason Code Map Manager reads it at initialization to preload reason code mapping details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for IFX Manager process.
 Example Entries: xmlreasoncodemap.filename=/user/fnbo/ESDH041/config/
 IFX/ReversalReasonCodeMap.xml
 Usage: Reversal Reason Code Map Manager

xmlreceiptmap.filename.n — The name of a Receipt Map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each filename specified. Receipt Map XML files define field codes specific to a device type for formatting receipts. Field codes common to all device types are defined in the Receipt Formatter Map XML file. Receipt formatting codes are tokens that represent fields to be printed on a receipt. This property should be configured in the Application Configuration data source (APPLD). The Receipt Map Manager reads it at initialization to preload receipt handling details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for Diebold 911/912 and NCR NDC+ Device Handler processes. This property is not used by the IFX Manager process.
 Example Entries: xmlreceiptmap.filename.0=C:/ECommerce/Version041/
 config/ReceiptFormatterMap.xml
 xmlreceiptmap.filename.1=C:/ECommerce/Version041/
 config/DieboldReceiptMap.xml
 xmlreceiptmap.filename.2=C:/ECommerce/Version041/
 config/NCRReceiptMap.xml
 Usage: Receipt Map Manager

xmlstatemap.filename.n — The name of a State Map XML file, where *n* is a sequential number starting at 0 and increased by 1 for each filename specified. State Map XML files determine the next state and screen to be displayed by the ATM based on the ISO response code returned and specified conditions. This

property should be defined in the Application Configuration data source (APPLD). The State Map Manager reads it at initialization to preload state handling details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for Diebold 911/912 and NCR NDC+ Device Handler processes. This property is not used by the IFX Manager process.

Example Entries: `xmlStateMap.filename.0=C:/ECommerce/Version041/config/DieboldStateMap.xml`
`xmlStateMap.filename.1=C:/ECommerce/Version041/config/NCRStateMap.xml`

Usage: State Map Manager

xmltokenmap.filename — The name of the Token Map XML file that specifies the mapping between token identifiers and the formatters used to parse and build those tokens when sending transactions to BASE24-atm for authorization. If routing messages to BASE24-atm for authorization, this property should be defined in the Application Configuration data source (APPLD). The Token Map Manager reads it at initialization to preload token handling details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for any ATM Device Handler processes if routing to BASE24-atm for authorization.

Example Entry: `xmltokenmap.filename=C:/src/jdh/config/DH-Common/TokenMap.xml`

Usage: Token Map Manager

xmltokenrequiredmap.filename — The name of the Token Required Map XML file that specifies which tokens are required to be added to STM request messages when sending transactions to BASE24-atm for authorization. This file is not used to validate tokens received in messages from BASE24-atm. If routing messages to BASE24-atm for authorization, this property should be defined in the Application Configuration data source (APPLD). The Token Required Map Manager reads it at initialization to preload token handling details into memory. The data is held in memory for as long as the application is running.

Required: Yes, for any ATM Device Handler process if routing to BASE24-atm for authorization.

Example Entry: `xmltokenrequiredmap.filename=C:/src/jdh/config/DH-Common/TokenRequiredMap.xml`

Usage: Token Required Map Manager

Sample Config.properties File

Different sample config.properties files are provided for the BASE24-es ATM Device Handler processes, IFX Manager process, and ATM Timer process as listed below.

Config.properties Files	Usage
ndcconfig.properties	BASE24-es ATM Device Handler process (NCR NDC+ terminals)
tcsconfig.properties	BASE24-es ATM Device Handler process (Diebold 911/912 terminals)
timerconfig.properties	ATM Timer process
IFXconfig.properties	BASE24-es IFX Manager process (IFX terminals)

ACI also provides sample config.properties files that you could use when routing transactions to BASE24-atm and BASE24-es. These sample config.properties files are located in the BASE24-Common and DHBase folders, respectively.

The following is an example of the ndcconfig.properties file used for the BASE24-es ATM Device Handler process and NCR NDC+ terminals. Any of the sample config.properties files you use may need to be edited during installation for site-specific information.

```
db.user.id=dhadmin
db.user.password=2c6bdef3138935745cfad7dbec3f8e21
db.datasource=jdbc:JTurbo://172.16.1.53:1433/DH044Golden/sp=false

#
# db.max.connections defines the maximum number of connections allowed to
# connect to
# the database.
#
db.max.connections=50

#
# db.static.connections defines the number of connections to acquire to the
# database
# when the jvm starts. These connections will stay active through the life of
# the
```

```
# executing JVM.
#
db.static.connections=1

#
# db.connection.wait.time defines the amount of time, in milliseconds, to wait
for
# another thread to free a Database connection in the Database connection
manager.
# This is used when the maximum number of connections have been acquired as
specified
# in the db.max.connections property.
#
db.connection.wait.time=10

#
# db.driver specifies the class name of the JDBC driver to use for all database
connections.
# The whole package name as it is found in the jar file containing the driver
should be specified.
#
db.driver=com.ashna.jturbo.driver.Driver

#
#
# db.type specifies the type of database the application is installed on.
# Valid values are:
#   DB2
#   ORACLE
#   MSSQL
#   SQLMX
#
db.type=MSSQL

#
# LogManager properties.
# The Log4J config file location for trace and event
#
event.log4j.config.url=file:///C:/esdh044/config/EventConfig.xml
trace.log4j.config.url=file:///C:/esdh044/config/TraceConfig.xml

#
# Messaging properties.
#
# Valid values for message.environment are:
#   DIRECT
#   JMS
#   EJB (Not yet fully implemented)
#
message.environment=JMS

#
```

```
# OPENJMS
#
message.jms.jndi.initialcontextfactory=org.exolab.jms.jndi.rmi.
RmiJndiInitialContextFactory
message.jms.jndi.providerurl=rmi://localhost:1099/JndiServer
message.jms.QueueConnectionFactory=JmsQueueConnectionFactory

#
# Initialization Manager Properties.
# For every entry increment the function counter. Each class must implement
the
# InitializationInterface. Message Router service should be last since the
application
# will start acquiring messages as soon as the MessageRouterService is done
initializing.
#
initialization.function.0.classname=com.aciworldwide.framework.
TransactionMapManager
initialization.function.1.classname=com.aciworldwide.common.TransMetaMapManager
initialization.function.2.classname=com.aciworldwide.base24.es.dh.common.config.
OpCodeMapManager
initialization.function.3.classname=com.aciworldwide.base24.es.dh.common.config.
CurrencySymbolMapManager
initialization.function.4.classname=com.aciworldwide.base24.es.dh.common.config.
ReceiptTemplateService
initialization.function.5.classname=com.aciworldwide.base24.es.dh.common.config.
ReceiptRequiredFlagsService
initialization.function.6.classname=com.aciworldwide.base24.es.dh.common.config.
StateMapManager
initialization.function.7.classname=com.aciworldwide.base24.es.dh.common.config.
ReceiptMapManager
initialization.function.8.classname=com.aciworldwide.base24.es.dh.common.config.
LocaleMapManager
initialization.function.9.classname=com.aciworldwide.base24.es.dh.common.config.
DispenseAlgorithmMapManager
initialization.function.10.classname=com.aciworldwide.base24.es.common.config.
ISO93MessageMapManager
initialization.function.11.classname=com.aciworldwide.base24.es.dh.common.
config.PrintTagConversionMapManager
initialization.function.12.classname=com.aciworldwide.base24.es.dh.common.
config.PrivateUseDataMapManager
initialization.function.13.classname=com.aciworldwide.base24.es.dh.common.
config.AuthMapManager
initialization.function.14.classname=com.aciworldwide.base24.es.common.config.
MessageClassifierMapManager
initialization.function.15.classname=com.aciworldwide.common.
MessageRouterService

#
# TranMap files (TransactionMapManager)
#
xmltranmap.filename.0=E:/config/DH-Common/DHCommonTranMap.xml
```

```
xmltranmap.filename.1=E:/config/DH-NCR/NCRDHTranMap.xml
xmltranmap.filename.2=E:/config/DH-Surcharge/BaseDHSurchargeTranMap.xml

#
#      TransMetaMap files (TransMetaMapManager)
#
xmltransmetamap.filename.0=E:/config/DH-Common/DHCommonTransMetaMap.xml
xmltransmetamap.filename.1=E:/config/DH-NCR/NCRDHTransMetaMap.xml
xmltransmetamap.filename.2=E:/config/DH-Surcharge/BaseDHSurchargeTransMetaMap.xml
xmltransmetamap.filename.3=E:/config/DH-NCR/NCRDHSurchargeTransMetaMap.xml

#
#      OpCodeMap files (OpCodeMapManager)
#
xmlopcodemap.filename.0=E:/config/DH-NCR/NCROpCodeMap.xml

#
#      CurrencySymbolMap file (CurrencySymbolMapManager)
#
xmlcurrencysymbolmap.filename=E:/config/DH-Common/CurrencySymbolMap.xml

#
#      StateMap files (StateMapManager)
#
xmlstatemap.filename.0=E:/config/DH-NCR/NCRStateMap.xml

#
#      ReceiptMap files (ReceiptMapManager)
#
xmlreceiptmap.filename.0=E:/config/DH-Common/ReceiptFormatterMap.xml
xmlreceiptmap.filename.1=E:/config/DH-NCR/NCRReceiptMap.xml

#
#      LocaleMap file (LocaleMapManager)
#
xmllocalemap.filename=E:/config/DH-Common/LocaleMap.xml

#
#      DispenseAlgorithmMap file (DispenseAlgorithmMapManager)
#
xmldispensealgorithmmap.filename=E:/config/DH-Common/DispenseAlgorithmMap.xml

#
#      ISO93MessageMap file (ISO93MessageMapManager)
#
xmlisomessagemap.filename.0=E:/config/BASE24-Common/ISOMessageMap.xml

#
#      PrintTagConversionMap files (PrintTagConversionMapManager)
#
```

```
xmlprinttagconversionmap.filename.0=E:/config/DH-NCR/NCRPrintTagConversionMap.xml

#
#      PrivateUseFieldFormatterMap files (PrivateUseDataMapManager)
#
xmlprivateusedatamap.filename.0=E:/config/DH-Common/PrivateUseFieldFormatterMap.xml

#
#      AuthorizationRoutingMap file (AuthMapManager)
#
xmlauthmap.filename=E:/config/DH-Common/AuthMap.xml

#
#      ResponseMessageClassificationMap (MessageClassifierMapManager)
#
xmlmessageclassifiermap.filename=E:/config/BASE24-Common/MessageClassifierMap.xml
```

ACI Worldwide Inc.

Glossary

The following entries describe technical and industry terms used in ACI's Java Server Framework.

caching — A technique whereby records initially retrieved from a data source on disk are maintained in cache memory for all subsequent data access requests until the cache memory is cleared. This technique increases the overall performance of data access.

clustered system — A server with a group of independent processors managed as a single system for higher availability, manageability, and scalability. Each processor has its own operating system and memory subsystem. Applications run in single-threaded mode on clustered systems such as the HP NonStop platform.

coarse-grained object — An object that maps to more than one data source. A fine-grained object maps to a single data source.

daemon — A process that runs in the background and performs a specified operation at predefined times or in response to certain events. The term daemon is a UNIX term, though many other operating systems provide support for daemons, though they're sometimes called other names. Windows, for example, refers to daemons as System Agents and services.

Enterprise JavaBeans (EJBs) — An architecture for setting up program components, written in the Java programming language, that run in the server parts of a computer network that uses the client/server model. Enterprise JavaBeans is built on the JavaBeans technology for distributing program components (which are called Beans, using the coffee metaphor) to clients in a network. Enterprise JavaBeans offers enterprises the advantage of being able to control change at the server rather than having to update each individual computer with a client whenever a new program component is changed or added. EJB components have the advantage of being reusable in multiple applications. To deploy an EJB Bean or component, it must be part of a specific application, which is called a container.

Extensible Markup Language (XML) — Describes a class of data objects called XML documents and partially describes the behavior of computer programs which process them. XML is an application profile or restricted form of the Standard Generalized Markup Language SGML. By construction, XML documents are conforming SGML documents.

Java Message Service (JMS) — A messaging standard that allows application components based on the Java 2 Platform, Enterprise Edition (J2EE) to create, send, receive, and read messages. It enables distributed communication that is loosely coupled, reliable, and asynchronous.

Java Server Page (JSP) — A technology for controlling the content or appearance of web pages through the use of servlets. Sun Microsystems also refers to JSP technology as the Servlet application program interface (API). JSP is comparable to Microsoft's Active Server Page (ASP) technology. Whereas a Java Server Page calls a Java program that is executed by a web server, an Active Server Page contains a script that is interpreted by a script interpreter (such as VBScript or JScript) before the page is sent to the user.

An HTML page that contains a link to a Java servlet is sometimes given the file name suffix of .JSP.

Java Virtual Machine (JVM) — The Java interpreter and runtime environment for Java applications. Each runtime instance of the JVM runs a single Java application and implements all the Java APIs for the platform on which it is deployed. A JVM runtime instance is created when you start a Java application. When you stop a Java application, the runtime instance is destroyed. A JVM instance starts running the single Java application by invoking the main () method of an initial class.

Java DataBase Connectivity (JDBC) — An industry standard database access method for database-independent connectivity between the Java programming language and a wide range of databases. The JDBC API provides a call-level API for SQL-based database access. JDBC technology allows you to use the Java programming language to exploit "Write Once, Run Anywhere" capabilities for applications that require access to enterprise data.

multithreaded — The ability of an application to process multiple transactions simultaneously without having to have multiple copies of the application running in the computer. Each transaction is kept track of as a thread with a separate identity. As applications work on behalf of the initial request for that thread and are

interrupted by other requests, the status of work on behalf of that thread is kept track of until the work is completed. JSF applications can run multithreaded on SMP systems.

Open DataBase Connectivity (ODBC) — A standard database access method developed by Microsoft Corporation. The goal of ODBC is to make it possible to access any data from any application, regardless of which database management system (DBMS) is handling the data. ODBC manages this by inserting a middle layer, called a database driver between an application and the DBMS. The purpose of this layer is to translate the application's data queries into commands that the DBMS understands. For this to work, both the application and the DBMS must be ODBC-compliant—that is, the application must be capable of issuing ODBC commands and the DBMS must be capable of responding to them.

optimistic locking — A technique by which the data to be updated is not locked in advance (i.e., when the data is read). This technique proceeds on the assumption that the data being updated has not changed since the read and does not provide concurrency control. Concurrency control is the mechanism that ensures that the data being written back to the database is consistent with what was read from the database in the first place (i.e., that no other transaction has updated the data after it was read).

pessimistic locking — A technique by which the data to be updated is locked in advance. Once the data to be updated has been locked, the application can make the required changes, and then commit or rollback the update, during which time the lock is automatically dropped. If the application attempts to acquire a lock of the same data during this process, it must wait until the first transaction has completed. This approach is called pessimistic because it assumes that another transaction might change the data between the read and the update. In order to prevent that change from happening, and the data inconsistency that would result, the read statement locks the data to prevent any other transaction from changing it. Pessimistic locking provides concurrency control. Concurrency control is the mechanism that ensures that the data being written back to the database is consistent with what was read from the database in the first place (i.e., that no other transaction has updated the data after it was read).

reflection — Enables Java code to discover information about the fields, methods and constructors of loaded classes, and to use reflected fields, methods, and constructors to operate on their underlying counterparts on objects, within security restrictions. The API accommodates applications that need access to either the public members of a target object (based on its runtime class) or the members declared by a given class.

Secure Sockets Layer (SSL) — A protocol developed by Netscape for transmitting private documents via the Internet. SSL works by using a private key to encrypt data transferred over the SSL connection from a client to a server.

serialized data object — A whole object written to and read from a raw byte stream. Object serialization allows Java objects and primitives to be encoded into a byte stream suitable for streaming to some type of network or to a file-system, or more generally, to a transmission medium or storage facility.

servlet — A small program specified in a web page and run on a web server to modify the web page before it is sent to the user who requested it.

single-threaded — The processing of transactions one-at-a-time by an application. The application must complete processing of a transaction before it starts processing another transaction. JSF applications run single-threaded on clustered systems, such as the HP NonStop platform.

socket — A connection. A secure socket is a connection that uses some type of security, such as Secure Sockets Layer (SSL), to secure the connection.

symmetric multiprocessing (SMP) — The processing of program code by multiple processors that share a common operating system and memory subsystem. In symmetric (or *tightly coupled*) multiprocessing, the processors share memory and the I/O bus. A single copy of the operating system manages all the processors. SMP systems do not usually exceed 16 processors, although some of the latest machines released by several UNIX vendors support up to 64 processors. Applications can run in multithreaded mode on SMP systems, such as Windows NT, UNIX, and IBM zSeries platforms.

XML document — A document made up of storage units called entities, which contain either parsed or unparsed data. Parsed data is made up of characters, some of which form character data, and some of which form markup. Markup encodes a description of the document's storage layout and logical structure. XML provides a mechanism to impose constraints on the storage layout and logical structure.

XML processor — A software module used to read XML documents and provide access to their content and structure.

Index

A

- Adapters, definition of, [1-4](#)
- Alternate Data Source data source (DATASRC)
 - database subsystem, [2-4](#)
 - introduction, [2-17](#)
- Application Configuration data source (APPLD), [1-14](#)
- Application configuration properties, [5-10](#)
- Application layers
 - adapters, [1-4](#)
 - components, [1-4](#)
 - framework, [1-5](#)
 - overview, [1-3](#)
 - services, [1-5](#)
- Application programming interfaces (APIs), [1-6](#)

C

- Cache Administration window, [4-2](#)
- Cache management subsystem, [2-13](#)
- Cache management, description of, [4-2](#)
- Cache Manager component, [1-6](#), [2-13](#)
- Class loading, [1-6](#)
- Clustering subsystem, [2-7](#)
- Components, definition of, [1-4](#)
- config.properties file
 - introduction, [1-14](#)
 - sample for BASE24-es ATM Device Handler processes, [B-25](#)
 - sample for BASE24-es Java User Interface servlets, [A-20](#)
- ConfigPropertyBuilder.class command window utility, [5-7](#)
- Configuration and instrumentation subsystem, [2-14](#)
- Configuration properties, [5-7](#)
- Configuration Properties window, [4-5](#)

D

- Database
 - Alternate Data Source data source (DATASRC), [2-17](#)
 - data sources, [2-17](#)
 - Event data source (EVENTD), [2-17](#)
 - Event Documentation data source (EVDOCD), [2-17](#)
 - External Connection data source (EXTCND), [2-18](#)
 - Listener data source (LSTND), [2-18](#)
 - overview diagram, [2-19](#)
 - Process data source (PROCD), [2-18](#)
 - properties files, [2-14](#)
 - XML documents, [2-16](#)
- Database Administration window, [4-6](#)
- Database subsystem, [2-2](#)
- Directory subsystem, [2-12](#)

E

- Encryption subsystem, [2-8](#)
- Enterprise JavaBeans (EJBs), [1-7](#)
- Event administration, [4-9](#)
- Event data source (EVENTD), [2-17](#)
- Event Documentation data source (EVDOCD), [2-17](#)
- Event message documentation, [4-13](#)
- EventConfig.xml file, [4-10](#)
- External Connection data source (EXTCND), [2-18](#)

F

- Framework layer, [1-5](#)
- Framework subsystems
 - cache management, [2-13](#)
 - clustering, [2-7](#)
 - configuration and instrumentation, [2-14](#)
 - database, [2-2](#)
 - directory, [2-12](#)
 - encryption, [2-8](#)
 - logging, [2-9](#)
 - message, [2-6](#)
 - message transformation, [2-11](#)
 - tracing, [2-10](#)

J

- Java API for XML binding (JAXB), [2-11](#)
- Java API for XML Processing (JAXP), [1-11](#)
- Java Authentication and Authorization Service (JAAS), [1-10](#)
- Java Cryptography Extensions (JCE), [1-5](#), [2-8](#)
- Java DataBase Connectivity (JDBC), [2-2](#)
- Java Message Service (JMS), [2-6](#)
- Java server administration, overview of, [1-16](#)

Java Server Framework (JSF)
 application layers, [1-3](#)
 application programming interfaces (APIs), [1-6](#)
 class loading, [1-6](#)
 database, [2-14](#)
 introduction, [1-2](#)

Java server management, overview of, [1-15](#)

Java servlets, definition of, [1-7](#)

Javadoc, [1-12](#)

JVM runtime arguments, -D option, [1-13](#)

L

Listener data source (LSTND), [2-18](#)

log4j application

 additional references, [4-13](#)

 use in logging subsystem, [1-6](#), [2-9](#)

Logging subsystem, [2-9](#)

M

Message subsystem, [2-6](#)

Message transformation subsystem, [2-11](#)

O

Open DataBase Connectivity (ODBC), [2-2](#)

P

Password encryption utility, [5-7](#)

Process data source (PROCD), [2-18](#)

Properties

 application configuration, [1-14](#)

 atm.dh.save.previous.day.totals, [B-3](#)

 ATMCHANNEL.connectionID.01, [B-2](#)

 ATMCHANNEL.connectionID.02, [B-2](#)

 ATMCHANNEL.connectionID.03, [B-3](#)

 ATMCHNLADMNPTC.connectionID, [B-2](#)

 atmcutover.clear.atd.totals.on.eopp, [B-3](#)

 ATMRELOAD.connectionID, [B-3](#)

 baos.manager.init.count, [5-11](#)

 baos.manager.init.stream.size, [5-11](#)

 baos.manager.max.pool.size, [5-11](#)

 cjsaf.enabled, [A-3](#)

 cjsaf.retry, [A-3](#)

 cjsaf.sleep, [A-3](#)

 cleanup.database.cache.interval.hours, [5-12](#)

 cleanup.dbcache.cleanup.starttime, [5-12](#)

 cleanup.dbcache.timeperiod.hours, [5-13](#)

 cleanup.userauditlog.cleanup.starttime, [5-12](#)

 cleanup.userauditlog.retention.days, [5-11](#)

 cleanup.userauditlog.timeperiod.hours, [5-12](#)

 config.filename, [5-2](#)

 config.home, [A-5](#)

 configuration, [1-13](#)

 context.manager.connection.id, [5-13](#)

 context.retention.period, [5-13](#)

 context.type, [5-13](#)

 contingency.saf.extconn, [A-5](#)

 crypto.jce.aescipher.algname, [5-14](#)

 crypto.jce.aescipher.provider, [5-15](#)

 crypto.jce.cast5cipher.algname, [5-16](#)

 crypto.jce.cast5cipher.provider, [5-16](#)

 crypto.jce.dbcipher.algname, [5-18](#)

 crypto.jce.dbcipher.provider, [5-18](#)

 crypto.jce.descipher.algname, [5-15](#)

 crypto.jce.descipher.provider, [5-15](#)

 crypto.jce.desedecipher.algname, [5-15](#)

 crypto.jce.desedecipher.provider, [5-15](#)

 crypto.jce.hmacsha1.algname, [5-17](#)

 crypto.jce.hmacsha1.provider, [5-17](#)

 crypto.jce.prng.algname, [5-14](#)

 crypto.jce.prng.provider, [5-14](#)

 crypto.jce.rsa.algname, [5-17](#)

 crypto.jce.rsa.provider, [5-18](#)

 crypto.jce.sha1.algname, [5-16](#), [5-17](#)

 crypto.jce.sha1.provider, [5-16](#), [5-17](#)

 crypto.jce.sha1rsasig.algname, [5-18](#)

 crypto.jce.sha1rsasig.provider, [5-18](#)

 crypto.pool.ceiling, [5-19](#)

 crypto.provider, [5-19](#)

 cutover.saf.max.row.count, [B-4](#)

 cutover.saf.msg.retries, [B-4](#)

 cutover.saf.run.delay.minutes, [B-5](#)

 cutover.saf.send.interval, [B-4](#)

 cutover.saf.start.delay.minutes, [B-5](#)

 dataValidation.schema.ATMXMLDataTypes, [B-5](#)

 dataValidation.schema.ESXMLDataTypes, [A-5](#)

 db.alt.datasource.name.database-service, [5-20](#)

 db.alt.datasource.name.db-service-name, [3-2](#)

 db.audit.db-service-name, [5-20](#)

 db.audited, [5-21](#)

 db.audited.db-service-name, [5-21](#), [A-6](#)

 db.broadcast, [5-22](#)

 db.cached.db-service-name, [5-23](#), [A-12](#), [B-5](#)

 db.connection.wait.time, [5-24](#)

 db.datasource, [4-6](#), [5-24](#)

 db.driver, [4-6](#), [5-25](#)

 db.max.connections, [4-6](#), [5-25](#)

 db.sqlmp, [A-12](#)

 db.static.connections, [4-6](#), [5-26](#)

 db.table.prefix, [5-26](#)

 db.type, [5-26](#)

 db.upshiftdata, [A-12](#)

 db.useisolationlevel, [5-26](#)

 db.user.id, [4-6](#), [5-27](#)

 db.user.password, [5-27](#)

 devicehandler.atmlog.cleanup.retention.days, [B-7](#)

 devicehandler.atmlog.cleanup.starttime, [B-7](#)

 devicehandler.atmlog.cleanup.timeperiod.hours, [B-7](#)

 devicehandler.balance.enquiry.reversal, [B-7](#)

 devicehandler.dkm.key.change.interval, [B-8](#)

 devicehandler.dkm.mac.error.limit, [B-8](#)

 devicehandler.dkm.mac.transaction.limit, [B-8](#)

 devicehandler.dkm.pin.error.limit, [B-9](#)

 devicehandler.dkm.pin.transaction.limit, [B-9](#)

 devicehandler.max.rowcount.inactivity, [B-10](#)

 devicehandler.max.rowcount.load.inactivity, [B-10](#)

Properties *continued*

- devicehandler.maximum.statement.data.chunk.size, B-9
- devicehandler.message.logging, B-10
- devicehandler.phonecard.amount.value.currency-code.n, B-11
- devicehandler.phonecard.minutes.value.n, B-12
- devicehandler.run.delay.minutes.inactivity, B-10
- devicehandler.run.delay.minutes.load.inactivity, B-10
- devicehandler.start.delay.minutes.inactivity, B-11
- devicehandler.start.delay.minutes.load.inactivity, B-11
- devicehandler.tss.external.connection, B-11
- diebold.override.operator.id.access, B-12
- elmtmap.jar.aci.atm-elmtmaps, B-13
- elmtmap.jar.aci.es-elmtmaps, A-13
- event.config, 4-9
- event.log4j.config.url, 4-9, 5-27
- event.sockets.encoding, 5-27
- event.sockets.host, 5-28
- event.sockets.port, 5-28
- extract.userauditlog.interval.days, A-13
- extract.userauditlog.starttime, A-13
- file.encoding, 5-2
- file.separator, 5-2
- filter.field.task-id.n, A-14
- ifx.formatter.ifx.ns.is.default.ns, B-13
- ifx.message.support.filename.n, B-13
- ifx.status.code.map.n, B-13
- initialization.function.n.classname, 5-28, A-15, B-14
- initialization.function.n.methodname, 5-29
- jaxb.context.0, B-16
- jdal.trace, 4-15, 5-5
- jdal.trace.method, 4-15, 5-6
- keymanager.provider, 5-29
- keystore.filename, 5-29
- keystore.hardware, 5-30
- keystore.password, 5-30
- keystore.provider, 5-30
- keystore.type, 5-31
- mail.smtp.auth.password, 5-32
- mail.smtp.auth.required, 5-32
- mail.smtp.auth.user, 5-32
- mail.smtp.from.addr, 5-31
- mail.smtp.host, 5-31
- mail.smtp.max.retries, 5-32
- message.environment, 5-33
- message.jms.Connection.DisableOnStartup, 5-33
- message.jms.jndi.initialcontextfactory, 5-34
- message.jms.jndi.providerurl, 5-35
- message.jms.queue.externalconnectionid, 5-33
- message.jms.QueueConnectionFactory, 5-34
- message.jms.queuesFromJNDI, 5-34
- message.jms.TopicConnectionFactory, B-16
- message.jms.use.nsk.threads, 5-35
- message.property.handling, 5-36
- ncr.mac.field.select.data.classname, B-16
- process.id, 5-2
- ptc.connectionID, A-15
- ptcf.connectionID, A-16
- report.userauditlog.loc, A-16
- reversal.saf.max.row.count, B-17
- reversal.saf.msg.retries, B-17
- reversal.saf.run.delay.minutes, B-17
- reversal.saf.send.interval, B-17
- reversal.saf.start.delay.minutes, B-17
- root.config, A-4
- sax2.parser.init.count, 5-36
- SCH, A-4
- SSL.client.keystore, 5-36
- SSL.client.keystore.password, 5-37
- SSL.client.truststore, 5-37
- SSL.client.truststore.password, 5-37
- SSL.debug, 5-38
- SSL.protocol.handler.pkgs, 5-38
- SSL.server.keystore, 5-38
- SSL.server.keystore.password, 5-39
- SSL.server.truststore, 5-39
- SSL.server.truststore.password, 5-39
- statement.cleanup.timeperiod.hours, B-18
- statementdata.cleanup.retention.days, B-18
- statementdata.cleanup.starttime, B-18
- system, 1-13
- timertask.function.n.classname, 5-40, A-17, B-18
- timertask.manager.processid, 5-40, A-17
- trace.config, 4-16
- trace.dataobject.enabled, 4-14, A-17
- trace.enabled, 4-14, 5-40
- trace.log4j.config.url, 4-16, 5-41
- tranmap.jar.aci.tranmap-name, A-18
- usec.host, A-18
- usec.port, A-18
- website.accountformat, 5-41
- website.character.encoding, 5-41
- website.datestyle, 5-42
- website.defaultlanguage, 5-42
- website.defaultlocale, 5-42
- website.domain, 5-43
- website.inactivetimeout, 5-43
- website.phoneformat, 5-41
- website.secureSSL, 5-44
- website.timestyle, 5-44
- website.xml.menu.map.filename.n, 5-44
- xml.builder.use.indent, 5-45
- xml.msgschema.jar.aci.schema-name, A-18
- xml.actioncodestatuscodemap, B-19
- xmlauthmap.filename, B-19
- xmlcurrencysymbolmap.filename, B-20
- xmldispensealgorithmmap.filename, B-20
- xmlisomessagemap.filename, B-20
- xmllocalemap.filename, B-21
- xmlmessageclassifiermap.filename, B-21
- xmlopcode.map.filename.n, B-21
- xmlprinttagconversionmap.filename.n, B-22
- xmlprivateusedatamap.filename, B-22
- xmlreasoncodemap.filename, B-23
- xmlreceiptmap.filename.n, B-23
- xmlstatemap.filename.n, B-23
- xmltokenmap.filename, B-24
- xmltokenrequiredmap.filename, B-24
- xmltranmap.filename.n, 5-45
- xmltransmetamap.filename.n, 5-46

Properties *continued*

- xmltransmetamap.trace.paths, [5-47](#)
- xmltransmetamap.trace.paths.errorconditions, [5-47](#)
- xmltransmetamap.trace.paths.filename, [5-47](#)

Properties files, introduction to, [2-14](#)

Protocol options

- CLIENTAUTH, [3-15](#)
- EOT, [3-9](#)
- HDR2, [3-9](#), [3-15](#)
- introduction, [3-9](#), [3-14](#)
- MDSHDR, [3-10](#)
- NONMQJMSTARGET, [3-10](#)
- NOREPLY, [3-9](#), [3-15](#)
- R2LF, [3-9](#)
- R2STR, [3-15](#)
- SSL, [3-10](#), [3-15](#)

S

Services, [1-5](#)

System Monitoring window, [4-8](#)

System Properties window, [4-5](#)

System-level properties, [5-2](#)

T

TCP/IP Socket Administration window, [4-3](#)

Third-party JARs

- BASE24-es, [A-2](#)
- Bouncy Castle, [1-9](#)
- commons-lang, [1-11](#)
- commons-logging, [1-11](#)
- Java Authentication and Authorization Service (JAAS), [1-10](#)
- Java Cryptography Extension (JCE), [1-8](#)
- Java Document Object Model (JDOM), [1-10](#)
- Java Message Service (JMS), [1-7](#)
- local_policy for Java Cryptography Extension (JCE), [1-8](#)
- log4j, [1-9](#)
- relaxngDatatype, [A-2](#)
- sunjce_provider for JCE, [1-8](#)
- US_export_policy for JCE, [1-8](#)
- xercesImpl, [1-11](#)
- xmlParserAPIs, [1-11](#)
- xsdlib, [A-2](#)

Trace administration

- JDAL tracing, [4-15](#)
- message tracing, [4-14](#)

TraceConfig.xml file, [4-17](#)

Tracing subsystem, [2-10](#)

X

XML documents, [2-16](#)

XML message subsystem, [2-11](#)